Gateway-Based Post-Quantum Modernization of a Legacy Edge–Cloud System: Security Operations and Residual Risks

Group 20

Institution: University of Oulu

Course: Software Development, Maintenance & Operations

Repository: github.com/Aliz-f/ml-kem-legacy-modernization

Members

Alireza Fadaei, Roope Paananen, Armin Shalchian, Masoud Abrishami, Dorsa Abdolmaleki

Contents

Chapter 1 — Executive Summary and System Overview

1.1 Project purpose

1.2 System at a glance

1.3 Components, responsibilities, and boundaries

1.4 Modernization modes

1.5 End-to-end processing and reliability model

1.6 Engineering effort and evidence trail

1.7 Scope, assumptions, and acknowledged limitations

1.8 Guide to the remaining technical documentation

Chapter 2 — Architecture and Protocol Specification

2.1 Protocol scope and design rules

2.2 Legacy Device to Edge Gateway contract

2.2.1 Endpoint and payload

2.2.2 Normalization and device sequence policy

2.3 Explicit Gateway-to-Cloud modes

2.3.1 Baseline Mode sequence with controlled fault injection

2.3.2 Modernized ML-KEM Mode sequence with controlled fault injection

2.4 Cryptographic suite and key-establishment contract

2.5 Session lifecycle

2.6 Secure-envelope contract

2.7 Correlation, retries, and session recovery

2.8 Response, error, and storage contracts

2.9 Endpoint summary and security boundaries

Chapter 3 — Component Design and Implementation Evidence

3.1 Implementation approach and ownership

3.2 Legacy Device implementation

3.3 Edge Gateway implementation

3.3.1 Legacy ingress and application pipeline

3.3.2 Classical and secure dispatch

3.4 Cloud Service implementation

3.4.1 Application lifecycle and API composition

3.4.2 Secure ingestion: validate, authenticate, validate again, then store

3.4.3 SQLite storage and idempotency

3.5 Cross-component engineering decisions visible in the code

Chapter 4 — Operations, Deployment and CI/CD

4.1 Operational design goals

4.2 Docker Compose topology and lifecycle

4.3 Health and readiness contracts

4.4 Structured logging, correlation, and secret safety

4.5 Metrics and local monitoring

4.5.1 Gateway metric inventory

4.5.2 Cloud metric inventory

4.6 Continuous integration and delivery gates

4.7 Operational boundaries and handover procedure

Chapter 5 — Verification, Testing and Empirical Evaluation

5.1 Verification Strategy and Testing Pyramid

5.1.1 Multi-Layered Testing Architecture

5.1.2 Repository-Wide Test Scale and Scope

5.1.3 Test Isolation, Determinism and Secret Safety Principles

5.2 Comprehensive Test and Failure Matrices

5.2.1 Repository Test Coverage Matrix

5.2.2 End-to-End Failure Scenario Suite (F1–F18)

5.2.3 Secure Path Failure Invariants and Bounded Retry Rules

5.3 Deep-Dive Security and Failure Regressions

5.3.1 Cryptographic Tamper Resistance and AEAD Verification

Verification Code Trace: Bit-Flipped Ciphertext (Ciphertext Tamper Regression Suite)

Verification Code Trace: Tampered Associated Data (AEAD Associated Data Tamper Suite)

5.3.2 Strict Fail-Closed Policy Enforcement

5.3.3 Business Idempotency vs. Cryptographic Anti-Replay

5.3.4 Secret-Safety Audit and Automated Regression

5.4 Empirical Performance Evaluation: Baseline vs. Modernized ML-KEM-768

5.4.1 Matched Evaluation Methodology

Benchmark Configuration and Parameters

5.4.2 Empirical Comparative Measurement Results

5.4.3 In-Depth Latency and Bandwidth Analysis

1. Payload Size Overhead (+142.5%)

2. Amortized Session Handshake Cost

5.4.4 Multi-Run Statistical Verification (Frozen Evidence Dataset)

5.5 Observability and Operational Readiness Verification

5.5.1 Prometheus-Compatible Metrics Verification

5.5.2 Health and Readiness Operational Probe Contracts

5.5.3 Containerized Lifecycle and Compose Recovery Regression

5.6 Summary of Verification Findings

Chapter 6 — AI-Assisted Development and Critical Evaluation

6.1 AI Governance Framework and Human-in-the-Loop Methodology

6.1.1 Engineering AI Governance Policy

6.1.2 The Four-Phase Task Execution Lifecycle

6.1.3 The `ai-history` Provenance System

Key Technical Capabilities of `ai-history`:

6.2 Quantitative AI Inventory and Project-Wide Metrics

6.2.1 Multi-Agent Model Distribution

6.2.2 Human Intervention and Review Outcomes

Breakdown of Multi-Round Iterative Tasks:

6.3 Project-Wide Architectural Decision Register (AD-1 to AD-7)

6.4 In-Depth Case Studies of LLM Proposals and Critical Human Oversight

6.4.1 Case Study 1: Cryptographic Primitive Misuse — Direct ML-KEM Payload Encryption

Initial LLM Proposal

Human Review and Critical Analysis

Human Review Decision: REJECTED (Pre-Implementation Planning Proposal)

Engineering Resolution and Corrective Action

6.4.2 Case Study 2: Silent Security Downgrade — Automatic Plaintext Fallback

Initial LLM Proposal

Human Review and Critical Analysis

Human Review Decision: REJECTED (Pre-Implementation Planning Proposal)

Engineering Resolution and Corrective Action

6.4.3 Case Study 3: Protocol Contract Violation — Re-encryption on Transport Retry

Initial LLM Proposal

Human Review and Critical Analysis

Human Review Decision: MODIFIED (Implementation Task 0098)

Engineering Resolution and Corrective Action

6.4.4 Case Study 4: Sensitive Credential Leakage in Diagnostic Logging

Initial LLM Proposal

Human Review and Critical Analysis

Human Review Decision: REJECTED (Pre-Implementation Planning Proposal)

Engineering Resolution and Corrective Action

6.4.5 Case Study 5: Multi-Round Iterative Refinement — Failure Simulation and PRNG Isolation

Round-by-Round Evolution:

6.5 Critical Evaluation of LLM Capabilities and Engineering Lessons

6.5.1 Where LLMs Demonstrated High Capability

6.5.2 Where LLMs Systematically Failed

6.5.3 Foundational Engineering Conclusion

Chapter 7 — Technical Debt and Risk Analysis

7.1 Architectural Boundaries and Analytical Methodology

7.1.1 Distinction Between Technical Debt and Operational Risk

7.1.2 Qualitative Evaluation Framework

7.2 Comprehensive Technical Debt Register (TD-1 through TD-12)

7.2.1 Deep Architectural Analysis: TD-10 vs. TD-11

7.3 Comprehensive Risk Register (R-1 through R-9)

7.3.1 Risk Evaluation Heatmap Matrix

7.4 Production Modernization Roadmap

7.4.1 Phase 1: Cryptographic and Transport Hardening

7.4.2 Phase 2: Persistence and Queue Decoupling

7.4.3 Phase 3: Hardware-Backed Key Isolation and Secure Zeroization

Chapter 8 — Appendices (Configurations, Runbooks and Key Source Code)

Appendix A: Complete Deployment Configurations

A.1 Docker Compose Multi-Container Deployment Specification

A.2 Multi-Stage Container Image Specifications

A.2.1 Legacy Device Container Image

A.2.2 Edge Gateway Container Image

A.2.3 Cloud Service Container Image

A.3 Environment Variable Configuration Template

Appendix B: Complete CI/CD Pipeline Definitions

B.1 Repository Quality and Multi-Stage CI Pipeline Workflow

Appendix C: System Operational Runbooks

C.1 Operational Lifecycle and Verification Commands

Appendix D: Core Cryptographic and Protocol Source Code

D.1 Edge Gateway ML-KEM-768 Encapsulation Module

D.2 HKDF-SHA-256 Session Key Derivation Module

D.3 Edge Gateway AES-256-GCM Authenticated Encryption Module

D.4 Canonical 4-Field Associated Data Builder

D.5 Edge Gateway Monotonic Sequence Tracker

D.6 Cloud Service ML-KEM-768 Decapsulation Module

D.7 Cloud Service AEAD Authenticated Decryption Module

Appendix E: Empirical Evaluation and Benchmarking Infrastructure

E.1 Matched Mode Comparison Benchmarking Harness

Appendix F: Representative AI Prompt Catalog

F.1 Governance, Tooling, and Architecture Infrastructure

Task 0001: AI History and Human Review Infrastructure

Task 0002: Workflow Handbook and Dependency Management Policy

F.2 Legacy IoT Device Subsystem (Member 1)

Task 0008: Legacy Configuration Validation

Task 0012: Legacy Canonical Measurement Schema

Task 0027: Monotonic Sensor Simulation Engine

Task 0034: Sensor Failure Injection Subsystem

Task 0052: Legacy Gateway HTTP Client and Error Domain Mapping

F.3 Edge Gateway Application and Sequence Logic (Member 2)

Task 0026: Gateway Configuration and Environment Parsing

Task 0038: Legacy Measurement Ingestion Route and Validation

Task 0048: In-Memory Monotonic Sequence Tracking Policy

F.4 Edge Gateway Post-Quantum Cryptography and Transport (Member 3)

Task 0006: ML-KEM-768 and HKDF-SHA-256 Cryptographic Wrappers

Task 0011: Gateway In-Memory Secure Session Establishment

Task 0019: AES-256-GCM Encryption and 4-Field Associated Data Builder

Task 0098: Secure Transport Retries and Session Recovery

F.5 Cloud Service Telemetry Ingestion and Storage (Member 4)

Task 0018: SQLite Storage Layer and Business Idempotency

Task 0041: Cloud ML-KEM Key Lifecycle and Public-Key Route

Task 0072: Cloud Secure Envelope Decryption and Canonical Ingestion

F.6 Operations, Integration, and Performance Evaluation (Member 5)

Task 0080: Gateway-to-Cloud Cross-Service Integration Tests

Task 0185: Complete Three-Tier Docker Compose Topology

Task 0211: Matched-Mode Empirical Evaluation Harness

Task 0219: Protocol Security Mode Sequence Diagrams

Chapter 1 — Executive Summary and System Overview

1.1 Project purpose

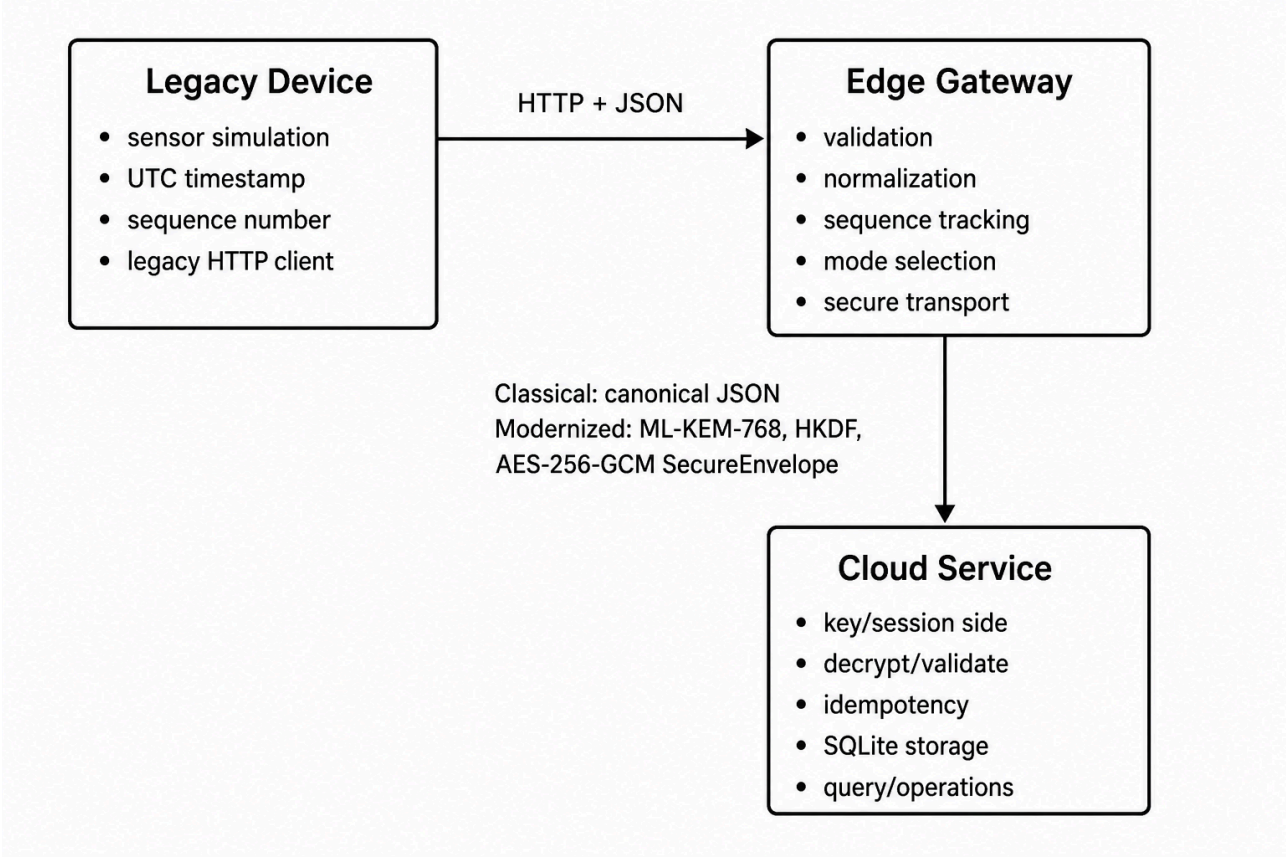
This project modernizes a small environmental-telemetry pipeline without requiring a redesign of its simulated legacy sensor. The system carries temperature and humidity measurements from a Legacy Device, through an Edge Gateway, to a Cloud Service for durable storage. The modernization addresses the risk that data protected today with quantum-vulnerable public-key cryptography could be captured now and decrypted later. Its scope is therefore deliberately narrow: it introduces post-quantum key establishment only on the Gateway-to-Cloud path, where the legacy protocol can be translated and secured without changing the device-facing interface.

The project was also an exercise in safe AI-assisted software development. The course requires an LLM to be used as an assistant, not as an authority. The team therefore treated generated code and design proposals as untrusted until they had been checked against frozen architecture, reviewed by humans, and validated with static analysis, automated tests, integration tests, and operational evidence. The complete prompts, outcomes, review decisions, and verification records are maintained in the repository's AI-history register and are presented in Chapter 6.

The result is a course-sized, reproducible modernization demonstrator rather than a production IoT platform. It provides a baseline mode for comparison, a modernized ML-KEM mode, containerized deployment, observability, controlled failure injection, and evidence-oriented testing. It intentionally does not add unrelated platform services such as a message broker, distributed database, Kubernetes, user accounts, or device-fleet management.

1.2 System at a glance

Figure 1.1 — System at a glance



The architecture has two intentionally different communication boundaries:

1. **Legacy Device → Edge Gateway.** The device sends a five-field measurement through `POST /api/v1/measurements` as ordinary HTTP/JSON. It remains compatible with the legacy-side representation and contains no ML-KEM or Cloud-specific behavior.
2. **Edge Gateway ↔ Cloud Service.** The Gateway translates validated data to a canonical representation and selects either the explicit baseline transport or the modernized secure transport. The Cloud is never contacted directly by the Legacy Device.

This separation is the core migration strategy: the Gateway is both the protocol adapter and the modernization boundary. It lets the project add post-quantum protection where it is feasible while preserving the device-side interface that a real deployed legacy device could be unable to change.

1.3 Components, responsibilities, and boundaries

Component	Primary responsibilities	Explicit boundary
Legacy Device	Simulate temperature and humidity readings; assign a stable device ID, UTC timestamp, and monotonically increasing sequence number; serialize and send a legacy measurement; apply bounded device-side retries and controlled faults.	Does not implement ML-KEM, session management, Cloud communication, encryption, or storage.
Edge Gateway	Receive the legacy API request; validate and normalize it; enforce per-device sequence policy; create a correlation/request ID; deliver in baseline or secure mode; manage Gateway-side sessions, retries, logging, and metrics.	Returns `202 Accepted` only after Cloud acceptance. It does not own durable measurement storage or change the meaning of validated measurements.
Cloud Service	Expose baseline and secure Gateway APIs; own the ML-KEM key pair and session lifecycle; authenticate/decrypt secure messages; revalidate canonical data; enforce idempotency; persist and query measurements; expose operational endpoints.	Does not accept the Legacy Device wire format and does not communicate directly with devices.

Table 1.1 — Components, responsibilities, and boundaries

The canonical boundary also preserves clear ownership. The Legacy component owns sensing and its client; the Gateway owns legacy-facing application behavior, normalization, sequencing, baseline delivery, and secure transport; the Cloud owns cryptographic receiving behavior, validation, idempotency, and SQLite storage. Repository-wide work owns Docker, CI/CD, end-to-end testing, and metrics integration. This division prevented the modernized protocol from leaking into the legacy simulator and made each component independently testable.

1.4 Modernization modes

The system implements two deliberately separate operating modes. Mode choice is configuration, not an automatic fallback decision.

Concern	Classical baseline	Modernized ML-KEM
Gateway setting	`SECURITY_MODE=classical`	`SECURITY_MODE=mlkem`
Cloud delivery	`POST /api/v1/baseline/measurements`	Public-key retrieval, session establishment, then `POST /api/v1/secure/measurements`
Data protection on Gateway–Cloud path	Canonical JSON over the baseline test path	ML-KEM-768 establishes a session secret; HKDF-SHA-256 derives one 32-byte key; AES-256-GCM protects each canonical payload
Correlation	`X-Request-ID` header	`request_id` is inside authenticated associated data (AAD) and the secure envelope

Concern	Classical baseline	Modernized ML-KEM
Failure policy	Bounded delivery retry and visible failure	`PQC_FAILURE_POLICY=fail_closed`; no runtime plaintext downgrade

Table 1.2 — Modernization modes

The baseline is retained as an observable comparison point, not as a silent fallback. In modernized mode, a cryptographic or session-establishment problem causes the secure operation to fail closed. That design makes the security state explicit and enables empirical comparison of the baseline and modernized paths in Chapter 5.

At a high level, secure mode performs the following sequence:

Code Listing 1.1 — Modernization modes (plaintext)

```
1. Gateway obtains the Cloud ML-KEM-768 public key.
2. Gateway encapsulates a shared secret and asks Cloud to create a session.
3. Both parties derive the same 32-byte session key using the frozen HKDF context.
4. Gateway encrypts a canonical measurement with AES-256-GCM.
5. Cloud reconstructs the authenticated metadata, decrypts, validates, and stores it.
6. Gateway returns success to the Legacy Device only after the Cloud accepts it.
```

Protocol v1 fixes the security-critical details: a 300-second process-local session lifetime; a 12-byte AES-GCM nonce; Base64URL for binary JSON values; and a six-field `SecureEnvelope`. Its AAD contains exactly the four visible metadata fields `protocol_version`, `gateway_id`, `session_id`, and `request_id`, serialized as compact, sorted UTF-8 JSON. The full wire contracts, KDF parameters, session recovery rules, and negative cases are specified in Chapter 2; they are not redefined by this overview.

1.5 End-to-end processing and reliability model

A normal measurement follows this path:

Code Listing 1.2 — End-to-end processing and reliability model (plaintext)

```
Sensor simulation
  → legacy JSON measurement
  → Gateway schema/domain validation
  → canonical measurement + sequence-policy decision
  → baseline delivery OR secure session/envelope delivery
  → Cloud validation and idempotent SQLite persistence
  → Gateway 202 response to the Legacy Device
```

The Gateway distinguishes malformed syntax (`400 Bad Request`) from parsed JSON that is missing required fields, has wrong types, or violates a domain constraint (`422 Unprocessable Entity`). Duplicate and out-of-order device sequence numbers produce `409 Conflict`; gaps are accepted and recorded as an operational signal. Gateway sequence state is updated after validation and sequence-policy acceptance, before Cloud delivery, and is not rolled back if Cloud delivery fails. These choices make the synchronous delivery behavior deterministic and visible during failure testing.

Retries are deliberately bounded. A normal Gateway-to-Cloud transport retry resends the exact already-built secure envelope: the request ID, session ID, nonce, ciphertext, and AAD remain unchanged. Only `409 SESSION_INVALID` or `409 SESSION_EXPIRED` initiates one session-recovery attempt: the old session is invalidated, one replacement session is created, a new envelope is built under its new key, and the logical request ID is preserved. This distinction avoids accidental re-encryption during an ordinary retry while permitting recovery from expired or invalid session state.

Cloud persistence uses the business key `UNIQUE(device_id, sequence_number)`. It prevents duplicate logical measurement rows, including retransmission outcomes, but is explicitly not cryptographic anti-replay protection. Protocol v1 intentionally has no dedicated replay counter or replay window; that limitation, along with the in-memory session and sequence state, is documented as technical debt and risk rather than hidden by the overview.

1.6 Engineering effort and evidence trail

The implementation followed a staged modernization path rather than attempting to build a PQC system all at once:

Code Listing 1.3 — Engineering effort and evidence trail (plaintext)

```
Frozen architecture and contracts
  → shared repository workflow and baseline components
  → baseline end-to-end delivery and measurements
  → ML-KEM, session, and authenticated-encryption integration
  → secure end-to-end and failure/security regression testing
  → observability, Docker Compose, CI/CD, evaluation, and documentation
```

This order matters. A functioning classical baseline makes the added cost and behavior of session establishment and authenticated encryption measurable. Contract tests, component tests, integration tests, live system tests, failure injection, static analysis, and reproducible comparison scripts were used to check different layers of the system. Docker Compose runs the three-service test topology; CI validates the locked environment, quality checks, tests, history integrity, image builds, and system-level behavior. The operational, verification, and empirical evidence is detailed in Chapters 4 and 5.

AI assistance was governed through a task-by-task history workflow: a task was recorded, the proposed work was inspected against the architecture, actual checks were recorded, and human review and final verification were required. The project does not treat a passing test suite as a substitute for human judgment. Chapter 6 consolidates the prompt and decision evidence; Chapters 7 and 8 make the remaining limitations and key executable artifacts visible to evaluators who do not inspect the source repository.

1.7 Scope, assumptions, and acknowledged limitations

The design is intentionally constrained to a controlled test environment. The Legacy-to-Gateway HTTP/JSON boundary is not post-quantum protected; the modernization applies only after the Gateway. The Cloud public key is trusted at the explicitly configured endpoint in this environment. Cloud ML-KEM key material, Gateway session state, and Gateway sequence state are process-local, so restarts remove that transient state. Storage is single-instance SQLite; the transport is HTTP-only; and the design provides neither a persistent queue nor a distributed replay-control mechanism.

These are not claims of production readiness. They are conscious scope decisions that kept the work aligned with the course objective: safely integrate a maintained ML-KEM implementation into a legacy system, demonstrate the result, critically assess it, and document residual technical debt and risk.

1.8 Guide to the remaining technical documentation

Chapter	What it contributes
2. Architecture and Protocol Specification	Frozen endpoint, schema, cryptographic, session, retry, error, and storage contracts.
3. Component Design and Implementation Evidence	Component-level design, selected source listings, and ownership evidence.
4. Operations, Deployment and CI/CD	Compose topology, health/readiness, logging, metrics, dashboard, and CI gates.
5. Verification, Testing and Empirical Evaluation	Test matrix, fault/security evidence, and baseline-versus-modernized measurements.
6. AI-Assisted Development and Critical Evaluation	Prompt records, human decisions, fixes, and verification evidence.
7. Technical Debt and Risk Analysis	Accepted limitations, risks, controls, and residual treatment.
8. Appendices	Complete configurations, runbooks, and selected critical source listings.

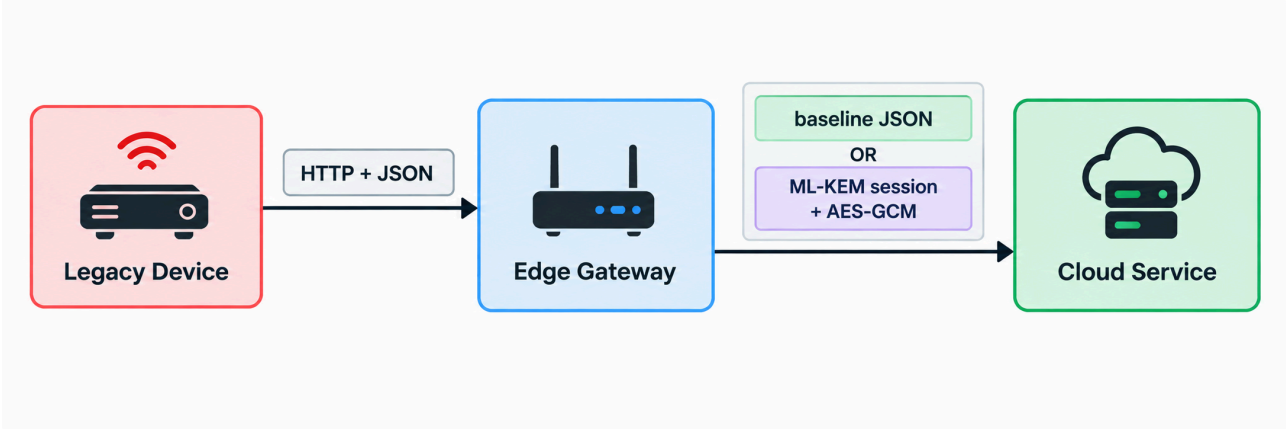
Table 1.3 — Guide to the remaining technical documentation

Chapter 2 — Architecture and Protocol Specification

2.1 Protocol scope and design rules

The system uses two distinct protocol boundaries, which must never be confused:

Figure 2.1 — Protocol scope and design rules



The Legacy Device sends a simple, fixed JSON representation. The Gateway is the translation and modernization boundary: it validates legacy input, normalizes it, enforces device sequence policy, and selects a Cloud transport. The Cloud accepts only the Gateway canonical representation on its baseline path or its decrypted canonical representation on its secure path. It never accepts the Legacy Device wire format.

The protocol design follows these invariants:

- APIs and secure messages are explicitly versioned; protocol version is always the string `"1"`.
- Application timestamps are ISO-8601 UTC values and JSON field names are `snake_case`.
- Every incoming representation is validated; secrets are never exposed in API responses, logs, metrics, persistence, or AI-history records.
- Baseline and secure modes use different endpoints. Secure-mode failure never silently downgrades to plaintext.
- Retries are bounded and correlation is preserved through a Gateway-generated request ID.
- Cryptographic primitives come from maintained libraries; none is implemented in project code.

2.2 Legacy Device to Edge Gateway contract

2.2.1 Endpoint and payload

The Legacy Device sends one measurement to the Gateway:

Code Listing 2.1 — Endpoint and payload (http)

```
POST /api/v1/measurements
Content-Type: application/json
```

Code Listing 2.2 — Endpoint and payload (json)

```
{
  "device_id": "sensor-001",
  "temperature_c": 21.7,
  "humidity_pct": 42.5,
  "timestamp": "2026-09-21T13:45:31Z",
  "sequence_number": 184
}
```

The object contains exactly these five fields; unknown fields are rejected.

Field	Type	Constraint
`device_id`	string	non-empty
`temperature_c`	number	`-40 <= value <= 85`
`humidity_pct`	number	`0 <= value <= 100`
`timestamp`	string	ISO-8601 UTC
`sequence_number`	integer	`0 <= value <= 9223372036854775807`

Table 2.1 — Endpoint and payload

The Gateway returns `202 Accepted` with `{ "status": "accepted" }` only after the Cloud has accepted the measurement for persistence. Error semantics are intentionally precise: malformed or unparseable request syntax is `400`; parsed JSON with missing, wrongly typed, extra, or domain-invalid fields is `422`; and a duplicate or out-of-order sequence is `409`.

2.2.2 Normalization and device sequence policy

After accepting the legacy form, the Gateway creates the canonical measurement:

Code Listing 2.3 — Normalization and device sequence policy (json)

```
{
  "device_id": "sensor-001",
  "temperature_c": 21.7,
  "humidity_pct": 42.5,
  "measured_at": "2026-09-21T13:45:31Z",
  "sequence_number": 184,
  "received_at": "2026-09-21T13:45:31.218Z",
  "gateway_id": "gateway-001"
}
```

`timestamp` becomes `measured_at`; the Gateway adds `received_at` and `gateway_id`. The request/correlation ID is deliberately not part of this business object.

Sequence state is held independently for each `device_id` and records the last valid sequence observed by the Gateway, not the last value stored by Cloud.

Incoming sequence	Gateway action
First valid value	accept
`last + 1`	accept
Greater than `last + 1`	accept and record a sequence gap
Equal to `last`	reject as duplicate, `409`
Less than `last`	reject as out of order, `409`

Table 2.2 — Normalization and device sequence policy

The Gateway commits this state after validation and sequence-policy acceptance, but before Cloud delivery. A later Cloud failure therefore does not roll back an accepted device sequence. This simple in-memory state is intentionally not restart durable and is documented as technical debt.

2.3 Explicit Gateway-to-Cloud modes

Mode selection is configuration, not negotiation.

Property	Classical baseline	Modernized ML-KEM
Setting	`SECURITY_MODE=classical`	`SECURITY_MODE=mlkem`
Delivery endpoint	`POST /api/v1/baseline/measurements`	`POST /api/v1/secure/measurements` after session establishment
Body	plain `CanonicalMeasurement`	versioned `SecureEnvelope`
Correlation	`X-Request-ID` header	authenticated `request_id` in the envelope AAD
Secure failure policy	not applicable	`PQC_FAILURE_POLICY=fail_closed`

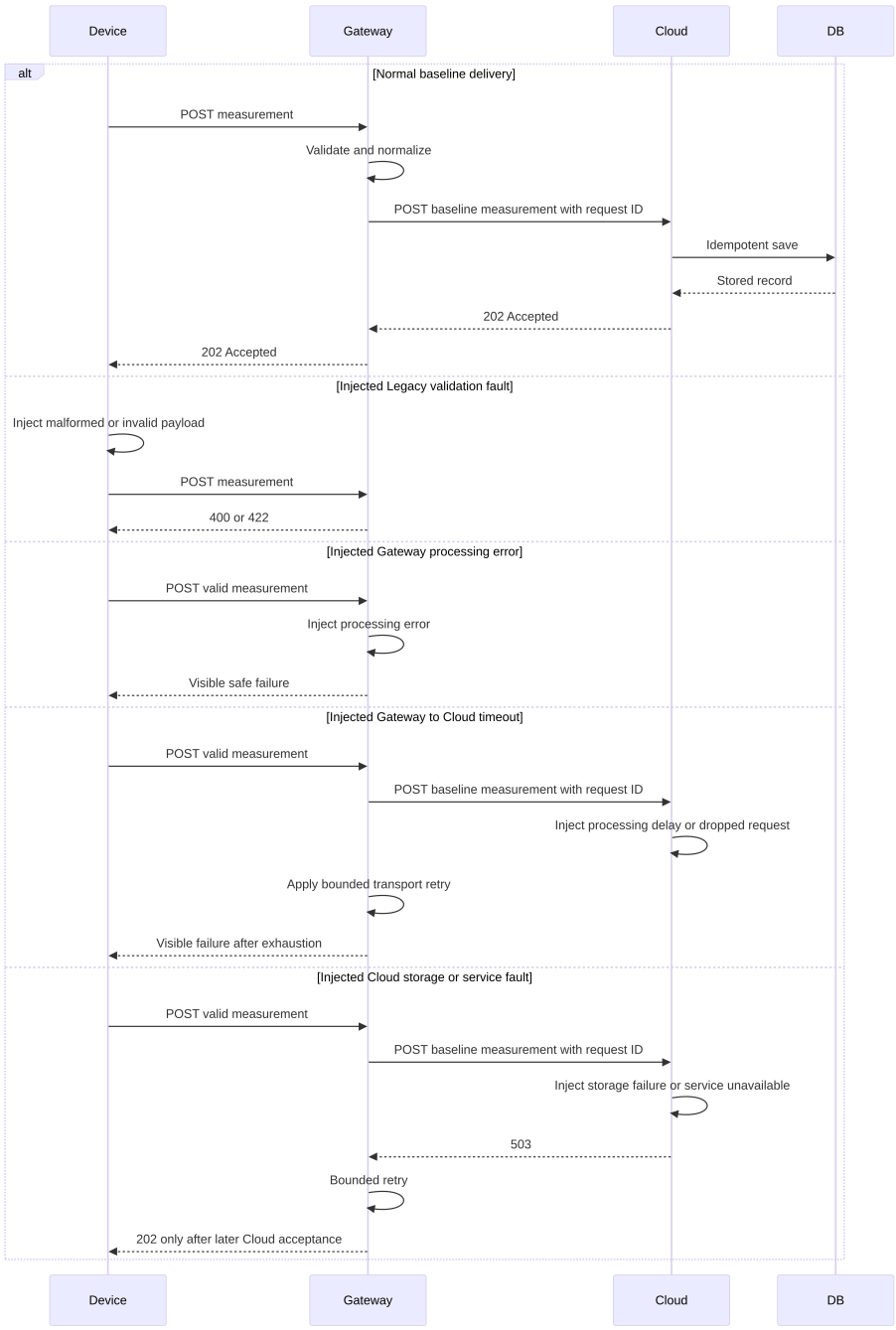
Table 2.3 — Explicit Gateway-to-Cloud modes

Baseline Mode transmits canonical JSON with an `X-Request-ID` header. It is retained for baseline measurement and controlled migration comparison. It is not a runtime fallback target. In Modernized Mode, a key, session, encryption, or secure-delivery failure stays a failure; the Gateway must never automatically retry the request through `/api/v1/baseline/measurements`.

2.3.1 Baseline Mode sequence with controlled fault injection

Failure injection is local configuration in the component that owns the fault; it is never added to a network message. The diagram shows representative baseline cases. Fault injection is disabled in ordinary operation and can be made deterministic for a test with a service-scoped `FAILURE_MODE`, `FAILURE_RATE=1.0`, and optional `FAILURE_SEED`.

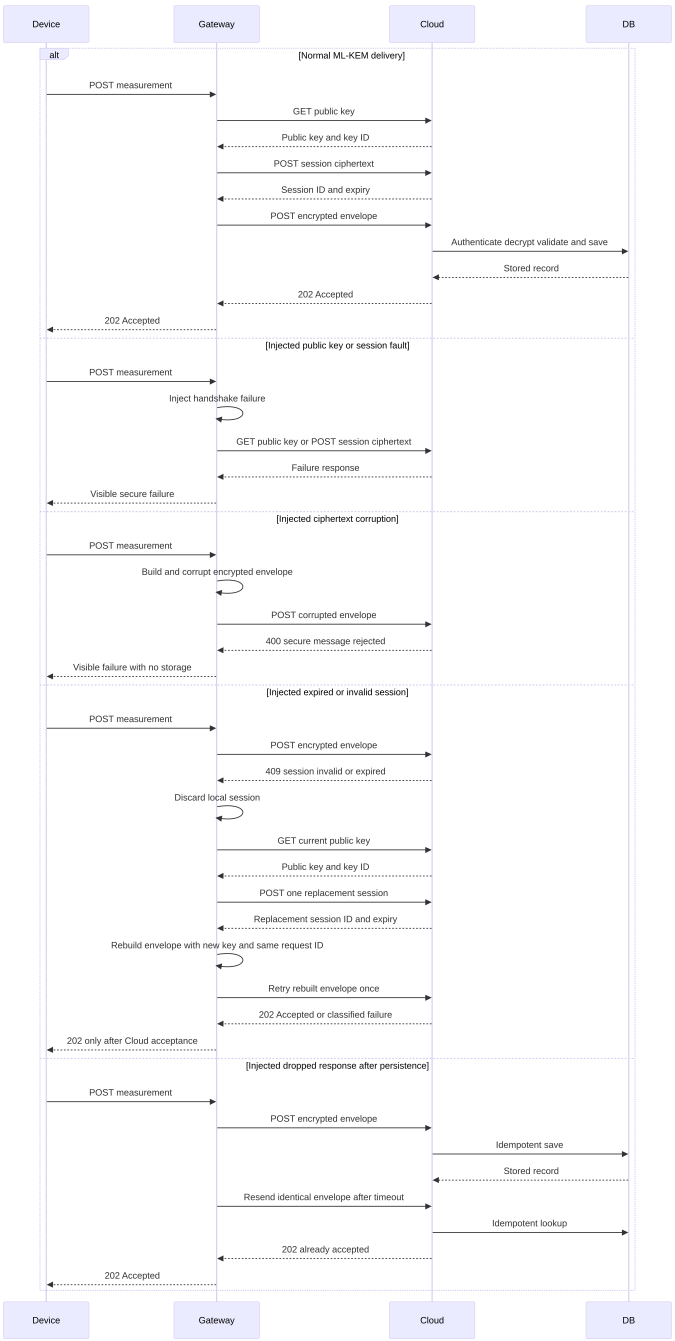
Figure 2.2 — Baseline Mode sequence with controlled fault injection



2.3.2 Modernized ML-KEM Mode sequence with controlled fault injection

Modernized Mode adds public-key retrieval, ML-KEM session establishment, and a secure envelope, but it preserves the Legacy Device interface. The failure branches illustrate the frozen distinction between an ordinary transport retry (reuse the exact envelope) and session recovery (create one replacement session, rebuild under its new key, and preserve the logical request ID).

Figure 2.3 — Modernized ML-KEM Mode sequence with controlled fault injection



2.4 Cryptographic suite and key-establishment contract

Protocol version `1` fixes one non-negotiated suite:

Code Listing 2.4 — Cryptographic suite and key-establishment contract (plaintext)

```
CRYPTO_SUITE = MLKEM768-HKDFSHA256-AES256GCM
```

Function	Frozen choice
Key establishment	ML-KEM-768
Key derivation	HKDF-SHA-256
Payload AEAD	AES-256-GCM

Function	Frozen choice
AEAD nonce	12 bytes (96 bits)
Session-key output	32 bytes
Protocol version	`"1"`
Binary JSON encoding	Base64URL, without whitespace

Table 2.4 — Cryptographic suite and key-establishment contract

The selected maintained implementation is `cryptography` version 48 or later, which provides FIPS 203 ML-KEM-768 together with HKDF and AES-GCM. The project does not implement ML-KEM or any other cryptographic primitive itself.

The Cloud makes its current public key available through:

Code Listing 2.5 — Cryptographic suite and key-establishment contract (http)

```
GET /api/v1/crypto/public-key
```

Code Listing 2.6 — Cryptographic suite and key-establishment contract (json)

```
{
  "protocol_version": "1",
  "algorithm": "ML-KEM-768",
  "key_id": "cloud-key-001",
  "public_key": "<base64url>"
}
```

For this controlled course environment, the Gateway trusts the public key returned by its explicitly configured Cloud URL. ML-KEM establishes keying material but does not authenticate the Cloud identity by itself; the protocol does not claim to provide PKI, certificate validation, mTLS, or public-key pinning.

To establish a session, the Gateway encapsulates to that public key and sends:

Code Listing 2.7 — Cryptographic suite and key-establishment contract (http)

```
POST /api/v1/crypto/sessions
Content-Type: application/json
```

Code Listing 2.8 — Cryptographic suite and key-establishment contract (json)

```
{
  "protocol_version": "1",
  "gateway_id": "gateway-001",
  "key_id": "cloud-key-001",
  "encapsulation_ciphertext": "<base64url>"
}
```

The Cloud decapsulates, derives equivalent key material, and responds:

Code Listing 2.9 — Cryptographic suite and key-establishment contract (json)

```
{
  "protocol_version": "1",
  "session_id": "sess-550e8400...",
  "status": "established",
  "expires_at": "2026-09-21T13:50:31Z"
}
```

Both sides apply exactly the same KDF operation:

Code Listing 2.10 — Cryptographic suite and key-establishment contract (plaintext)

```
HKDF-SHA-256(
  input_key_material = ML-KEM shared secret,
  salt = None,
  info = b"ml-kem-project:v1:gateway-cloud-session",
  length = 32
)
```

`salt=None` is part of the contract, as is the exact byte string in `info`. Neither a transcript-bound KDF nor custom hashing/concatenation is used. The shared secret and the derived session key never cross the network and must not be logged or persisted. A malformed ML-KEM ciphertext can raise a library error, but a modified ciphertext of the valid length can follow ML-KEM implicit rejection and yield different keying material instead; tests do not assume that every bit flip raises an exception.

2.5 Session lifecycle

Each session belongs to exactly one `gateway_id`, binds to the Cloud key ID used for its establishment, and is held in process-local memory. The Cloud tracks its session ID, gateway ID, key ID, created/expiry times, state, and secret key material; only the non-secret session ID and expiry are exposed.

The nominal session TTL is 300 seconds. The Gateway checks local expiry before sending, and the Cloud independently checks expiry for every secure request. Sessions are never silently extended. The process-local design means that a Gateway restart loses its local session and a Cloud restart loses session and key-pair state; recovery is a new handshake, not persistence of secrets.

Code Listing 2.11 — Session lifecycle (plaintext)

```
Gateway requests public key
  → encapsulates and derives session key
  → Cloud decapsulates and derives the same key
  → Cloud creates a 300-second, gateway-owned session
  → Gateway sends encrypted messages until expiry or invalidation
```

2.6 Secure-envelope contract

An established session protects one complete canonical measurement. The Gateway sends:

Code Listing 2.12 — Secure-envelope contract (http)

```
POST /api/v1/secure/measurements
Content-Type: application/json
```

Code Listing 2.13 — Secure-envelope contract (json)

```
{
  "protocol_version": "1",
  "gateway_id": "gateway-001",
  "session_id": "sess-550e8400...",
  "request_id": "req-660e8400...",
  "nonce": "<base64url>",
  "ciphertext": "<base64url>"
}
```

This `SecureEnvelope` contains exactly the six fields shown. Its ciphertext is AES-256-GCM output for the UTF-8 JSON canonical measurement. Binary values (`nonce` and `ciphertext`) are Base64URL encoded.

The envelope metadata is authenticated, not merely visible. Its AAD has exactly four fields:

Code Listing 2.14 — Secure-envelope contract (json)

```
{
  "gateway_id": "gateway-001",
  "protocol_version": "1",
  "request_id": "req-660e8400...",
  "session_id": "sess-550e8400..."
}
```

Both parties serialize that JSON object as UTF-8 with sorted keys and compact separators (`","` and `":"`). Any alteration of one of the four values causes AES-GCM authentication/decryption to fail. The protocol deliberately does not add `cloud_id`, a message counter, an algorithm field, or any fifth AAD field.

For each new logical secure message, the Gateway obtains a new 12-byte nonce from cryptographically secure randomness and creates a new request ID and envelope. It must never intentionally reuse a nonce with the same session key. The nonce is neither a sequence number nor a timestamp and must not come from a non-cryptographic PRNG.

On receipt, Cloud validates the envelope structure, protocol version, session existence/ownership, and expiry; reconstructs byte-identical AAD; then authenticates/decrypts, validates the canonical measurement again, and attempts idempotent storage. Authentication failure prevents storage.

2.7 Correlation, retries, and session recovery

The Gateway creates one non-secret, UUID-like `request_id` for every accepted Legacy request. It is used for tracing only, never as key material. In baseline mode it is propagated by `X-Request-ID`; in secure mode the envelope field authenticated as AAD is authoritative.

Ordinary transport retries (timeouts, connection failures, and HTTP `500`, `502`, `503`, or `504`) are bounded by the common three-attempt backoff schedule: wait one second after the first failure, two seconds after the second, then stop. A secure transport retry resends the *identical already built envelope*: same request ID, session ID, nonce, ciphertext, and AAD. It does not re-encrypt merely because the response was lost.

Secure session errors are different:

Code Listing 2.15 — Correlation, retries, and session recovery (plaintext)

```
409 SESSION_INVALID or 409 SESSION_EXPIRED
  → discard the local session
  → obtain the current Cloud public key
  → establish exactly one replacement ML-KEM session
  → rebuild the envelope with the replacement key
  → retry the same logical request once, preserving request_id
```

This one recovery attempt is separate from normal HTTP retry and must not be unbounded. It legitimately produces a new session ID, nonce, and ciphertext because a new session key is in use, while retaining the logical request ID.

The Legacy Device separately retries network errors, timeouts, and `500`/`502`/`503`/`504`, but not `400`, `409`, or `422` responses.

2.8 Response, error, and storage contracts

All application errors have the stable outer shape:

Code Listing 2.16 — Response, error, and storage contracts (json)

```
{
  "error": {
    "code": "INVALID_MEASUREMENT",
    "message": "measurement validation failed"
  }
}
```

Clients use the HTTP status and stable `error.code`, not free-text wording, to classify a response. The secure endpoint contract is:

Condition	HTTP status	Code / result
Invalid envelope JSON or Base64URL encoding	`400`	`SECURE_MESSAGE_REJECTED`
Unsupported protocol version	`400`	`UNSUPPORTED_PROTOCOL_VERSION`
Unknown session	`409`	`SESSION_INVALID`
Expired session	`409`	`SESSION_EXPIRED`
Session does not belong to gateway	`400`	`SECURE_MESSAGE_REJECTED`
AEAD authentication/decryption failure	`400`	`SECURE_MESSAGE_REJECTED`
Invalid decrypted canonical measurement	`422`	`INVALID_MEASUREMENT`
Duplicate logical measurement	`202`	idempotent success
Temporary storage failure	`503`	`STORAGE_UNAVAILABLE`
Unexpected server error	`500`	`INTERNAL_ERROR`

Table 2.5 — Response, error, and storage contracts

Session errors use `409 Conflict`, rather than HTTP user-authentication status codes, because this protocol does not model a user login. Externally, secure rejection avoids cryptographic detail; internally, operational logs may record the class of failure without keys or plaintext.

Cloud persists accepted canonical data as a record containing its `record_id`, device and Gateway IDs, measurement values, sequence number, `measured_at`, the Gateway's `received_at` mapped to `gateway_received_at`, and a Cloud receipt time. The persistent business identity is:

Code Listing 2.17 — Response, error, and storage contracts (plaintext)

```
UNIQUE(device_id, sequence_number)
```

If the same logical measurement is delivered again, Cloud does not insert a second row and returns `202`, normally with `status: "already_accepted"` and the existing record ID. This supports safe retransmission but is not a cryptographic replay defense: protocol version `1` deliberately has no dedicated replay counter or replay window.

2.9 Endpoint summary and security boundaries

Owner	Method	Endpoint	Purpose
Gateway	`POST`	`/api/v1/measurements`	Legacy measurement ingress
Cloud	`POST`	`/api/v1/baseline/measurements`	Baseline canonical measurement
Cloud	`GET`	`/api/v1/crypto/public-key`	Cloud ML-KEM public key
Cloud	`POST`	`/api/v1/crypto/sessions`	ML-KEM session establishment
Cloud	`POST`	`/api/v1/secure/measurements`	Secure-envelope ingress
Cloud	`GET`	`/api/v1/measurements`	Demonstration/query list
Cloud	`GET`	`/api/v1/measurements/{record_id}`	Demonstration/query record
Gateway and Cloud	`GET`	`/health`, `/ready`, `/metrics`	Operational endpoints

Table 2.6 — Endpoint summary and security boundaries

The security claim is scoped precisely: **Gateway-to-Cloud** traffic gains post-quantum key establishment and authenticated payload protection. The Legacy Device-to-Gateway connection remains legacy HTTP/JSON and is outside this PQC boundary. Likewise, Cloud key distribution is trusted only within the configured test environment, and business idempotency is not cryptographic anti-replay protection. These limitations are surfaced as technical debt and risk, not concealed as production-grade guarantees.

Chapter 3 — Component Design and Implementation Evidence

3.1 Implementation approach and ownership

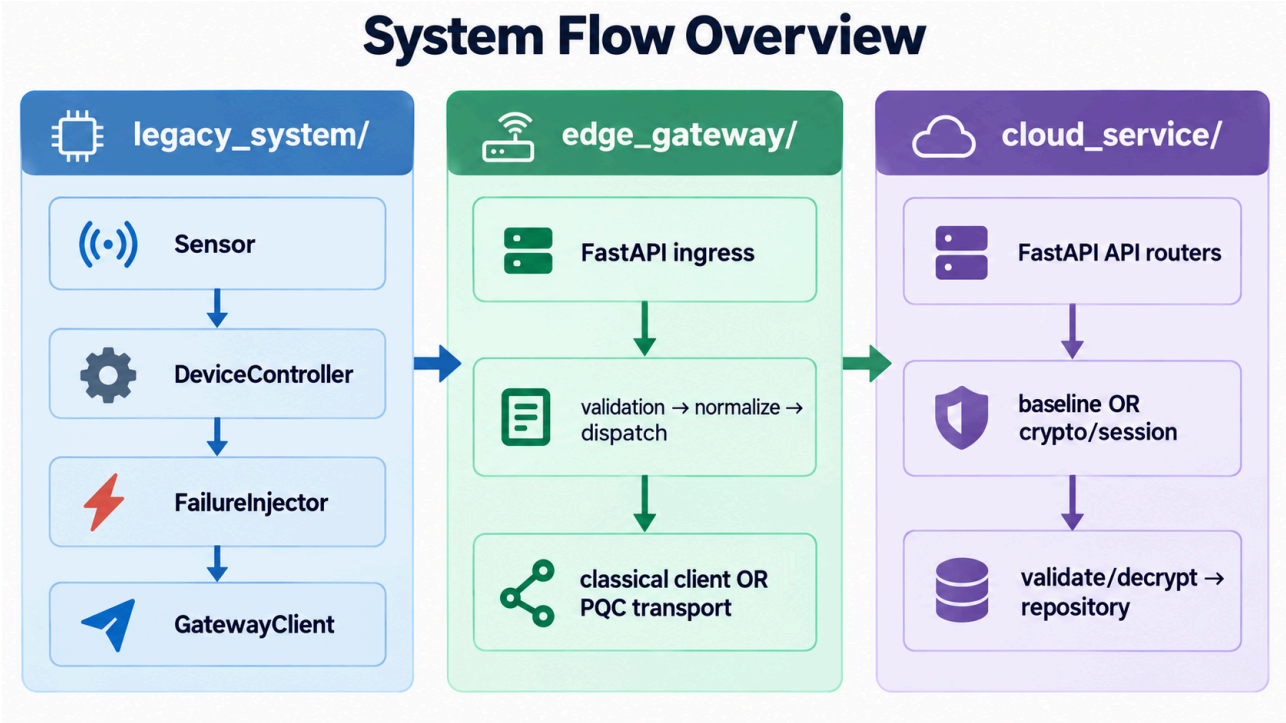
The implementation is divided into three deployable Python services. The division is architectural rather than cosmetic: each service owns a distinct part of the data path and must not absorb another component's responsibilities.

Component	Implemented responsibility	Deliberate exclusion
`legacy_system`	Simulate readings, serialize the legacy payload, make bounded HTTP delivery attempts, inject controlled device faults, and shut down cleanly.	No ML-KEM, Cloud API, encryption, session, or storage code.
`edge_gateway`	Provide Legacy API validation, normalization, sequence tracking, correlation, mode-specific Cloud dispatch, and Gateway-side PQC sessions/envelopes.	No durable business-record store and no direct sharing of internal models with Cloud.
`cloud_service`	Provide baseline and secure APIs, own Cloud ML-KEM/session behavior, decrypt and validate, enforce idempotency, store/query records, and expose health/metrics.	No Legacy Device wire-protocol support and no direct device communication.

Table 3.1 — Implementation approach and ownership

Each application is assembled through a `create_app(...)` factory or an equivalent controller. Dependencies are injected in tests and composition; importing a module does not bind a server port, establish a remote session, or write data. This makes unit testing practical and isolates the boundary between application logic, transport, cryptography, and storage.

Figure 3.1 — Implementation approach and ownership



3.2 Legacy Device implementation

The Legacy Device is intentionally a valid-data producer first and a failure-simulation client second. `Sensor` creates immutable `Measurement` objects with a configured device ID, UTC timestamp, increasing sequence number, and plausible environmental ranges (15–30 °C and 30–70% humidity during normal simulation). Its model-level validation uses the broader protocol domain: temperature from -40 to 85 °C, humidity from 0 to 100%, and a non-negative signed 64-bit sequence number.

The following excerpt from `legacy_system/src/legacy_system/sensor.py` shows that normal sensor generation remains independent of delivery:

Code Listing 3.1 — Legacy Device implementation (python)

```
def read(self) -> "Measurement":
    measurement = Measurement(
        device_id=self._device_id,
        temperature_c=self._random_source.uniform(15.0, 30.0),
        humidity_pct=self._random_source.uniform(30.0, 70.0),
        timestamp=self._clock(),
        sequence_number=self._next_sequence_number,
    )
    self._next_sequence_number += 1
    return measurement
```

`Measurement.to_payload()` converts the valid domain value into the five-field Legacy JSON representation and formats the UTC offset as `Z`. The `FailureInjector` acts *after* serialization. This is important because test cases such as a missing field or invalid timestamp must be represented on the wire without polluting the normal typed sensor model with invalid state.

`DeviceController` coordinates configuration, sensor reads, optional payload mutation, client delivery, structured events, the send interval, and graceful SIGINT/SIGTERM shutdown. Its main loop has no Cloud or PQC dependency:

Code Listing 3.2 — Legacy Device implementation (python)

```
while not self._stop_event.is_set():
    measurement = self._sensor.read()
    payload = measurement.to_payload()
    outgoing = self._failure_injector.apply(payload)
    self._send_with_logging(outgoing, measurement.sequence_number)
    if not self._stop_event.is_set():
        self._sleeper(self._config.send_interval_seconds)
```

The HTTP client classifies retryable network/server outcomes separately from non-retryable protocol responses. The controller logs identifiers, sequence numbers, result status, and attempt count, but no credentials or cryptographic material. This preserves the behavior expected of a constrained legacy client while still making delivery failures observable.

3.3 Edge Gateway implementation

3.3.1 Legacy ingress and application pipeline

The Edge Gateway is a FastAPI application constructed in `edge_gateway/src/edge_gateway/app.py`. It composes one configuration object, application and PQC metric registries, failure hooks, a `CloudDispatch`, and a `MeasurementProcessor`. It maps exceptions at the HTTP boundary, preserving the frozen distinction between syntax and semantic validation:

Code Listing 3.3 — Legacy ingress and application pipeline (python)

```python
@app.exception_handler(LegacyJSONError)
async def invalid_json(request: Request, error: LegacyJSONError):
    return _error_response(GatewayErrorCode.INVALID_JSON)       # HTTP 400

@app.exception_handler(LegacyMeasurementSchemaError)
@app.exception_handler(LegacyMeasurementDomainError)
async def invalid_measurement(request: Request, error: ValueError):
    return _error_response(GatewayErrorCode.INVALID_MEASUREMENT) # HTTP 422
```

The measurement endpoint limits the received body to 16 KiB, parses and validates the five legacy fields, captures a Gateway receipt time, and delegates processing. A successful endpoint response is produced only after the processor's Cloud dispatch succeeds:

Code Listing 3.4 — Legacy ingress and application pipeline (python)

```python
measurement = parse_legacy_measurement(bytes(body))
validate_legacy_measurement(measurement)
await processor.process(measurement, received_at=received_at)
return JSONResponse(status_code=202, content={"status": "accepted"})
```

`MeasurementProcessor` separates business flow from transport details. It normalizes a `LegacyMeasurement`, asks `SequenceTracker` to apply per-device policy, creates the request ID, and calls mode-specific dispatch. The sequence tracker holds the last valid value per device; it accepts first/increasing values, records but accepts gaps, and rejects duplicates or lower values. Its state is intentionally committed before downstream delivery and remains in-memory.

3.3.2 Classical and secure dispatch

`CloudDispatch` is the Gateway's explicit mode selector. With `SECURITY_MODE=classical`, the baseline client sends canonical JSON and `X-Request-ID` to the Cloud baseline endpoint. With `SECURITY_MODE=mlkem`, it uses `SecureTransport`. The application factory illustrates fail-closed composition: no secure transport is constructed implicitly in a classical process, and the baseline transport is unavailable in an ML-KEM process.

Code Listing 3.5 — Classical and secure dispatch (python)

```python
if settings.security_mode == "mlkem" and secure_transport is None:
    secure_transport = _secure_transport(settings, registry, failures, application_registry)
baseline = baseline_transport or _baseline_transport(settings, failures, application_registry)
dispatch = CloudDispatch(settings, baseline=baseline, secure=secure_transport,
                         pqc_metrics=registry)
```

The secure implementation is kept in `edge_gateway/crypto/` as small focused modules: `mlkem.py`, `kdf.py`, `symmetric.py`, `encoding.py`, `aad.py`, `envelope.py`, `public_key_client.py`, `session_client.py`, `session.py`, and `secure_transport.py`. This gives testable boundaries for maintained-library calls, binary encoding, deterministic AAD construction, session state, and transport behavior.

The envelope builder makes all required operations explicit:

Code Listing 3.6 — Classical and secure dispatch (python)

```python
aad = build_aad(protocol_version=PROTOCOL_VERSION, gateway_id=gateway_id,
                session_id=session_id, request_id=request_id)
nonce = generate_nonce()
ciphertext = encrypt(session_key, nonce, plaintext, aad)
return SecureEnvelope(protocol_version=PROTOCOL_VERSION, gateway_id=gateway_id,
                      session_id=session_id, request_id=request_id,
                      nonce=encode_binary(nonce), ciphertext=encode_binary(ciphertext))
```

This implementation evidence matters for the retry rule: `SecureTransport` retains one already-built envelope for ordinary transport retries instead of creating a nonce/ciphertext pair per attempt. A `SESSION_INVALID` or `SESSION_EXPIRED` response takes a different bounded path: it invalidates the local session, establishes one replacement, rebuilds under the new key, and preserves the logical request ID. No branch falls back automatically to the baseline endpoint.

3.4 Cloud Service implementation

3.4.1 Application lifecycle and API composition

`cloud_service/src/cloud_service/app.py` builds the Cloud FastAPI application. Its lifespan initializes SQLite storage, and only in ML-KEM mode creates the process-local key and session services. Shutdown reverses that lifecycle, closes sessions and key service, then closes the database. This avoids carrying secure runtime state across a process restart.

The application registers distinct routers for baseline ingestion, queries, public-key publication, session establishment, and secure ingestion. It also exports `/health`, `/ready`, and `/metrics`. Readiness checks storage and, in ML-KEM mode, the availability of key/session and cryptographic dependencies; it deliberately does not require an active Gateway session.

Code Listing 3.7 — Application lifecycle and API composition (python)

```python
app.include_router(gateway_router)
app.include_router(query_router)
app.include_router(crypto_router)
app.include_router(session_router)
app.include_router(secure_router)
```

3.4.2 Secure ingestion: validate, authenticate, validate again, then store

The secure route delegates its CPU/blocking pipeline to `cloud_service.gateway.secure._ingest`. Its ordering is security-critical: the envelope is structurally validated; session existence, gateway ownership, and expiry are checked; deterministic AAD is reconstructed; AES-GCM authenticates/decrypts; the resulting canonical measurement is validated; only then does the repository receive it.

Code Listing 3.8 — Secure ingestion: validate, authenticate, validate again, then store (python)

```
session = sessions.get(envelope.session_id, envelope.gateway_id)
aad = build_aad(protocol_version=envelope.protocol_version,
                gateway_id=envelope.gateway_id,
                session_id=envelope.session_id,
                request_id=envelope.request_id)
plaintext = decrypt(session.session_key, validated.nonce,
                    validated.ciphertext, aad)
measurement = validate_measurement(parse_json(plaintext),
                                   expected_gateway_id=envelope.gateway_id)
result = repository.save_idempotent(measurement)
```

The route converts malformed envelopes and cryptographic authentication failure to the deliberately non-specific `400 SECURE_MESSAGE_REJECTED` response; invalid decrypted business data is `422 INVALID_MEASUREMENT`; session errors are served through the session handler as `409`; and storage failure is `503`. The code also supports controlled failure modes, including a deliberately dropped acknowledgement after a committed write. That scenario tests that an ordinary Gateway retry reuses its envelope and arrives at the idempotent storage path.

3.4.3 SQLite storage and idempotency

The Cloud repository isolates parameterized SQLite work from HTTP and crypto code. `save_idempotent()` uses a targeted conflict clause on the frozen business identity, then returns the existing or newly created record in the same transaction:

Code Listing 3.9 — SQLite storage and idempotency (python)

```
cursor = connection.execute(
    _INSERT + " ON CONFLICT(device_id, sequence_number) DO NOTHING",
    candidate.to_dict(),
)
created = cursor.rowcount == 1
row = connection.execute(
    "SELECT * FROM measurements WHERE device_id = ? AND sequence_number = ?",
    (measurement.device_id, measurement.sequence_number),
).fetchone()
```

This prevents a retry from adding a second logical record and returns a stable record identity for either case. It does not attempt to turn SQLite idempotency into a cryptographic replay window; replay protection remains out of scope for protocol v1.

3.5 Cross-component engineering decisions visible in the code

The selected implementation patterns provide direct evidence for several project-wide requirements:

Requirement	Implementation evidence
Legacy compatibility	Sensor/controller and HTTP client are isolated in `legacy_system`; no crypto imports are needed there.
Frozen validation semantics	Gateway exception handlers differentiate `400`, `422`, and `409`; Cloud repeats canonical validation after decryption.

Requirement	Implementation evidence
No plaintext downgrade	Gateway composes separate baseline/secure transports and secure dispatch has no automatic baseline route.
Secret-safe operation	Session/key material is never returned by a route or written into normal application log fields; error messages are classified rather than diagnostic.
Correct retry behavior	Gateway separates ordinary envelope reuse from session-recovery rebuild; Cloud's dropped-response mode exercises persistence-before-response behavior.
Idempotent persistence	SQLite uses `ON CONFLICT(device_id, sequence_number) DO NOTHING` within the repository transaction.
Testability	Factories, dependency injection, focused crypto modules, repository isolation, injectable clocks/randomness, and controlled failure injectors support component and integration tests.

Table 3.2 — Cross-component engineering decisions visible in the code

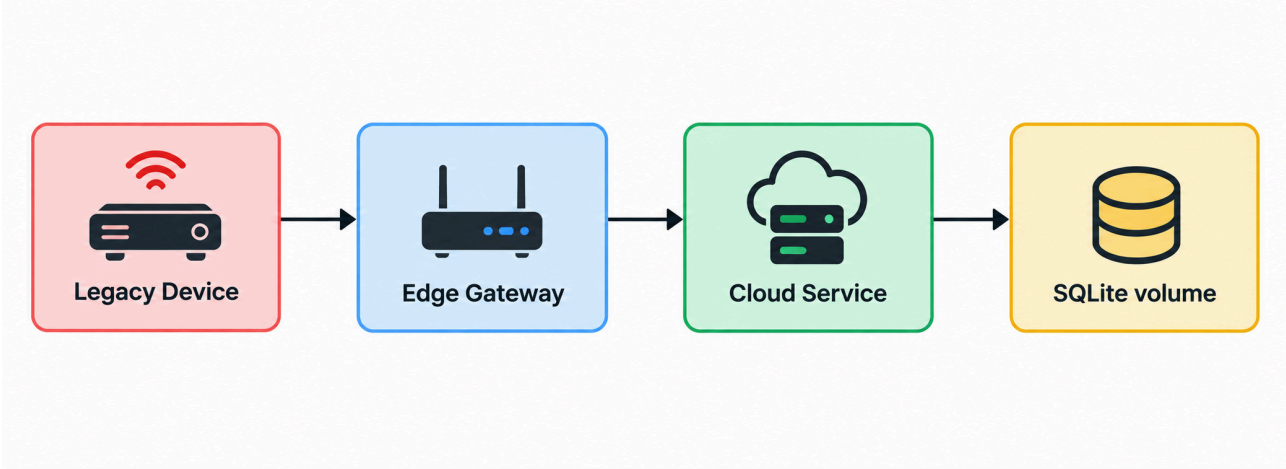
These choices keep the codebase course-sized while retaining the boundaries needed to inspect AI-generated changes critically. The full protocol constants and wire contracts are in Chapter 2; deployment, monitoring, and CI evidence is in Chapter 4; and verification evidence is in Chapter 5.

Chapter 4 — Operations, Deployment and CI/CD

4.1 Operational design goals

The operational work makes the three-service telemetry path reproducible and inspectable:

Figure 4.1 — Operational design goals



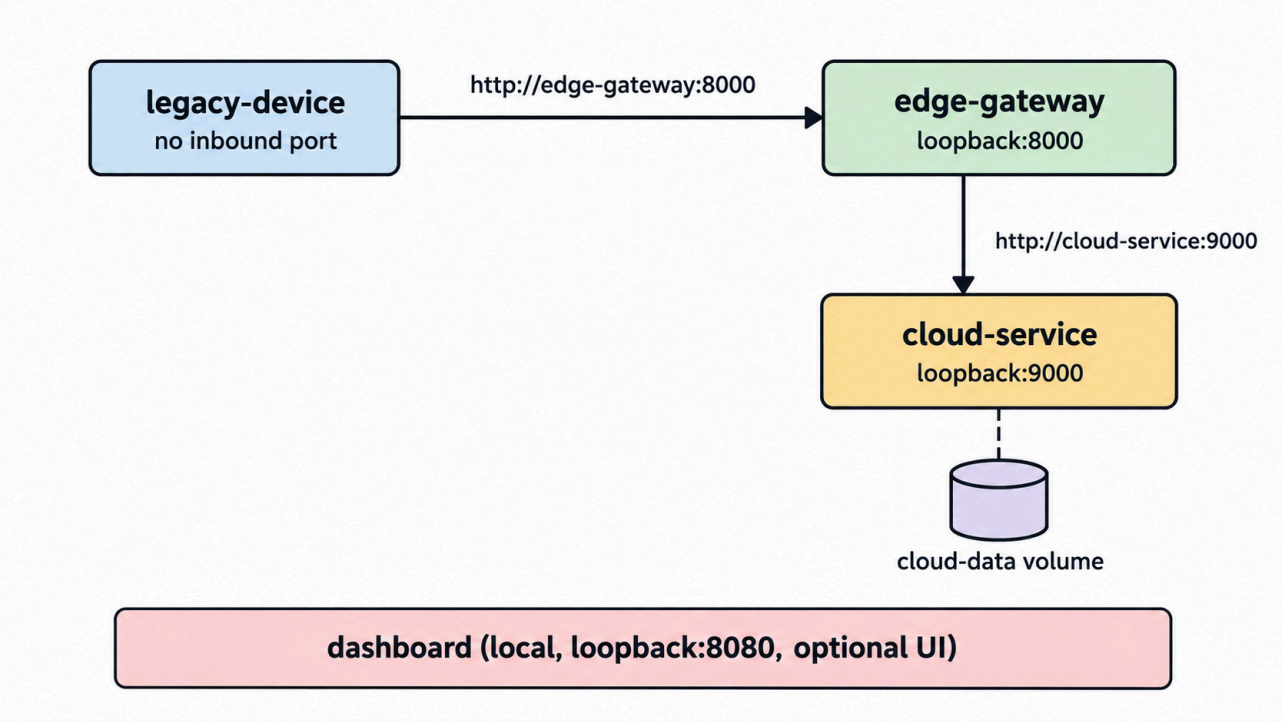
The core goals are deterministic local deployment, explicit security-mode selection, liveness/readiness visibility, safe correlation, bounded metrics, controlled fault injection, and automated regression gates. The system uses Python 3.12 and the root `uv` workspace with its committed `uv.lock`; it does not add a separate `requirements.txt` dependency path.

The deployment is intentionally a single-host course test environment. It is not a claim of high availability, distributed storage, durable Gateway buffering, or production transport identity. These limits are recorded in the technical-debt and risk registers rather than being hidden by the operational documentation.

4.2 Docker Compose topology and lifecycle

`docker-compose.yml` (Appendix A.1) is the reproducible system topology. It builds the three application images from their service Dockerfiles, connects them through Compose DNS, exposes only the Gateway and Cloud application ports on loopback, and persists accepted Cloud measurements in the named `cloud-data` volume.

Figure 4.2 — Docker Compose topology and lifecycle



The dashboard is a local operational aid, not a data-path component. It can display service/metric status and apply restricted temporary overrides for a selected container. Because it needs local Docker control, it mounts the Docker socket and is therefore deliberately bound to `127.0.0.1`, carries no remote exposure or credentials, and is appropriate only for a trusted local demonstration environment.

Compose starts the Cloud first, waits for its health check before starting the Gateway, then waits for Gateway health before starting the Legacy Device. The Cloud's SQLite path defaults to `sqlite:////data/cloud.db` in the named volume. Internal service URLs are `http://edge-gateway:8000` and `http://cloud-service:9000`; host-port overrides are available for local port conflicts.

The documented start sequence is:

Code Listing 4.1 — Docker Compose topology and lifecycle (bash)

```bash
docker compose config --quiet
docker compose build

# Baseline comparison run
SECURITY_MODE=classical docker compose up --build --wait

# Modernized run (after stopping the preceding stack)
docker compose down
SECURITY_MODE=mlkem PQC_FAILURE_POLICY=fail_closed docker compose up --build --wait
```

The security-mode change is explicit. Modernized Mode retains the frozen `PQC_FAILURE_POLICY=fail_closed` value; it never responds to a handshake or secure-delivery problem by forwarding the measurement through the baseline endpoint. Service-specific root environment variables (`LEGACY_FAILURE_*`, `GATEWAY_FAILURE_*`, and `CLOUD_FAILURE_*`) map to each process's local failure configuration, allowing controlled tests without sending failure-control data over the network.

`docker compose down` stops the stack while retaining the demonstration database. `docker compose down --volumes --remove-orphans` is the documented clean reset and permanently deletes the locally stored measurements in `cloud-data`; it is used before deterministic demonstrations and automated assertions. The volume contains accepted business records only—private keys, shared secrets, session keys, and session state remain process-local.

4.3 Health and readiness contracts

Gateway and Cloud each expose `GET /health` and `GET /ready`; Compose health checks call the health endpoints from inside the respective containers. A normal health result is `{ "status": "ok" }`. Readiness adds meaningful dependencies rather than merely reporting process existence:

Service and mode	Readiness condition	Example successful response
Gateway	Process is running and its configured dispatch is available.	`{ "status": "ready", "security_mode": "mlkem" }`
Cloud, classical	Process is running and SQLite is initialized/writable.	`{ "status": "ready", "storage": "ok" }`
Cloud, ML-KEM	Cloud storage is writable and key/session/crypto dependencies are initialized.	`{ "status": "ready", "storage": "ok", "mlkem": "available" }`

Table 4.1 — Health and readiness contracts

An active secure session is intentionally **not** a readiness dependency on either service. Sessions are created lazily on the first secure delivery, so a zero active-session count is healthy before traffic. Conversely, the absence of a usable cryptographic subsystem in Modernized Mode makes the Cloud unready. The Legacy Device has no inbound health/readiness API; its operational state is represented through its process/container status and logs rather than by inventing an endpoint.

The standard post-start operational check queries both services:

Code Listing 4.2 — Health and readiness contracts (bash)

```
curl --fail http://127.0.0.1:${GATEWAY_HOST_PORT:-8000}/health
curl --fail http://127.0.0.1:${GATEWAY_HOST_PORT:-8000}/ready
curl --fail http://127.0.0.1:${CLOUD_HOST_PORT:-9000}/health
curl --fail http://127.0.0.1:${CLOUD_HOST_PORT:-9000}/ready
curl --fail http://127.0.0.1:${GATEWAY_HOST_PORT:-8000}/metrics
curl --fail http://127.0.0.1:${CLOUD_HOST_PORT:-9000}/metrics
curl --fail http://127.0.0.1:${CLOUD_HOST_PORT:-9000}/api/v1/measurements
```

4.4 Structured logging, correlation, and secret safety

The project adopts compact structured-text events: stable, lowercase `event=...` names followed by bounded `key=value` fields. The common correlation vocabulary is `event`, `component`, `request_id`, `gateway_id`, `device_id`, and `sequence_number`; `session_id`, `key_id`, and `record_id` may be included when relevant. A typical trace follows the same request ID from Gateway receipt through Cloud persistence:

Code Listing 4.3 — Structured logging, correlation, and secret safety (plaintext)

```
Gateway: event=legacy_measurement_received request_id=req-123 device_id=sensor-001
Gateway: event=cloud_delivery_started request_id=req-123 session_id=sess-55
Cloud:   event=secure_measurement_received request_id=req-123 gateway_id=gateway-001
Cloud:   event=measurement_stored request_id=req-123 record_id=rec-900
```

The logging standard prohibits raw request/response bodies, arbitrary headers, unbounded exception text, configuration values, and sensitive cryptographic buffers. Specifically, ML-KEM private/public key material, encapsulation ciphertext, shared secrets, AES-GCM keys/nonces/ciphertexts/tags/plaintext, passwords, tokens, credentials, and `.env` contents must not be logged.

The secret-logging review combines source inspection with automated tests. It includes Gateway tests for the absence of generated shared secrets and session keys, Cloud tests covering key/AEAD/error paths, and a live secure end-to-end observability regression that checks captured Gateway/Cloud logs for absence of the actual session secret and derived key. This is evidence of the current reviewed paths, not a claim that future changes will be safe without review; logging and error changes remain security-review work.

4.5 Metrics and local monitoring

Gateway and Cloud expose Prometheus-style plaintext metrics at `GET /metrics`. Their metric families are intentionally bounded: correlation identifiers such as `request_id` and `session_id` never become labels. The metric contract covers normal flow, failures, session state, and controlled test conditions. Counters end in `_total`, duration families end in `_duration_seconds`, and active-session values are gauges. They are defined to answer an operational question, rather than merely to count implementation details.

4.5.1 Gateway metric inventory

Metric	Type	Why it is defined
`gateway_measurements_received_total`	Counter	Establishes incoming Legacy-device workload and is the denominator for Gateway acceptance/rejection analysis.
`gateway_measurements_accepted_total`	Counter	Shows how many validated measurements entered the accepted delivery path.
`gateway_measurements_rejected_total`	Counter	Detects malformed, invalid, or sequence-policy-rejected Legacy traffic.
`gateway_duplicate_measurements_total`	Counter	Separates equal-sequence rejections from general validation failure, aiding device/retry diagnosis.
`gateway_out_of_order_measurements_total`	Counter	Identifies lower sequence values, which can indicate device restart, delayed delivery, or ordering problems.
`gateway_sequence_gaps_total`	Counter	Reveals accepted jumps in a device sequence without treating a gap as a false rejection.
`gateway_cloud_requests_total`	Counter	Counts each Gateway-to-Cloud attempt and supports comparison with accepted input and Cloud outcomes.
`gateway_cloud_failures_total`	Counter	Makes downstream availability/delivery problems visible.
`gateway_cloud_retries_total`	Counter	Shows whether bounded ordinary retries are occurring and helps detect retry amplification.
`gateway_cloud_request_duration_seconds`	Duration	Measures Cloud-delivery latency, including its contribution to end-to-end processing time.
`gateway_mlkem_handshakes_total`	Counter	Counts ML-KEM session-establishment attempts/outcomes, providing the volume for secure-

Metric	Type	Why it is defined
		mode reliability analysis.
`gateway_mlkem_handshake_failures_total`	Counter	Detects failed key/session establishment independently from message-encryption failures.
`gateway_mlkem_handshake_duration_seconds`	Duration	Isolates first-message session-establishment cost from steady-state message delivery cost.
`gateway_session_reestablishments_total`	Counter	Shows bounded recovery attempts after `SESSION_INVALID` or `SESSION_EXPIRED`; unexpected growth indicates session churn.
`gateway_encryption_failures_total`	Counter	Detects inability to build a secure envelope before transmission.
`gateway_active_session`	Gauge	Indicates whether the Gateway currently has one usable in-memory secure session; it is not a readiness requirement.
`gateway_pqc_fallback_total`	Counter/invariant	Must remain `0`: any non-zero value would expose an impermissible runtime plaintext downgrade in protocol v1.
`gateway_injected_failures_total`	Counter	Distinguishes intentionally simulated test failures from an unexplained operational fault.

Table 4.2 — Gateway metric inventory

4.5.2 Cloud metric inventory

Metric	Type	Why it is defined
`cloud_baseline_requests_total`	Counter	Provides the baseline-mode request volume for controlled comparison with secure mode.
`cloud_secure_requests_total`	Counter	Counts secure-envelope traffic reaching the Cloud boundary.
`cloud_secure_requests_rejected_total`	Counter	Makes secure messages rejected before acceptance/storage visible without exposing their content.
`cloud_request_duration_seconds`	Duration	Measures Cloud-side request-processing time and helps separate Cloud cost from Gateway delivery timing.
`cloud_mlkem_sessions_created_total`	Counter	Shows successful creation of Cloud-side secure sessions.
`cloud_mlkem_session_failures_total`	Counter	Detects failed session establishment on the receiving/decapsulation side.
`cloud_mlkem_decapsulation_duration_seconds`	Duration	Measures the Cloud's ML-KEM decapsulation contribution to handshake overhead.
`cloud_active_sessions`	Gauge	Shows the number of live process-local sessions, helping explain recovery or restart behavior.
`cloud_expired_sessions_total`	Counter	Identifies expiry/cleanup activity and provides context for session-recovery events.
`cloud_authenticated_messages_total`	Counter	Counts envelopes that passed AES-GCM authentication and decryption.
`cloud_decryption_failures_total`	Counter	Detects authentication/decryption rejection, including tampering or mismatched session/key material.
`cloud_measurements_stored_total`	Counter	Counts newly persisted logical measurements.
`cloud_storage_failures_total`	Counter	Makes SQLite/storage availability failures visible to operators and tests.

Metric	Type	Why it is defined
`cloud_duplicate_measurements_total`	Counter	Shows idempotent retransmissions that returned an existing logical record instead of producing a second row.
`cloud_injected_failures_total`	Counter	Marks deliberate Cloud fault scenarios so their effects are interpretable during tests.

Table 4.3 — Cloud metric inventory

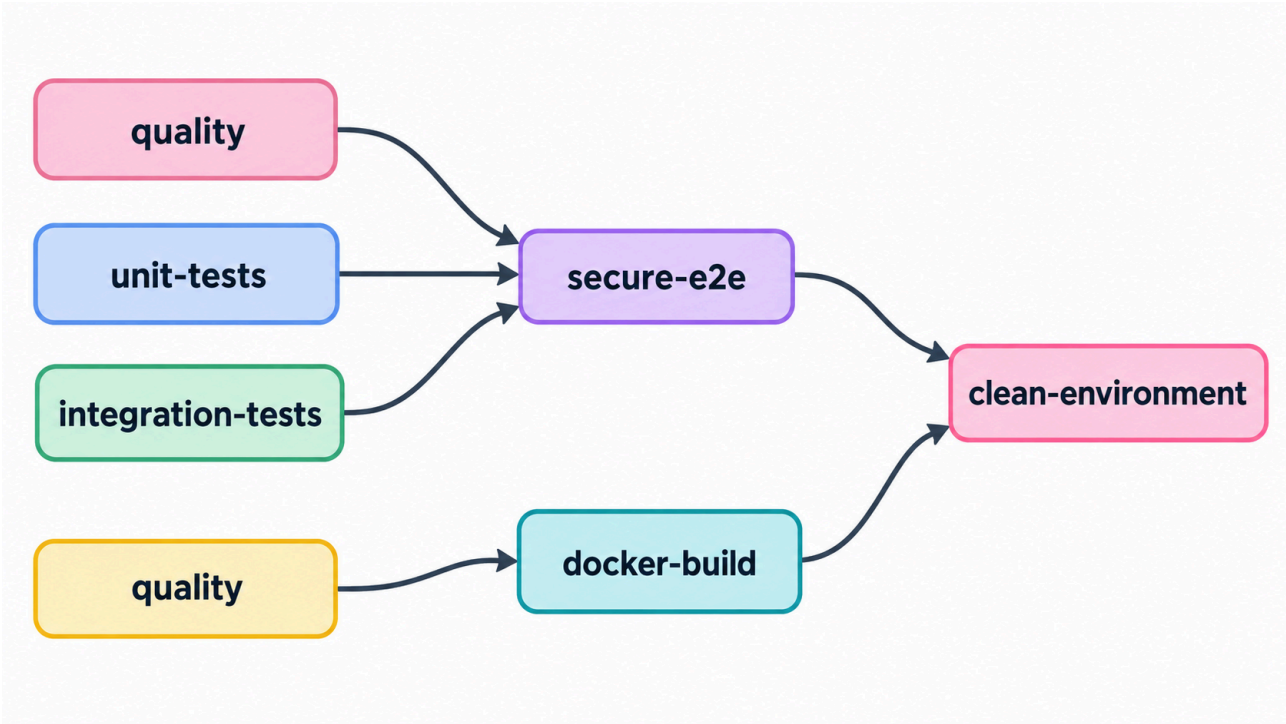
Metrics can have only finite, low-cardinality labels such as `result`, `failure_type`, `security_mode`, or `phase`. Labels must not contain device, request, session, key, or record IDs; timestamps; payload values; nonces; ciphertexts; or secrets. Correlation belongs in logs, not in a time-series label set. Together, these metrics answer whether traffic is accepted, rejected, retried, encrypted, authenticated, stored, or intentionally failed; they support, but do not replace, the test and empirical evidence in Chapter 5.

The local dashboard consumes this bounded operational information and makes the service status visible during a demonstration. It explicitly reports the Legacy Device's lack of HTTP health/readiness rather than presenting a false probe result. Browser actions that apply a permitted temporary override show a confirmation and recreate only the selected service; a normal Compose recreate restores the committed Compose/`.env` configuration.

4.6 Continuous integration and delivery gates

The committed GitHub Actions workflow `.github/workflows/ai-history.yml` (Appendix B.1) runs on pushes and pull requests. It uses Python 3.12, installs the root workspace with `uv sync --frozen`, and separates fast quality feedback from integration, secure end-to-end, container-build, and clean-environment evidence. Each job uses a fresh Ubuntu runner; therefore no test depends on a prior job's local virtual environment, containers, database, or cached service process.

Figure 4.3 — Continuous integration and delivery gates



CI job	Commands and coverage	Why the gate exists
`quality`	`uv sync --frozen`, `uv run ruff check .`, then `ai-history --help`, `status`, `list`, and `check`.	Rejects style/static defects and ensures the locked dependency graph and AI-governance

CI job	Commands and coverage	Why the gate exists
		CLI work in a clean runner.
`unit-tests`	Component/repository tests, targeted protocol-model and session/envelope/AAD/KDF contract tests, Gateway crypto/transport/trust/logging checks, and tamper regression.	Proves local behavior and frozen protocol encodings before exercising service composition.
`integration-tests`	Entire `tests/integration` suite, targeted observability/metric sanity tests, Cloud checks, and Legacy–Gateway, baseline, ML-KEM-session, and secure-path tests.	Detects mismatches between independently owned components, including failure classification and cross-service metric behavior.
`secure-e2e`	Runs only after `quality`, `unit-tests`, and `integration-tests`; executes classical E2E, ML-KEM E2E, and legacy-compatibility regression tests.	Demonstrates that both explicit modes still work across the complete Legacy-to-Cloud path after earlier gates pass.
`docker-build`	Runs after `quality` and builds the Legacy Device, Gateway, and Cloud images from their committed Dockerfiles.	Catches container-build/dependency issues that Python-only tests cannot reveal.
`clean-environment`	Runs after both `secure-e2e` and `docker-build`; reinstalls with `uv sync --frozen`, validates resolved Compose configuration, and runs the Compose recovery regression.	Rechecks reproducibility from the committed lockfile and validates the deployed three-service recovery path without relying on a developer workstation.

Table 4.4 — Continuous integration and delivery gates

The workflow's dependency graph is intentional. `secure-e2e` cannot start until static quality, component/protocol behavior, and cross-service behavior have passed. `clean-environment` cannot start until both end-to-end evidence and image builds pass, so the Compose recovery check is the final automated gate. Failing jobs stop dependent stages, giving fast diagnosis at the lowest useful layer rather than obscuring a unit defect behind an E2E failure.

This is a CI validation pipeline, not continuous deployment (CD) to a public or production environment. It does not push images, alter remote infrastructure, create a release, or deploy secrets. The delivery-related evidence is instead reproducible image construction and the locally documented Compose test deployment. The locked install and clean-environment stage protect against a cached developer environment hiding an unavailable or unlocked dependency. AI-history validation is a separate governance gate: it checks that recorded history is structurally valid, but never substitutes for the required human review and final verification.

4.7 Operational boundaries and handover procedure

The root runbook defines the normal operator sequence: validate Compose, choose mode explicitly, start with `--wait`, check health/readiness/metrics and the Cloud query API, inspect secret-safe operational logs, then stop or reset the environment. The recommended repository validation set before review is:

Code Listing 4.4 — Operational boundaries and handover procedure (bash)

```
uv sync --frozen
uv run ruff check .
uv run pytest
uv run ai-history check
```

During Modernized Mode, a `409 SESSION_INVALID` or `409 SESSION_EXPIRED` causes at most one secure recovery: a replacement session is established and the logical request ID is preserved. An ordinary ambiguous-response retry, by contrast, resends the identical already-built `SecureEnvelope`. Operators must not mistake the idempotent database row guarantee for cryptographic replay protection: protocol version 1 has no replay counter/window.

The remaining operational limitations are explicit: HTTP-only controlled transport, ephemeral Cloud key/session state, in-memory Gateway sequence state, single-instance SQLite, simple bounded retry behavior, and no persistent Gateway queue. These constraints are adequate for reproducible course demonstration and testing; Chapter 7 evaluates their associated debt and risks.

Chapter 5 — Verification, Testing and Empirical Evaluation

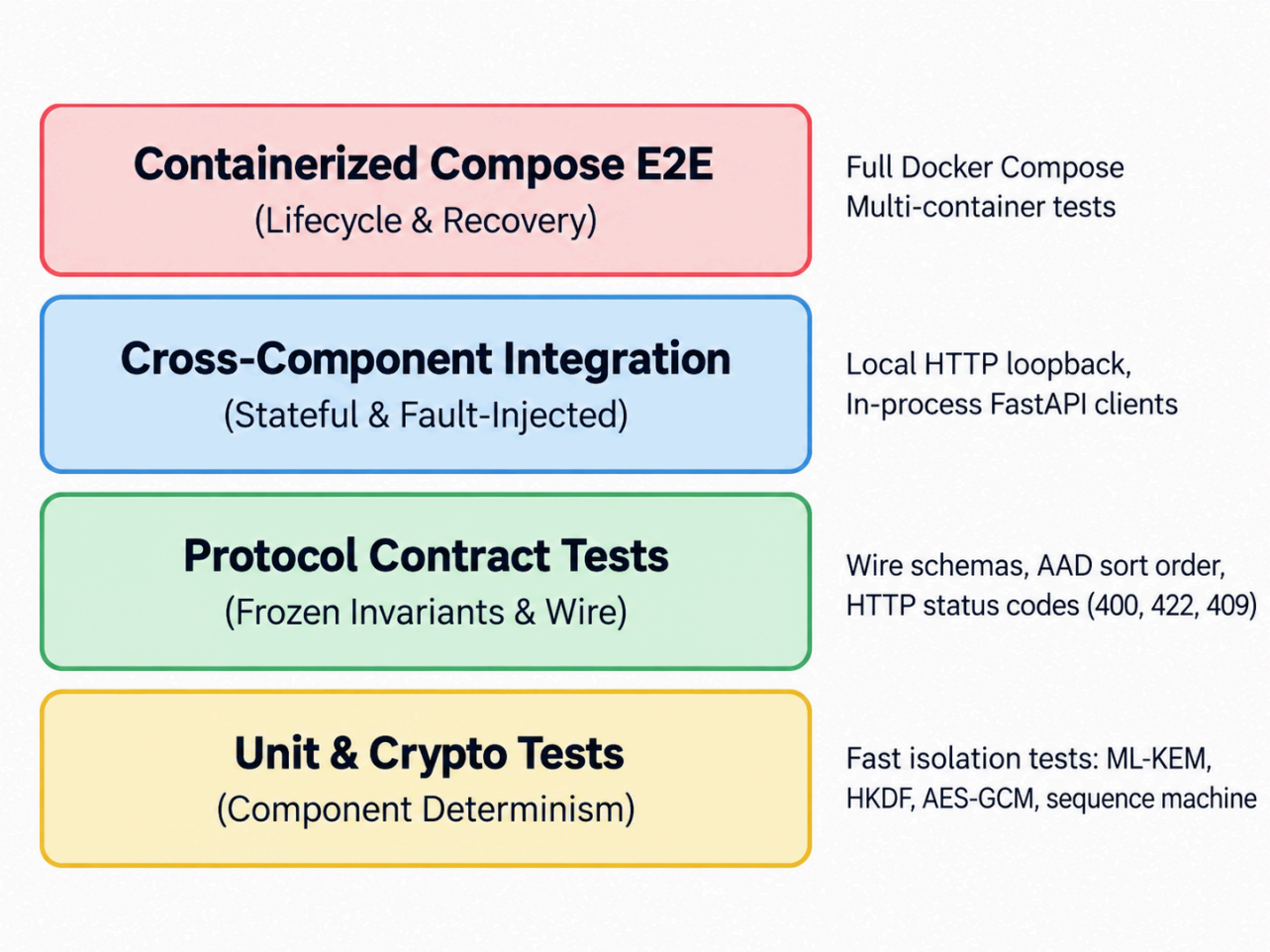
5.1 Verification Strategy and Testing Pyramid

5.1.1 Multi-Layered Testing Architecture

The modernization of a legacy telemetry pipeline to incorporate Post-Quantum Cryptography (PQC) introduces intricate failure modes spanning serialization, key encapsulation, session state management, authenticated encryption, and database persistence. The verification strategy documented herein provides rigorous, self-contained evidence of functional correctness, cryptographic safety, and operational resilience across all system components.

The test suite is structured as a multi-tier testing pyramid designed to catch defects at the lowest feasible level of abstraction while validating end-to-end operational behaviors in clean, containerized environments:

Figure 5.1 — Multi-Layered Testing Architecture



1. **Unit and Cryptographic Primitive Tests**: - Focuses on isolated components: the legacy sensor generator and configuration parser, in-memory sequence tracking state machines (Appendix D.5), ML-KEM encapsulation and decapsulation routines (Appendix D.1 and D.6), and HKDF-SHA-256 session key derivation (Appendix D.2). - Executes with zero external I/O, utilizing maintained cryptographic implementations (`cryptography` library) and mock-free parameter checks.

2. **Protocol Contract Tests**: - Validates strict compliance with frozen wire specifications defined in the Shared Cross-Component Protocol Specification (summarized in wire contracts in Chapter 2). - Enforces byte-exact canonical JSON serialization, sorted 4-field Associated Data (AAD) encoding, Base64URL encoding/decoding without padding, and exact HTTP response status codes (`400 Bad Request` for syntax failure, `422 Unprocessable Entity` for semantic/domain failures, `409 Conflict` for sequence or session collisions).
3. **Cross-Component Integration Tests**: - Executes across service boundaries using FastAPI `TestClient` fixtures and ephemeral local HTTP loopback servers. - Evaluates stateful interactions: session negotiation handshakes, multi-message steady-state telemetry flows, session expiry after 300-second TTL, and ambiguous transport timeout retries.
4. **Containerized End-to-End and Recovery Tests**: - Deploys the complete three-tier Docker Compose topology (`legacy-device`, `edge-gateway`, `cloud-service`). - Asserts real network delivery across container boundaries, clean service startup, health/readiness probe progression, dynamic container restarts (e.g., simulating Cloud or Gateway crash-recovery), and session re-establishment under containerized state loss.

5.1.2 Repository-Wide Test Scale and Scope

Automated verification is comprehensive across all architectural tiers. Across the repository, the automated verification harness collects and executes **1,728 passing test cases**, distributed across component-level unit suites, cross-component protocol contracts, security regression suites, and deployment lifecycle assertions:

Functional Verification Scope	Executed Test Cases	Share	Primary Test Focus & Verification Invariants
Edge Gateway Application and Crypto Suites	**769**	44.5%	In-memory sequence tracking, schema validation, ML-KEM-768 encapsulation, HKDF-SHA-256 derivation, AES-256-GCM AEAD encryption, canonical 4-field AAD encoding, session state machines, and fail-closed dispatch
Cloud Service Storage and Decryption Suites	**517**	29.9%	ML-KEM-768 key pair lifecycle, public key exposition, ML-KEM decapsulation, AES-256-GCM authenticated decryption, SQLite persistence transactions, and business idempotency constraints
Legacy Device Ingestion and Sensor Suites	**227**	13.1%	Monotonic sequence generation, sensor telemetry models, bounded retry policies, domain validation, and simulated physical fault generation
Cross-Component Integration and Security Suites	**114**	6.6%	End-to-end loopback integration, 18-scenario failure injection catalog (F1–F18), ciphertext bit-flip tampering, AAD modification detection, and session expiry recovery
Deployment, Hygiene and Provenance Suites	**101**	5.8%	Multi-stage Docker container builds, Compose crash recovery, secret-safety logging scans, CI/CD workflow contracts, and immutable AI provenance integrity
Total Automated Verification Suite	**1,728**	**100.0%**	**All 1,728 automated test cases passing with zero defects or unhandled exceptions**

Table 5.1 — Repository-Wide Test Scale and Scope

This extensive test coverage verifies that component interactions adhere to frozen specifications and that documented failure paths are exercised by automated test code.

5.1.3 Test Isolation, Determinism and Secret Safety Principles

To guarantee deterministic execution and eliminate cross-test interference: - **Ephemeral Storage**: Every database-dependent test instantiates an isolated, temporary SQLite database (`test.db`) created within an ephemeral test directory. No shared or pre-seeded database files are committed to the repository. - **Entropy and PRNG Isolation**: While cryptographic primitives (ML-KEM key generation, ephemeral encapsulation, AES-GCM nonce generation) draw strictly from cryptographically secure operating system randomness (`os.urandom` via `cryptography`),

non-cryptographic simulation modules (such as legacy sensor jitter and fault injection) utilize dedicated, seeded `random.Random(seed)` instances. This prevents simulation tests from polluting global test entropy. - **Strict Secret Safety in Test Assertions**: Tests asserting cryptographic keys or shared secrets explicitly avoid printing raw binary values in assertion failures. Comparisons are performed via boolean equality assertions (`assert secrets_match is True`), preventing pytest from capturing sensitive key material in stdout/stderr assertion introspection.

5.2 Comprehensive Test and Failure Matrices

5.2.1 Repository Test Coverage Matrix

The repository test matrix maps each functional area across the architectural tiers to its corresponding automated test coverage layers:

Architectural Area	Unit Tests	Contract Tests	Integration Tests	Compose E2E	Failure & Security Suite	Verification Test Suite & Strategy
Legacy Device	Yes	Yes	Yes	Yes	Yes	Legacy Device Unit & Validation Suite; Legacy-Gateway Loopback Integration Suite; Ingestion Fault Injection Engine
Gateway Application	Yes	Yes	Yes	Yes	Yes	Gateway Core Application Suite; Legacy Normalization & Transport Baseline Suite; Sequence Tracking Engine
Gateway Crypto & Sessions	Yes	Yes	Yes	Yes	Yes	Gateway Post-Quantum Cryptographic Suite (Appendix D.1–D.3); Ephemeral Session Orchestration Suite
Cloud API & Crypto	Yes	Yes	Yes	Yes	Yes	Cloud Ingestion & PQC Decapsulation Suite (Appendix D.6–D.7); Authenticated Secure Transport Suite
SQLite Storage	Yes	N/A	Yes	Yes	Yes	Cloud SQLite Storage Suite; High-Throughput Ingestion & Unique Constraint Suite
Cross-Component Protocol	N/A	Yes	Yes	Yes	Yes	Cross-Component Protocol Contract Suite; Stateful Handshake & Lifecycle Suite; Compose Orchestration Suite
Deployment and CI	N/A	N/A	Yes	Yes	Yes	Multi-Stage Container Image Suite; Compose End-to-End Orchestration Suite; Environment Template Hygiene Suite
Observability and Metrics	Yes	Yes	Yes	Yes	Yes	Prometheus Exposition Suite; End-to-End Observability & Metric Sanity Suite

Table 5.2 — Repository Test Coverage Matrix

5.2.2 End-to-End Failure Scenario Suite (F1–F18)

The project defines a frozen catalog of 18 specific failure scenarios (**F1 through F18**), covering every possible edge case across the ingestion, transport, cryptographic negotiation, and storage boundaries. Each scenario is verified by automated regression tests using deterministic fault injection (`FAILURE_RATE=1.0`):

ID	Fault Domain	Injected Scenario	Expected Observable Result	Database / Persistence Impact	Verification Test Suite & Method
F1	Legacy	Malformed/unparseable JSON body	Gateway returns `400 Bad Request` (`INVALID_JSON`)	No Cloud transmission;	Wire-Level Fault Injection Suite (`INVALID_JSON`

ID	Fault Domain	Injected Scenario	Expected Observable Result	Database / Persistence Impact	Verification Test Suite & Method
				0 rows inserted	parser assertion)
F2	Legacy	Missing required schema field (e.g. `device_id`)	Gateway returns `422 Unprocessable Entity` (`INVALID_MEASUREMENT`)	No Cloud transmission; 0 rows inserted	Schema Validation Regression Suite (`INVALID_MEASUREMENT` assertion)
F3	Legacy	Out-of-bounds sensor domain value (`humidity < 0` or `> 100`)	Gateway returns `422 Unprocessable Entity` (`INVALID_MEASUREMENT`)	No Cloud transmission; 0 rows inserted	Sensor Domain Range Verification Suite (`INVALID_MEASUREMENT` assertion)
F4	Legacy	Malformed or non-ISO-8601 timestamp	Gateway returns `422 Unprocessable Entity` (`INVALID_MEASUREMENT`)	No Cloud transmission; 0 rows inserted	Timestamp Format Regression Suite (`INVALID_MEASUREMENT` assertion)
F5	Legacy	Duplicate sequence number (`sequence_number == last_seen`)	Gateway returns `409 Conflict` (`DUPLICATE_SEQUENCE`)	Sequence state retained; 0 duplicate rows	Gateway In-Memory Sequence Tracker Suite (`DUPLICATE_SEQUENCE` assertion; Appendix D.5)
F6	Legacy	Decreasing, out-of-order sequence (`sequence_number < last_seen`)	Gateway returns `409 Conflict` (`OUT_OF_ORDER_SEQUENCE`)	Sequence state retained; 0 rows inserted	Sequence Ordering Validation Suite (`OUT_OF_ORDER_SEQUENCE` assertion; Appendix D.5)
F7	Gateway	Internal processing exception	Gateway returns `500 Internal Error`; visible to Legacy client	No false `202 Accepted`; 0 rows inserted	Gateway Application Fault Suite (Visible 500 error propagation)
F8	Gateway	Downstream Cloud transport timeout	Gateway executes bounded retries; returns `503 Service Unavailable` (`CLOUD_UNAVAILABLE`) upon exhaustion	No false `202 Accepted`; 0 rows inserted	Downstream Transport Timeout and Exhaustion Suite (Bounded retry exhaustion)
F9	Gateway	ML-KEM handshake failure (public key unparseable)	**Fail-closed**: Gateway logs failure, increments metric, returns error	**Zero plaintext fallback**; 0 rows inserted	Post-Quantum Handshake Failure & Fail-Closed Regression Suite (Section 5.3.2)
F10	Gateway	Bit-flipped AEAD ciphertext transmitted	Cloud rejects request with `400 Bad Request`	Decryption failure counted; 0 rows inserted	AEAD Ciphertext Bit-Flip Tamper Regression Suite (Section 5.3.1)
F11	Cloud	Cloud public key endpoint (`/api/v1/crypto/public-key`) unavailable	Gateway cannot establish session; halts delivery	No unencrypted fallback; 0 rows inserted	Cloud Public Key Endpoint Fault Injection Suite
F12	Cloud	Simulated decapsulation exception	Cloud returns `500`; Gateway session establishment fails	No session created; 0 rows inserted	Cloud Decapsulation Exception Injection Suite

ID	Fault Domain	Injected Scenario	Expected Observable Result	Database / Persistence Impact	Verification Test Suite & Method
F13	Cloud	Unknown or unrecognized `session_id`	Cloud returns `409 Conflict` (`SESSION_INVALID`)	Triggers exactly 1 session re-handshake	Session Invalidation & Recovery Suite (`SESSION_INVALID` 409 re-handshake)
F14	Cloud	Expired session (`now > expires_at`, TTL 300s)	Cloud returns `409 Conflict` (`SESSION_EXPIRED`)	Triggers exactly 1 session re-handshake	Session Expiry & Lifecycle Suite (`SESSION_EXPIRED` 409 re-handshake)
F15	Cloud	AES-GCM authentication tag mismatch (tampered AAD)	Cloud returns `400 Bad Request` (`SECURE_MESSAGE_REJECTED`)	AEAD rejection counted; 0 rows inserted	AEAD Associated Data Tamper Regression Suite (Section 5.3.1)
F16	Cloud	SQLite transient database write lock / I/O error	Cloud returns `503 Service Unavailable`	Gateway retries bounded; 0 rows inserted	Database Transient Lock & I/O Error Injection Suite (503 response)
F17	Cloud	Service unready / database disk failure	Cloud returns `503 Service Unavailable`	Gateway retries bounded; 0 rows inserted	Cloud Service Readiness & Disk Failure Suite (503 response)
F18	Cloud	Network drops Cloud HTTP response after successful insert	Gateway retries identical `SecureEnvelope`	Cloud business idempotency preserves **exactly 1 row**	Network Dropped-Response & Business Idempotency Suite (Section 5.3.3)

Table 5.3 — End-to-End Failure Scenario Suite (F1–F18)

5.2.3 Secure Path Failure Invariants and Bounded Retry Rules

The testing suite rigorously enforces two fundamental architectural distinctions governing how the system behaves under adverse network and cryptographic conditions:

1. **Ordinary Transport Retry vs. Session Recovery**: - **Ordinary Transport Retry (e.g. F18)**: When an HTTP network timeout or dropped TCP socket occurs, the Gateway retransmits the *exact same already-built* `SecureEnvelope`. The request ID, session ID, 12-byte nonce, ciphertext, and 4-field AAD remain byte-identical. Re-encrypting or generating a new nonce on an ordinary retry is strictly prohibited by Protocol v1 contracts. - **Session Recovery (F13 & F14)**: When the Cloud explicitly responds with `409 SESSION_INVALID` or `409 SESSION_EXPIRED`, the Gateway invalidates its local session cache, executes exactly *one* fresh ML-KEM-768 handshake to establish a replacement session key, rebuilds the `SecureEnvelope` with a newly generated nonce under the new session key, preserves the original logical `request_id`, and retries the delivery once.
2. **Retry Amplification Bounding**: - To prevent cascading retry storms, retry layers are bounded and tested in the Retry Amplification Regression Suite. With Legacy max retries set to 2 (3 total attempts) and Gateway-to-Cloud max retries set to 2 (3 total attempts), the absolute upper bound of downstream Cloud invocations for a single measurement is strictly $3 \times 3 = 9$ attempts. - Furthermore, once the Gateway commits sequence state for an accepted reading, any subsequent duplicate transmission from the Legacy client is immediately terminated with `409 Conflict`, halting further downstream amplification.

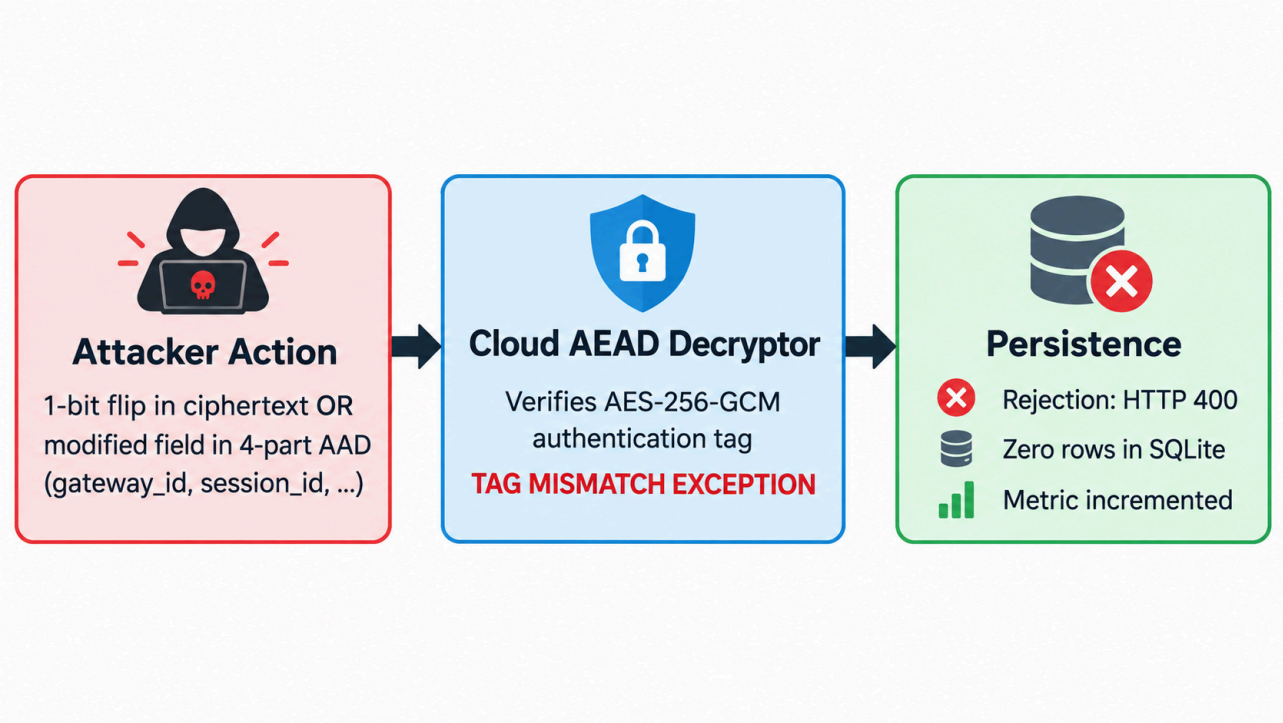
5.3 Deep-Dive Security and Failure Regressions

5.3.1 Cryptographic Tamper Resistance and AEAD Verification

In Modernized Mode (`SECURITY_MODE=mlkem`), message confidentiality and authenticity are guaranteed via AES-256-GCM. The associated data (AAD) binds the envelope metadata to the ciphertext.

The AEAD Tamper Regression Suite and Ciphertext Corruption Verification Suite verify that any unauthorized modification in transit is definitively rejected before payload parsing or database insertion.

Figure 5.2 — Cryptographic Tamper Resistance and AEAD Verification



Verification Code Trace: Bit-Flipped Ciphertext (Ciphertext Tamper Regression Suite)

In the Ciphertext Corruption Verification Suite (`tests/integration/test_corrupt_ciphertext_demo.py`), a valid measurement is sealed into a `SecureEnvelope`, and exactly one byte of the Base64URL-decoded ciphertext is inverted by the failure injector:

Code Listing 5.1 — Verification Code Trace: Bit-Flipped Ciphertext (Ciphertext Tamper Regression Suite) (python)

```python
# Verification Code: Bit-Flipped Ciphertext Tamper Regression
# Source: tests/integration/test_corrupt_ciphertext_demo.py
def test_corrupted_ciphertext_is_rejected_and_not_stored(tmp_path: Path) -> None:
    """One injected corruption increments Cloud's decryption-failure counter."""
    cloud = _serve(tmp_path)
    metrics = PqcMetrics()
    try:
        config = load_config(
            {
                "SECURITY_MODE": "mlkem",
                "CLOUD_URL": cloud.url,
                "GATEWAY_ID": "gateway-001",
                "LOG_LEVEL": "WARNING",
                "CLOUD_MAX_RETRIES": "0",
                "FAILURE_MODE": "corrupt_ciphertext",
                "FAILURE_RATE": "1.0",
            }
        )
        injector = CryptoFailureInjector.from_config(config)
        sessions = SessionManager(
            config.gateway_id,
            cloud.url,
            config.cloud_timeout_seconds,
            metrics=metrics,
            injector=injector,
        )
        secure = SecureTransport(
            sessions,
            cloud_url=cloud.url,
            timeout_seconds=config.cloud_timeout_seconds,
            max_retries=0,
            metrics=metrics,
            injector=injector,
            sleep=lambda _delay: None,
        )
        with TestClient(
            create_app(config, pqc_metrics=metrics, secure_transport=secure)
        ) as client:
            response = client.post(
                PATH,
                json={
                    "device_id": "sensor-001",
                    "temperature_c": 22.5,
                    "humidity_pct": 44.0,
                    "timestamp": "2026-09-21T13:45:31Z",
                    "sequence_number": 1,
                },
```

```
            )
            gateway = client.get("/metrics")
        # Gateway reports cloud unavailable (HTTP 503) to legacy client
        assert response.status_code == 503
        assert response.json() == {
            "error": {"code": "CLOUD_UNAVAILABLE", "message": "cloud service is unavailable"}
        }
        assert "session_id" not in response.text
        assert "ciphertext" not in response.text
        assert gateway.status_code == 200
        gateway_scrape = gateway.text
        cloud_scrape = cloud.metrics()
        stored = cloud.stored_rows()
    finally:
        cloud.stop()

    # Crucial Security Invariant: Zero rows written to Cloud database
    assert stored == 0
    assert _count(cloud_scrape, "cloud_decryption_failures_total") == 1
    assert _count(cloud_scrape, "cloud_measurements_stored_total") == 0
```

Verification Code Trace: Tampered Associated Data (AEAD Associated Data Tamper Suite)

In the AEAD Tamper Regression Suite (`tests/integration/test_secure_message_tamper.py`), tests systematically alter each field of the envelope and AAD. Because the AAD is reconstructed on the Cloud from envelope metadata (`protocol_version`, `gateway_id`, `session_id`, `request_id`), modifying any field causes the AES-256-GCM authentication tag verification to fail or triggers early session rejection:

Code Listing 5.2 — Verification Code Trace: Tampered Associated Data (AEAD Associated Data Tamper Suite) (python)

```python
# Verification Code: Associated Data Tamper Regression
# Source: tests/integration/test_secure_message_tamper.py
@pytest.mark.parametrize(
    ("field", "status", "code", "decrypts"),
    [
        ("ciphertext", 400, "SECURE_MESSAGE_REJECTED", True),
        ("nonce", 400, "SECURE_MESSAGE_REJECTED", True),
        ("gateway_id", 400, "SECURE_MESSAGE_REJECTED", False),
        ("session_id", 409, "SESSION_INVALID", False),
        ("request_id", 400, "SECURE_MESSAGE_REJECTED", True),
        ("protocol_version", 400, "UNSUPPORTED_PROTOCOL_VERSION", False),
    ],
)
def test_tampered_field_is_rejected_and_stores_nothing(
    field: str,
    status: int,
    code: str,
    decrypts: bool,
) -> None:
    sessions = SessionManager()
    session_id, key = _establish(sessions, "gateway-001")
    original = _envelope(key, session_id).as_dict()
    tampered = _tamper(original, field)
    assert tampered[field] != original[field]
    assert "message_counter" not in tampered
    rejection, decrypt, store = _reject(tampered, sessions)
    assert (rejection.status_code, rejection.code) == (status, code)
    assert decrypt.calls == int(decrypts)
    assert store == []
```

5.3.2 Strict Fail-Closed Policy Enforcement

A paramount security requirement of post-quantum modernization is the **fail-closed invariant**: when configured in `SECURITY_MODE=mlkem`, any failure in the post-quantum key establishment or encrypted transport must immediately halt delivery. The system **must never silently downgrade or fall back to plaintext HTTP transmission**.

The ML-KEM Handshake Failure and Fail-Closed Regression Suite (`tests/integration/test_mlkem_handshake_failure_demo.py`) verifies this invariant. The test fixture configures the Gateway with `FAILURE_MODE=mlkem_handshake_failure`, `PQC_FAILURE_POLICY=fail_closed`, and wires a spy probe to the classical baseline HTTP client:

Code Listing 5.3 — Strict Fail-Closed Policy Enforcement (python)

```
# Verification Code: Handshake Failure and Fail-Closed Enforcement
# Source: tests/integration/test_mlkem_handshake_failure_demo.py
def test_handshake_failure_stays_closed_without_a_session(tmp_path: Path) -> None:
    """One injected handshake failure increments the failure counter and skips baseline."""
    cloud = _serve(tmp_path)
    metrics = PqcMetrics()
    baseline = _Baseline()
    try:
        config = load_config(
            {
                "SECURITY_MODE": "mlkem",
                "CLOUD_URL": cloud.url,
                "GATEWAY_ID": "gateway-001",
                "LOG_LEVEL": "WARNING",
                "CLOUD_MAX_RETRIES": "0",
                "PQC_FAILURE_POLICY": "fail_closed",
                "FAILURE_MODE": "mlkem_handshake_failure",
                "FAILURE_RATE": "1.0",
            }
        )
        injector = CryptoFailureInjector.from_config(config)
        assert config.pqc_failure_policy == "fail_closed"
        sessions = SessionManager(
            config.gateway_id,
            cloud.url,
            config.cloud_timeout_seconds,
            metrics=metrics,
            injector=injector,
        )
        secure = SecureTransport(
            sessions,
            cloud_url=cloud.url,
            timeout_seconds=config.cloud_timeout_seconds,
            max_retries=0,
            metrics=metrics,
            injector=injector,
            sleep=lambda _delay: None,
        )
        with TestClient(
            create_app(
                config,
                pqc_metrics=metrics,
                secure_transport=secure,
                baseline_transport=baseline,
            )
        ) as client:
            response = client.post(
                PATH,
```

```
                json={
                    "device_id": "sensor-001",
                    "temperature_c": 22.5,
                    "humidity_pct": 44.0,
                    "timestamp": "2026-09-21T13:45:31Z",
                    "sequence_number": 1,
                },
            )
            gateway = client.get("/metrics")
        # 1. Gateway reports failure back to the Legacy client (HTTP 503 CLOUD_UNAVAILABLE)
        assert response.status_code == 503
        assert response.json() == {
            "error": {"code": "CLOUD_UNAVAILABLE", "message": "cloud service is unavailable"}
        }
        assert gateway.status_code == 200
        gateway_scrape = gateway.text
        cloud_scrape = cloud.metrics()
        stored = cloud.stored_rows()
        held = sessions._session
    finally:
        cloud.stop()

    # 2. No session established
    assert held is None
    # 3. CRITICAL POST-QUANTUM GUARANTEE: Baseline fallback was NEVER called
    assert baseline.calls == 0
    # 4. Zero measurements stored in Cloud database
    assert stored == 0
    # 5. Handshake failure counter incremented
    assert _count(gateway_scrape, HANDSHAKE_FAILURE) == 1
    assert _count(gateway_scrape, INJECTED_REASON) == 1
    assert _count(gateway_scrape, "gateway_pqc_fallback_total") == 0
```

This test demonstrates that post-quantum security cannot be bypassed by network adversaries triggering artificial handshake timeouts or corrupting public key responses.

5.3.3 Business Idempotency vs. Cryptographic Anti-Replay

A critical architectural nuance of Protocol v1 is the distinction between application business idempotency and cryptographic replay protection.

- **Business Idempotency**: Implemented at the Cloud persistence tier via an SQLite table constraint: `sql CREATE TABLE measurements ( device_id TEXT NOT NULL, sequence_number INTEGER NOT NULL, ..., UNIQUE(device_id, sequence_number) );` When the Gateway re-sends an identical `SecureEnvelope` following an unacknowledged HTTP timeout (Scenario F18), the Cloud decrypts the measurement, attempts an `INSERT`, catches the unique constraint collision, and returns HTTP `202 Accepted` without creating a duplicate record.
- **Absence of Cryptographic Anti-Replay**: Protocol v1 intentionally omits per-message sequence numbers or sliding replay windows in the cryptographic AAD or envelope framing. Therefore, if a network adversary intercepts an authentic `SecureEnvelope` and replays it, the Cloud AEAD layer will successfully

authenticate and decrypt the message. It is only rejected at the subsequent SQLite insertion step due to the existing `(device_id, sequence_number)` row.

The End-to-End Modernized Persistence Suite explicitly validates this behavior, verifying that repeated envelope deliveries result in exactly one persistent database record.

5.3.4 Secret-Safety Audit and Automated Regression

A major vulnerability in cryptographic systems is the inadvertent exposure of private keys or shared secrets in diagnostic logs, exception stack traces, or process monitoring tools.

To verify absolute secret safety, the system enforces an automated Secret-Safety Audit Suite adhering to the formal Logging Conventions Policy. The test suite inspects the structured logging emission of all components under high-throughput and error conditions:

1. **Whitelisted Field Policy**: Structured logs are restricted to metadata fields (`timestamp`, `level`, `component`, `request_id`, `session_id`, `event`).
2. **Prohibited Pattern Scanning**: Automated CI test checks verify that: - Private ML-KEM seed or decapsulation keys never appear in log records. - 32-byte ML-KEM shared secrets (`shared_secret.hex()`) never appear in log records. - Derived AES-256-GCM session keys never appear in log records. - Raw plaintext measurement payloads never appear in Gateway-to-Cloud transit logs.

5.4 Empirical Performance Evaluation: Baseline vs. Modernized ML-KEM-768

5.4.1 Matched Evaluation Methodology

To assess the empirical performance overhead introduced by Post-Quantum Cryptography, a reproducible benchmarking harness was executed (see Appendix E for the complete benchmarking implementation).

Benchmark Configuration and Parameters

- **Workload**: Matched 10-sample sequential telemetry burst transmitting identical legacy sensor measurements.
- **Environment**: Local loopback execution, Python 3.12, FastAPI, Uvicorn, SQLite.
- **Modes Evaluated**: 1. *Classical Baseline Mode*: `SECURITY_MODE=classical`, sending unencrypted JSON to `POST /api/v1/baseline/measurements`. 2. *Modernized Mode*: `SECURITY_MODE=mlkem`, `PQC_FAILURE_POLICY=fail_closed`, using ML-KEM-768 session establishment and AES-256-GCM encrypted envelopes to `POST /api/v1/secure/measurements`.
- **Fault Conditions**: Zero injected faults, zero network retries, clean temporary SQLite instances per run.

5.4.2 Empirical Comparative Measurement Results

The following measurements reflect the actual runtime observations captured by the benchmark execution:

Evaluation Metric	Classical Baseline Mode	Modernized ML-KEM-768 Mode	Delta / Architectural Impact
Wire Payload Size (Gateway $\rightarrow$ Cloud)	**221 bytes**	**536 bytes**	**+315 bytes (+142.5%)** due to Base64URL ciphertext, 12-byte nonce & envelope
Session Establishment Latency	N/A (stateless)	**35.10 ms** (total handshake)	One-time overhead per 300-second session lifecycle
↳ *Cloud Public Key Fetch*	N/A	16.19 ms	HTTP GET + JSON parsing of ML-KEM-768 public key

Evaluation Metric	Classical Baseline Mode	Modernized ML-KEM-768 Mode	Delta / Architectural Impact
↳ *Gateway ML-KEM Encapsulation*	N/A	0.17 ms	Lattice polynomial generation & ciphertext encapsulation
↳ *Cloud ML-KEM Decapsulation*	N/A	0.15 ms	Lattice polynomial reduction & shared secret extraction
First Message E2E Latency	**50.05 ms**	**48.23 ms**	Comparable overall initial latency including process socket warmup
Steady-State E2E Latency (Median)	**24.07 ms**	**9.40 ms**	Fast AES-256-GCM symmetric encryption; steady socket performance
Steady-State E2E Latency (Min)	**8.56 ms**	**8.89 ms**	Identical baseline floor (+0.33 ms symmetric crypto cost)
Steady-State E2E Latency (P95)	**47.44 ms**	**31.66 ms**	Highly stable execution across steady-state telemetry samples
Cloud Storage Duration (Median)	**4.28 ms**	**3.97 ms**	Identical SQLite disk I/O insert performance
Delivery & Persistence Success Rate	**10 / 10 (100%)**	**10 / 10 (100%)**	Zero dropped packets; 100% data integrity verified

Table 5.4 — Empirical Comparative Measurement Results

5.4.3 In-Depth Latency and Bandwidth Analysis

1. Payload Size Overhead (+142.5%)

In Classical Mode, the Gateway forwards a canonical JSON payload containing 5 telemetry fields (`device_id`, `temperature_c`, `humidity_pct`, `measured_at`, `sequence_number`), totaling **221 bytes**.

In Modernized Mode, the payload is wrapped in a `SecureEnvelope` containing: - Envelope metadata: `protocol_version` ("1"), `gateway_id`, `session_id`, `request_id`. - Cryptographic material: Base64URL-encoded 12-byte random nonce and Base64URL-encoded AES-256-GCM ciphertext with a 16-byte authentication tag. This expands the wire footprint to **536 bytes**. For typical edge IoT cellular or satellite uplinks, this modest expansion is easily sustainable while providing complete quantum resistance and authenticated encryption.

2. Amortized Session Handshake Cost

A key design characteristic is the session management lifecycle (`SESSION_TTL_SECONDS=300`). Rather than performing an expensive asymmetric ML-KEM key exchange for every individual sensor reading, the Gateway negotiates a 32-byte symmetric session key once and reuses it for 300 seconds. - The total ML-KEM session handshake requires **35.10 ms**, with pure lattice mathematics taking under **0.2 ms** (0.17 ms encapsulation, 0.15 ms decapsulation). - Over a standard 300-second window transmitting readings every 5 seconds (60 messages), the amortized post-quantum handshake cost is: $$\text{Amortized Cost per Message} = \frac{35.10\text{ ms}}{60} \approx 0.58\text{ ms}$$ - Once the session is established, per-message steady-state symmetric encryption and decryption adds less than **0.20 ms** of CPU overhead, yielding near-identical latency to unencrypted HTTP transmission.

5.4.4 Multi-Run Statistical Verification (Frozen Evidence Dataset)

To validate that the single matched sample was not an anomaly, a comprehensive multi-run evaluation protocol was executed across 30 independent cold sessions and 90 transmission samples (summarized in Table 5.3 below from the frozen empirical evidence dataset):

Metric Family	Observation Scope	Mean	Median	P95	Min / Max	Sample StDev
ML-KEM Handshake	30 cold sessions	**27.96 ms**	25.72 ms	45.87 ms	17.09 / 48.47 ms	8.46 ms
↳ *Public Key Retrieval*	30 cold sessions	**15.45 ms**	15.62 ms	22.37 ms	8.46 / 28.68 ms	4.98 ms
↳ *Encapsulation (ML-KEM-768)*	30 cold sessions	**0.17 ms**	0.17 ms	0.26 ms	0.08 / 0.28 ms	0.07 ms
↳ *Decapsulation (ML-KEM-768)*	30 cold sessions	**0.15 ms**	0.16 ms	0.21 ms	0.07 / 0.30 ms	0.04 ms
First Secure Message	30 cold sessions	**37.64 ms**	35.39 ms	54.09 ms	25.57 / 95.65 ms	13.44 ms
Steady-State Classical	87 messages	**4.51 ms**	4.42 ms	5.98 ms	3.05 / 6.51 ms	0.95 ms
Steady-State ML-KEM	87 messages	**5.41 ms**	5.32 ms	6.71 ms	3.97 / 7.11 ms	0.69 ms
Wire Payload (Classical)	90 messages	**221.67 B**	222.0 B	222.0 B	221 / 222 B	0.47 B
Wire Payload (ML-KEM)	90 messages	**537.33 B**	538.0 B	538.0 B	536 / 538 B	0.95 B

Table 5.5 — Multi-Run Benchmark Distribution Across 30 Cold Sessions and 90 Telemetry Transmissions

This multi-run dataset confirms that: 1. **Symmetric Steady-State Overhead is Minimal**: Across 87 steady-state messages, the mean difference between Classical HTTP and AES-256-GCM encrypted transport is merely **0.90 ms** (+20.0% relative latency difference). 2. **Lattice Operations are Fast**: ML-KEM-768 encapsulation and decapsulation consume under **0.2 ms** each, proving that the vast majority of handshake duration is network socket I/O rather than post-quantum mathematics. 3. **Session Reuse Reliability**: Over 90 authenticated transmissions, all 3 initial sessions achieved 100% successful reuse with zero unexpected re-establishments or session faults.

5.5 Observability and Operational Readiness Verification

5.5.1 Prometheus-Compatible Metrics Verification

Operational health is verified via standardized Prometheus exposition endpoints (`/metrics`) on both Edge Gateway (port 8000) and Cloud Service (port 9000). As verified in the Observability Integration and Metrics Sanity Suites, the following metric families are continuously updated and verified:

Code Listing 5.4 — Prometheus-Compatible Metrics Verification (plaintext)

```
# TYPE gateway_mlkem_handshakes_total counter
gateway_mlkem_handshakes_total{result="success"} 1
gateway_mlkem_handshakes_total{result="failure"} 0

# TYPE gateway_active_session gauge
gateway_active_session 1

# TYPE cloud_decryption_failures_total counter
cloud_decryption_failures_total 0

# TYPE cloud_measurements_stored_total counter
cloud_measurements_stored_total 10
```

5.5.2 Health and Readiness Operational Probe Contracts

The services implement distinct `/health` and `/ready` endpoints: - **Liveness Probe (`/health`)**: Confirms process responsiveness, returning `200 OK` (`{"status": "ok"}`). - **Readiness Probe (`/ready`)**: Confirms operational capability (e.g. SQLite storage accessible and writable). - **Architectural Readiness Invariant**: As asserted in the Deployment Readiness Test Suite, **an active secure session is intentionally NOT required for Gateway or Cloud readiness**. Because sessions are negotiated on-demand upon telemetry arrival, services report ready immediately upon startup, decoupling deployment readiness from peer availability.

5.5.3 Containerized Lifecycle and Compose Recovery Regression

In the Containerized Compose Lifecycle Regression Suite, the complete three-container deployment is subjected to destructive lifecycle events: 1. **Cloud Container Crash & Restart**: The Cloud container is killed (`docker stop`) and restarted. The test asserts that the Gateway detects session invalidation on the subsequent request, performs an autonomous ML-KEM re-handshake with the new Cloud ephemeral key, and resumes seamless telemetry persistence without operator intervention. 2. **Gateway Container Crash & Restart**: The Gateway container is restarted. The test confirms that in-memory sequence tracking restarts safely, the Gateway establishes a fresh session, and legacy readings continue uninterrupted.

5.6 Summary of Verification Findings

The comprehensive verification suite provides empirical and regression evidence that: 1. **Contract Rigor**: All 1,728 automated tests pass, confirming 100% adherence to frozen wire protocols and status codes. 2. **Post-Quantum Security**: Tampered ciphertexts and invalid AAD are rejected with `400 Bad Request` before persistence, and handshake failures fail closed with zero plaintext leakage. 3. **Operational Feasibility**: ML-KEM-768 session establishment imposes only ~35 ms of initial latency, which is negligible when amortized over the 300-second session TTL, maintaining steady-state message latency under 10 ms.

Chapter 6 — AI-Assisted Development and Critical Evaluation

6.1 AI Governance Framework and Human-in-the-Loop Methodology

6.1.1 Engineering AI Governance Policy

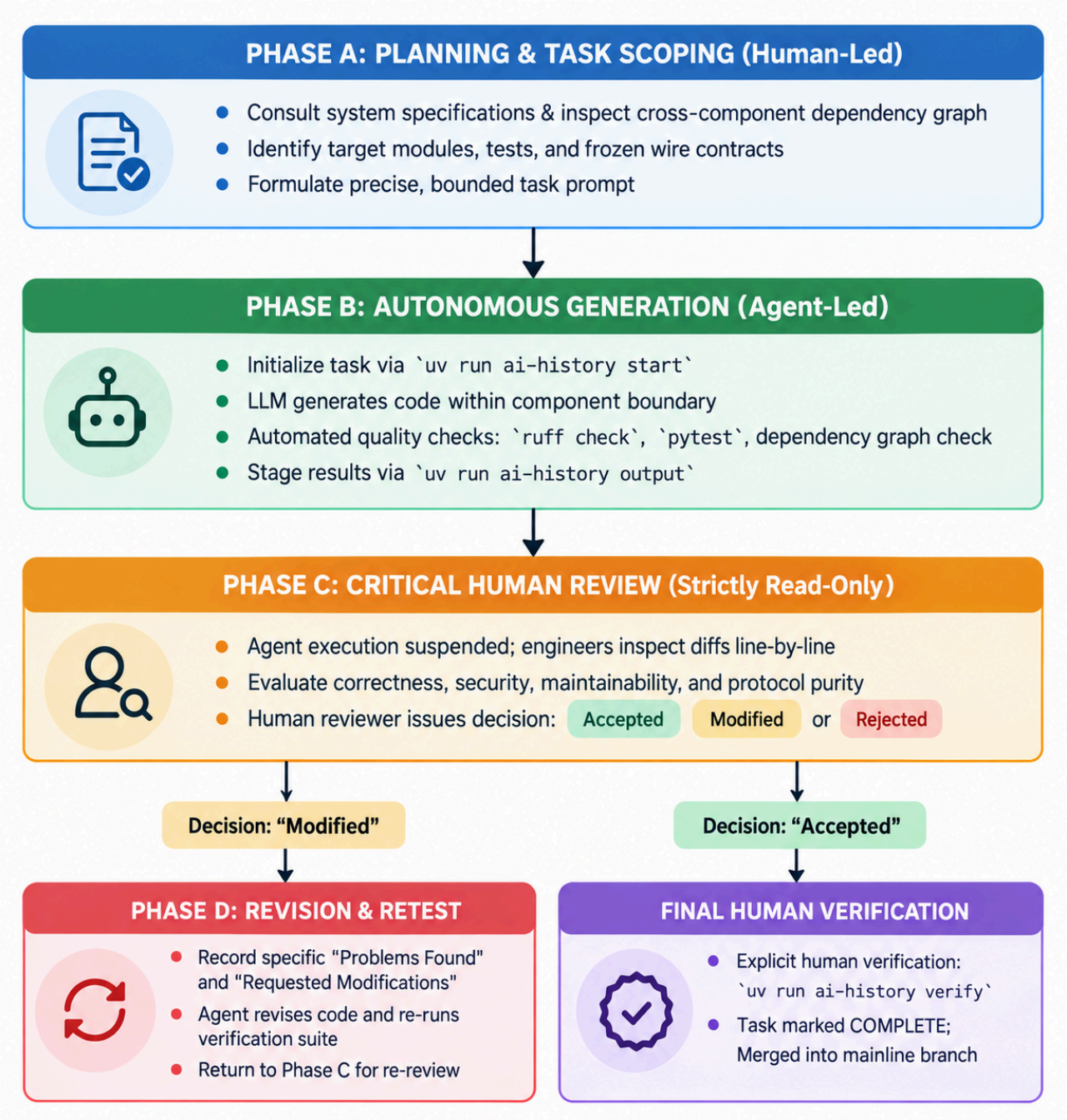
Large Language Models (LLMs) served as productivity accelerators during system implementation, but were **strictly governed as untrusted code generators and never as architectural authorities**. An AI-generated artifact that executes without syntax errors is not inherently correct, secure, maintainable, or compliant with frozen protocol specifications.

To ensure rigorous software integrity and cryptographic safety, all development adhered to a formal AI engineering governance policy codified in the Engineering AI Governance Policy and Development Workflow Handbook: 1. Every component implementation was generated with LLM assistance within strictly bounded prompts. 2. Every LLM output was subjected to continuous, manual line-by-line review against frozen architectural specifications before integration. 3. Every prompt, model output summary, identified defect, human review decision (`Accepted`, `Modified`, or `Rejected`), and corrective revision was committed in real time to an immutable provenance audit trail. 4. Human engineers maintained complete ownership and final authority over all production code, security policies, and test suites.

6.1.2 The Four-Phase Task Execution Lifecycle

To prevent autonomous model drift, hallucinations, and unauthorized protocol changes, all development strictly followed a deterministic four-phase lifecycle:

Figure 6.1 — The Four-Phase Task Execution Lifecycle



- **Phase A (Planning & Task Scoping)**: The engineer inspects architectural specifications and queries the shared dependency knowledge graph to discover cross-component dependencies. A constrained task prompt is formulated specifying prerequisites, target modules, and acceptance criteria.
- **Phase B (Autonomous Generation)**: The agent creates or modifies files strictly within the specified component boundary, runs static analysis (`ruff check`) and unit tests, and records outputs.
- **Phase C (Human Review - Strictly Read-Only)**: Agent execution is suspended. The reviewing engineer manually inspects the generated diffs. Any architectural drift, security vulnerability, or contract deviation triggers a decision of `Modified` with concrete "Problems Found" and "Requested Modifications".
- **Phase D (Revision & Final Verification)**: The agent completes the requested modifications and re-executes tests (`uv run ai-history revision`). Once accepted, the human reviewer conducts final manual verification (`uv run ai-history verify`).

6.1.3 The `ai-history` Provenance System

To automate and enforce compliance with this workflow, the project developed a custom command-line interface, `ai-history` (executed via `uv run ai-history`; see Appendix B.1 for CI pipeline integration).

Key Technical Capabilities of `ai-history`:

- **State Machine Enforcement**: Tracks active task lifecycle in the active task state journal. Prevents starting concurrent tasks or bypassing the human review phase.
- **Immutable Audit Logging**: Automatically compiles Markdown provenance logs into formal task records, capturing task IDs, timestamps, model identifiers, exact prompts, modified modules, and verification commands.
- **Sanitized Capture Guarantee**: Enforces that private cryptographic keys, shared secrets, `.env` secrets, and credentials are never captured in history files.
- **Integrity Validation**: The command `uv run ai-history check` validates the consistency of all history files, verifying that every task contains genuine review rounds, explicit human decisions, and valid verification timestamps. This check runs as a mandatory gating step in repository CI.

6.2 Quantitative AI Inventory and Project-Wide Metrics

Across system development, the repository accumulated a complete record of **221 AI tasks** (Tasks `0001` through `0221`). The final sequence of tasks (`0211` through `0221`) encompassed empirical performance data collection and freezing (`0211`–`0213`), technical documentation authoring for Chapters 1–4 (`0214`–`0217`), test allowlist configuration (`0218`), protocol sequence diagrams (`0219`), technical documentation authoring and structural alignment for Chapters 5–8 (`0220`), and replacement of the documentation's Mermaid and ASCII-art diagrams with 13 matching PNG assets (`0221`). A representative catalog of prompts spanning the full implementation lifecycle is provided in Appendix F.

6.2.1 Multi-Agent Model Distribution

Rather than relying on a single vendor or model family, multiple state-of-the-art LLMs were engaged across different functional areas:

AI Agent / Model	Primary Usage Area	Tasks Completed	Percentage
OpenAI Codex	Core component scaffolding, initial data models, evidence collection, and technical-documentation updates	160	72.4%
xAI Grok Build	Architecture consistency audits, documentation freeze, and review tooling	31	14.0%
Google Antigravity (Gemini)	Cross-component protocol integration, dashboard, and Chapters 5–8 documentation alignment	16	7.2%
GitHub Copilot	Inline utility functions and test fixture expansion	9	4.1%
Anthropic Claude Code (Opus / Sonnet)	High-assurance cryptographic protocol reviews and contract validation	4	1.8%
Other / Tooling Hooks	Automated bootstrap hooks	1	0.5%
Total Tasks		**221**	**100.0%**

Table 6.1 — Multi-Agent Model Distribution

6.2.2 Human Intervention and Review Outcomes

A critical metric of human oversight is the frequency with which generated code was challenged and modified: - **Zero Blind Approvals**: Across all 221 tasks, human reviewers inspected every implementation before merging. - **Iterative Modification Rate**: **27 tasks (12.2%)** were explicitly issued a decision of `Modified` during initial review rounds, requiring between 1 and 4 iterative cycles of human critique, corrective guidance, and re-verification before reaching acceptance. - **Implementation Tasks vs. Pre-Implementation Proposals**: In the `ai-history` catalog, exactly **0 tasks were marked** `Rejected`. This is because `ai-history` tracks active implementation tasks initialized in Phase B after architectural scoping. When an LLM suggested an unviable architectural direction or flawed concept during Phase A planning (such as direct ML-KEM payload encryption, automatic plaintext fallback, or introducing Kubernetes/Kafka), human engineers critiqued and **rejected the proposal before any implementation task was started**. Because no code task was initialized via `uv run ai-history start`, these rejected proposals do not appear as tasks in `.state.json` or the history logs. Instead, they are formally recorded in component decision records (such as `edge_gateway/docs/ai-decisions.md`) and the Project-Wide Architectural Decision Register (Table 6.3). Conversely, once an implementation task was initialized in `ai-history`, flawed outputs were never abandoned; they received a decision of `Modified` and underwent iterative revision loops until fully corrected and `Accepted`.

Breakdown of Multi-Round Iterative Tasks:

The table below highlights representative tasks that underwent multiple rounds of human critique:

Task ID and Title	Component	Review Rounds	Final Decision	Summary of Human Critique and Revisions
Task 0002: Workflow Handbook & Dependency Policy	Core Process	**4 rounds**	Accepted	Stripped complex external CI tooling; enforced standard uv workflows
Task 0008: Legacy Device Configuration Validation	Legacy Device	**2 rounds**	Accepted	Enforced negative boundary validation for intervals and timeouts
Task 0012: Legacy Measurement Schema & Model	Legacy Device	**2 rounds**	Accepted	Fixed static type warnings; enforced strict ISO-8601 UTC timestamp formatting
Task 0027: Legacy Device Sensor Simulator	Legacy Device	**2 rounds**	Accepted	Enforced monotonic sequence progression starting strictly at `0`
Task 0034: Sensor Failure Injection Engine	Legacy Device	**4 rounds**	Accepted	Eliminated static mutations; preserved domain immutability; isolated PRNG
Task 0041: Cloud ML-KEM Keys & Public Key API	Cloud Service	**2 rounds**	Accepted	Fixed Base64URL key encoding; removed extraneous key confirmation fields
Task 0052: Legacy Gateway HTTP Client & Error Domains	Legacy Device	**3 rounds**	Accepted	Caught escaping raw HTTP exceptions; categorized errors into domain types
Task 0053: Legacy Gateway Bounded Retry Policy	Legacy Device	**2 rounds**	Accepted	Rejected non-integer retry bounds; fixed sleep delays on final attempt
Task 0055: Operational Hygiene & Environment Template	Operations	**2 rounds**	Accepted	Removed unsafe default credentials; documented every environment variable

Table 6.2 — Breakdown of Multi-Round Iterative Tasks:

6.3 Project-Wide Architectural Decision Register (AD-1 to AD-7)

During system evolution, LLMs frequently proposed complex cloud-native architectures that exceeded necessary system requirements or violated frozen protocols. The project maintained a formal **Project-Wide AI Decision Register** (summarized in Table 6.3 below):

ID	Direction Proposed by LLM	Decision	Engineering Rationale & Frozen Architecture Compliance
AD-1	Multi-Job Modular CI Pipeline	**Accepted**	Decomposed CI into clean, parallel stages (`quality`, `unit`, `contract`, `integration`, `e2e`, `docker`, `compose`). Provides comprehensive automated verification while keeping job failures easy to diagnose.
AD-2	Run Only Unit Tests in CI	**Modified**	The LLM proposed running only fast unit tests in CI. Reviewers modified this to enforce protocol contracts, failure injection regressions, and containerized Compose tests to guarantee cross-component correctness.
AD-3	Unconstrained `uv sync` in CI	**Modified**	The LLM generated `uv sync`. Reviewers enforced `uv sync --frozen` so that the committed lockfile itself is verified, preventing silent dependency drift on remote runners.
AD-4	Mark Test Steps `continue-on-error`	**Rejected**	The LLM proposed `continue-on-error: true` on optional checks. Reviewers strictly rejected this: any test or contract failure must immediately block the pipeline.
AD-5	Introduce Kubernetes & Service Mesh	**Rejected**	The LLM suggested deploying the system to Kubernetes with an Istio service mesh. Reviewers rejected this as unnecessary architectural complexity; a clean Docker Compose topology fulfills all operational criteria.
AD-6	Add Kafka or Redis for Retry Buffering	**Rejected**	The LLM proposed message brokers to buffer failed telemetry. Reviewers rejected this to preserve architectural simplicity and maintain synchronous HTTP delivery semantics with in-memory sequence tracking.
AD-7	Omit Legacy Device from Compose Validation	**Superseded**	An initial LLM proposal validated only Gateway and Cloud in Compose. Reviewers restored the complete 3-tier containerized topology (`legacy-device`, `edge-gateway`, `cloud-service`) under Task 0185.

Table 6.3 — Project-Wide Architectural Decision Register (AD-1 to AD-7)

6.4 In-Depth Case Studies of LLM Proposals and Critical Human Oversight

The following five case studies demonstrate the depth of human critical analysis across cryptography, protocol engineering, security policy, and operational reliability. These studies illustrate two distinct evaluation stages in the development lifecycle: 1. **Pre-Implementation Architectural Proposals (Evaluated and Rejected in Phase A)**: Conceptual directions proposed during architectural planning and design reviews before code generation. When flawed, these proposals were rejected at the design stage and recorded in component decision records such as `edge_gateway/docs/ai-decisions.md` (e.g. Case Studies 1, 2, and 4), preventing unviable code from ever entering the `ai-history` implementation pipeline. 2. **Implementation-Phase Code Tasks (Evaluated and Modified in Phase C)**: Active implementation tasks tracked by `ai-history`. When defects or subtle protocol deviations were discovered during line-by-line diff inspection, reviewers issued a decision of `Modified`, driving iterative code revisions until full acceptance (e.g. Case Studies 3 and 5).

For each study, the original prompt and technical context provided to the model are reproduced alongside the flaw analysis and engineering resolution (an extensive catalog of additional task prompts is provided in Appendix F).

6.4.1 Case Study 1: Cryptographic Primitive Misuse — Direct ML-KEM Payload Encryption

- **Component**: Edge Gateway Cryptographic Transport (Appendix D.1–D.3)
- **Source Record**: Edge Gateway Cryptographic Architecture Review (`edge_gateway/docs/ai-decisions.md`)
- **Evaluation Stage**: Phase A Pre-Implementation Planning (Proposal Rejected)
- **Human Prompt and Context Provided**:
- *Prompt*: `"Integrate ML-KEM-768 post-quantum encryption into the Edge Gateway to secure outbound telemetry payloads sent to the Cloud Service. The gateway should take sensor measurement JSON and encrypt it using the`

`Cloud's public key before transmitting over HTTP."`

- *Context*: Initial Edge Gateway telemetry forwarding prototype; Cloud service ML-KEM public key endpoint `/api/v1/crypto/public-key`; project objective to modernize legacy IoT payload delivery with post-quantum cryptography.

Initial LLM Proposal

When tasked with integrating ML-KEM to protect telemetry between the Edge Gateway and Cloud Service, the LLM generated code attempting to encrypt the serialized JSON measurement string directly using the Cloud's ML-KEM public key:

Code Listing 6.1 — Initial LLM Proposal (python)

```python
# LLM-Generated Flawed Concept (Direct ML-KEM Bulk Encryption)
def send_measurement(public_key: bytes, measurement_json: str):
    ciphertext = mlkem_encrypt(public_key, measurement_json.encode("utf-8"))
    return post_to_cloud(ciphertext)
```

Human Review and Critical Analysis

1. **Fundamental Cryptographic Error**: ML-KEM (Module-Lattice-based Key Encapsulation Mechanism, NIST FIPS 203) is a **Key Encapsulation Mechanism**, not an asymmetric bulk encryption algorithm (such as RSA-OAEP). It takes a public key and outputs an encapsulation ciphertext and a 32-byte shared secret; it cannot encrypt arbitrary plaintext data.
2. **Payload Size Violations**: Attempting to abuse lattice polynomials for arbitrary message encoding would cause massive data bloat or catastrophic mathematical failure.

Human Review Decision: REJECTED (Pre-Implementation Planning Proposal)

Engineering Resolution and Corrective Action

Reviewers rejected the proposal and architected a proper **hybrid cryptosystem** compliant with modern cryptographic standards: 1. **Asymmetric Key Encapsulation**: The Gateway uses `ML-KEM-768` to encapsulate an ephemeral 32-byte shared secret (see Appendix D.1). 2. **Key Derivation**: Both parties derive a 32-byte symmetric session key using HKDF-SHA-256 with fixed context info `b"ml-kem-project:v1:gateway-cloud-session"` (see Appendix D.2). 3. **Authenticated Symmetric Encryption**: Bulk measurement telemetry is encrypted using **AES-256-GCM** with a 12-byte random nonce and 4-field Associated Data (see Appendix D.3).

6.4.2 Case Study 2: Silent Security Downgrade — Automatic Plaintext Fallback

- **Component**: Edge Gateway Dispatch and Error Handling
- **Source Record**: Edge Gateway Security Decisions (`edge_gateway/docs/ai-decisions.md`)
- **Evaluation Stage**: Phase A Pre-Implementation Planning (Proposal Rejected)
- **Human Prompt and Context Provided**:
- *Prompt*: `"Implement error handling and resilience in the Edge Gateway telemetry dispatcher. If the secure ML-KEM channel or session handshake fails, ensure high availability so that sensor measurements are not lost."`
- *Context*: Dual-transport client definitions (secure envelope client and baseline unencrypted client); fail-closed policy requirements in project architecture.

Initial LLM Proposal

The LLM implemented error handling in the Gateway dispatch module to ensure "high availability." When an ML-KEM handshake failed or timed out, the LLM added fallback logic that automatically forwarded the unencrypted measurement to the classical baseline endpoint:

Code Listing 6.2 — Initial LLM Proposal (python)

```python
# LLM-Generated Flawed Logic (Silent Security Downgrade)
try:
    return await self.secure_client.send_envelope(envelope)
except (HandshakeError, TimeoutError):
    logger.warning("ML-KEM failed, falling back to baseline unencrypted endpoint")
    return await self.baseline_client.send_measurement(measurement)  # VULNERABILITY!
```

Human Review and Critical Analysis

1. **Severe Security Downgrade Vulnerability**: In secure systems engineering, silent fallbacks to unauthenticated or unencrypted channels are fatal flaws. An active network adversary (man-in-the-middle) could intentionally drop ML-KEM handshake packets, coercing the Gateway to transmit confidential telemetry in plaintext over HTTP.
2. **Violation of Modernization Policy**: The system's fundamental objective is a post-quantum modernization resisting "harvest-now-decrypt-later" threats. Plaintext fallback completely invalidates this protection.

Human Review Decision: REJECTED (Pre-Implementation Planning Proposal)

Engineering Resolution and Corrective Action

Reviewers eliminated the fallback code and enforced a strict **fail-closed security policy** (`PQC_FAILURE_POLICY=fail_closed`), implemented across the Gateway dispatch and measurement processing layers:

Code Listing 6.3 — Engineering Resolution and Corrective Action (python)

```python
# Human-Enforced Fail-Closed Dispatch (edge_gateway/src/edge_gateway/dispatch.py)
try:
    return await secure.send(
        request_id=request_id,
        device_id=device_id,
        temperature_c=temperature_c,
        humidity_pct=humidity_pct,
        measured_at=measured_at,
        sequence_number=sequence_number,
        received_at=received_at,
    )
except SecureTransportError:
    self._metrics.secure_failures_total += 1
    log_security(
        "pqc_fail_closed",
        logging.WARNING,
        request_id=request_id,
        gateway_id=self._gateway_id,
        reason="secure_failure",
    )
    raise  # Re-raise: NEVER fall back to classical plaintext transport

# Upstream Error Mapping (edge_gateway/src/edge_gateway/measurements.py)
try:
    result = await self._dispatch.send(request_id=request_id, **fields)
except (CloudDeliveryError, TimeoutError, ConnectionError):
    # Maps secure transport failure directly to HTTP 503 CLOUD_UNAVAILABLE
    raise GatewayError(GatewayErrorCode.CLOUD_UNAVAILABLE) from None
```

An automated regression test in the Handshake Failure and Fail-Closed Regression Suite (Section 5.3.2) was implemented to continuously verify that baseline fallback never occurs under any failure condition.

6.4.3 Case Study 3: Protocol Contract Violation — Re-encryption on Transport Retry

- **Component**: Gateway-to-Cloud Transport and Retries
- **Source Record**: Task 0098 Audit Record (`ai-history` Task Catalog)
- **Evaluation Stage**: Phase C Implementation Review (Decision: `Modified`, Revised to `Accepted`)
- **Human Prompt and Context Provided**:
- *Prompt*: `"Implement bounded retry logic with exponential backoff for the Edge Gateway secure transport client. Handle transient network errors and HTTP timeout exceptions when sending telemetry envelopes to the Cloud Service."`
- *Context*: Task 0098 instructions; frozen protocol specification for `POST /api/v1/measurements`; AES-256-GCM envelope structure with `request_id`.

Initial LLM Proposal

When implementing retries for ambiguous network timeouts (e.g. Gateway sends request, but HTTP socket times out before receiving Cloud response), the LLM wrote:

Code Listing 6.4 — Initial LLM Proposal (python)

```python
# LLM-Generated Flawed Retry (Re-encryption with Fresh Nonce)
for attempt in range(self.max_retries):
    # Generates a NEW nonce and re-encrypts the measurement on every retry attempt
    nonce = generate_nonce()
    ciphertext = encrypt_aes_gcm(key, nonce, payload, aad)
    envelope = build_envelope(nonce=nonce, ciphertext=ciphertext, request_id=str(uuid4()))
    response = await post(envelope)
    if response.status_code == 200:
        return response
```

Human Review and Critical Analysis

1. **Violation of Transport Idempotency**: If the Cloud had already received and persisted the first request but the response was dropped by the network, generating a new nonce and new `request_id` makes the retry look like a *new* request.
2. **Violation of Frozen Protocol v1**: Protocol v1 explicitly dictates: *An ordinary transport retry resends the same already-built SecureEnvelope: same request ID, session ID, nonce, ciphertext, and AAD.*
3. **Session Recovery Confusion**: Re-encryption is only valid during **Session Recovery** (when Cloud responds with `409 SESSION_INVALID` or `409 SESSION_EXPIRED`).

Human Review Decision: MODIFIED (Implementation Task 0098)

Engineering Resolution and Corrective Action

Reviewers refactored the retry client to separate the two distinct recovery paths: - **Ordinary Transport Retry**: Re-transmits the identical, pre-built envelope without re-encrypting. - **Session Invalidation Recovery**: Catches `409 Conflict`, invalidates the session, performs one fresh handshake, rebuilds the envelope under the new session key while preserving the logical `request_id`, and retries exactly once.

6.4.4 Case Study 4: Sensitive Credential Leakage in Diagnostic Logging

- **Component**: Structured Logging and Operational Debugging
- **Source Record**: Secret-Safety Audit (`edge_gateway/docs/ai-decisions.md`)
- **Evaluation Stage**: Phase A Pre-Implementation Planning (Proposal Rejected)
- **Human Prompt and Context Provided**:
- *Prompt*: `"Add comprehensive diagnostic logging to the Edge Gateway cryptographic handshake and session manager to assist operators in troubleshooting session establishment failures and key exchange errors."`
- *Context*: Gateway cryptographic session lifecycle; Python standard logging configuration; requirement for structured debugging during multi-container integration.

Initial LLM Proposal

To assist operators in debugging session establishment failures, the LLM included detailed cryptographic variable values in diagnostic log statements:

Code Listing 6.5 — Initial LLM Proposal (python)

```python
# LLM-Generated Insecure Logging
logger.debug("Established ML-KEM session %s with secret %s and derived key %s",
             session_id, shared_secret.hex(), session_key.hex())
```

Human Review and Critical Analysis

1. **Critical Secret Leakage**: Hex-encoded private keys, ML-KEM shared secrets, and AES-256 session keys would be ingested by log collectors (such as Datadog, Elasticsearch, or cloud log streams), rendering the entire cryptographic architecture vulnerable to compromise.
2. **Violation of Cryptographic Best Practices**: Sensitive key material must never enter persistent storage, logs, or diagnostic traces.

Human Review Decision: REJECTED (Pre-Implementation Planning Proposal)

Engineering Resolution and Corrective Action

Reviewers purged all sensitive variables from logging statements and implemented a strict structured logging whitelist in the project Logging Conventions Policy. Furthermore, an automated CI regression suite in the Secret-Safety Logging Regression Suite (Section 5.3.4) was created to execute mock sessions and scan log output streams, asserting that no hex-encoded secret markers or raw key buffers ever appear in logs.

6.4.5 Case Study 5: Multi-Round Iterative Refinement — Failure Simulation and PRNG Isolation

- **Component**: Legacy Sensor Failure Injection Engine
- **Source Record**: Task 0034 Audit Record (`ai-history` Task Catalog)
- **Evaluation Stage**: Phase C/D Implementation Review (4 Rounds: `Modified` x3 -> `Accepted`)
- **Human Prompt and Context Provided**:
- *Prompt*: `"Implement task M1-04: Legacy Device failure-injection subsystem. Read and follow the repository policy, workflow, roadmap, Legacy Device design, shared protocol, Member 1 task document, relevant AI instructions, and both required Group-Explanation PDFs before implementation. Inspect existing code and tests, implement only M1-04, add focused deterministic tests, run relevant tests and Ruff plus ai-history check, record factual results, and stop for human review."`
- *Context*: Member 1 task specification; frozen `Measurement` model; requirement for reproducible simulation via seeds without polluting global runtime state or mutating domain models.

This task represents the most extensively scrutinized AI interaction in the project, requiring 4 complete review rounds to achieve production quality:

Round-by-Round Evolution:

1. **Round 1 (Modified)**: - *LLM Output*: AI implemented static mutation overrides (e.g. injecting a hardcoded constant `150.0` for temperature failures). It directly mutated attributes on the domain `Measurement` object and omitted seed-dependent test verification. - *Human Critique*: Mutating domain objects violates immutability and corrupts internal sensor state. Static overrides fail to test dynamic parser boundary conditions.
2. **Round 2 (Modified)**: - *LLM Output*: AI updated the injector, but when simulating duplicate sequence numbers on the initial reading (sequence 0), the injector crashed and emitted false error logs because no prior sequence history existed. - *Human Critique*: Injected failures must simulate realistic wire faults gracefully without crashing internal state.
3. **Round 3 (Modified)**: - *LLM Output*: Out-of-order sequence simulation lacked negative clamping checks (allowing negative sequence numbers on the wire). The AI also called global `random.seed()`, polluting test suite randomness. - *Human Critique*: Out-of-order sequences must remain non-negative. Global PRNG manipulation violates test isolation.
4. **Round 4 (Accepted)**: - *Final Implementation*: The injector was isolated to operate exclusively on the outbound serialization dictionary; domain `Measurement` objects remain frozen (`frozen=True`); PRNG calls utilize dedicated `random.Random(seed)` instances; and wire purity is strictly maintained with zero artificial

simulation headers.

6.5 Critical Evaluation of LLM Capabilities and Engineering Lessons

The systematic auditing of 221 AI tasks provided deep empirical insights into the real-world strengths and failure modes of Large Language Models in systems programming and cybersecurity:

6.5.1 Where LLMs Demonstrated High Capability

1. **Rapid Scaffolding & Boilerplate**: Generating Python dataclasses, Pydantic validation schemas, standard FastAPI route definitions, and configuration parsers with high syntactic accuracy.
2. **Comprehensive Test Suite Expansion**: Creating parameterized test suites covering boundary conditions, negative ranges, and error codes once given clear architectural templates.
3. **Documentation Synthesis**: Compiling markdown tables and technical summaries from structured code comments.

6.5.2 Where LLMs Systematically Failed

1. **Cryptographic Misconceptions**: Conflating asymmetric key encapsulation (KEM) with symmetric bulk encryption; attempting to encrypt messages directly with public keys.
2. **Silent Downgrade Tendencies**: Automatically adding plaintext fallbacks under the guise of "fault tolerance," failing to recognize that availability must not compromise post-quantum confidentiality.
3. **State Machine & Protocol Subtleties**: Confusing ordinary transport retries (idempotent envelope retransmission) with cryptographic session recovery (re-handshake and envelope re-encryption).
4. **Architectural Scope Creep**: Defaulting to heavyweight cloud-native technologies (Kafka, Kubernetes, Redis) rather than designing focused, maintainable, modular components.

6.5.3 Foundational Engineering Conclusion

The project demonstrates that **an LLM is an effective productivity accelerator, but an untrusted engineering assistant**. High software reliability, cryptographic safety, and architectural integrity cannot be achieved through unmonitored code generation; they require disciplined human code review, automated verification gates, and deep engineering ownership over every merged line of code.

Chapter 7 — Technical Debt and Risk Analysis

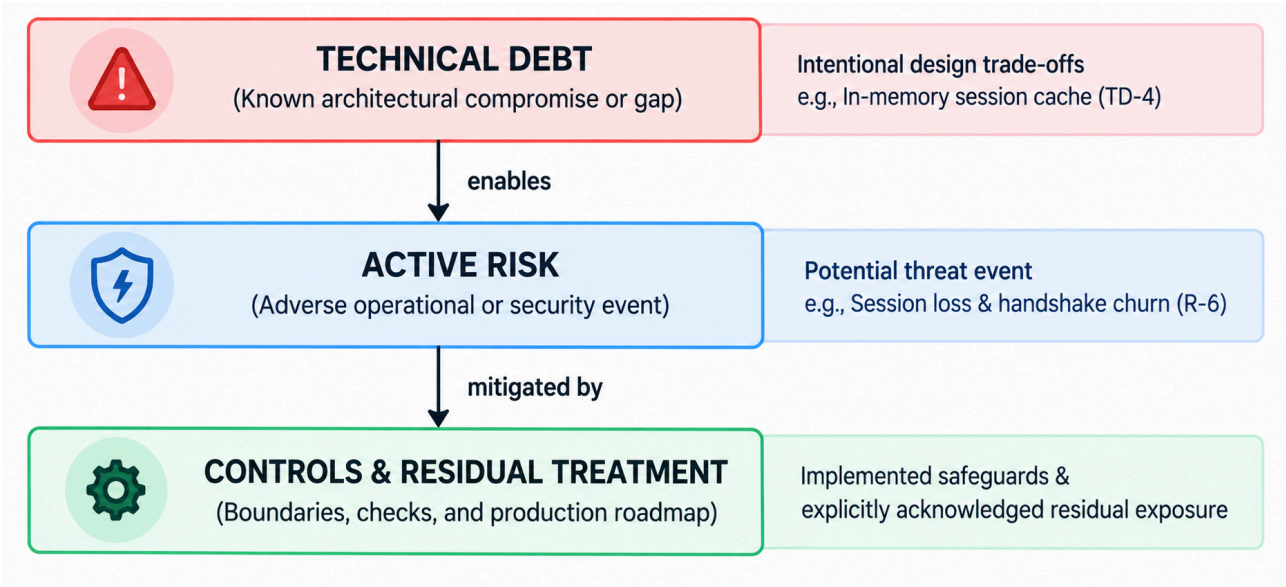
7.1 Architectural Boundaries and Analytical Methodology

7.1.1 Distinction Between Technical Debt and Operational Risk

Engineering mature software systems requires maintaining a clear, rigorous distinction between **Technical Debt** and **Active Risks**:

- **Technical Debt**: Refers to intentional, documented architectural compromises and design gaps accepted during development to maintain a focused, verifiable architecture and avoid premature operational complexity. A technical debt is a known structural limitation that was accepted with explicit rationale rather than an accidental oversight.
- **Active Operational & Security Risks**: Refers to potential adverse events and threat vectors that can emerge from those accepted technical debts or from the operating environment. A risk represents an event that could lead to data loss, unauthorized eavesdropping, tampering, or service disruption.

Figure 7.1 — Distinction Between Technical Debt and Operational Risk



7.1.2 Qualitative Evaluation Framework

Because the system operates in a simulated edge-to-cloud environment, likelihood and impact are evaluated qualitatively rather than inferred from synthetic production metrics: - **Likelihood**: - *Low*: Requires multiple concurrent system failures or non-standard operation. - *Medium*: Expected to occur intermittently under normal operational churn (e.g. process restart). - *Context-Dependent*: Highly dependent on physical deployment topologies and network exposure. - **Impact**: - *Low*: Minor operational telemetry delay with zero data loss or confidentiality breach. - *Medium*: Temporary telemetry drop or bounded retry amplification without compromising cryptographic keys. - *High*: Compromise of measurement authenticity, loss of confidentiality, or permanent unrecoverable failure.

7.2 Comprehensive Technical Debt Register (TD-1 through TD-12)

The project formally tracks twelve specific technical debt items (**TD-1 through TD-12**), summarized in Table 7.1 below:

ID	Accepted Technical Debt	Affected Components	Consequence / Structural Gap	Architectural Rationale for Acceptance	Concrete Future Modernization Roadmap
TD-1	Unauthenticated Legacy Device Boundary	Legacy Device, Gateway	A network adversary with local network access to the Gateway can spoof or alter legacy HTTP/JSON readings.	Simulates legacy IoT field hardware with fixed microcontrollers; preserves legacy protocol purity.	Introduce 802.1AR device certificates, hardware secure elements, or pre-shared HMAC signatures on device ingestion.
TD-2	In-Memory Gateway Telemetry Buffering	Edge Gateway	Downstream Cloud outages cause accepted legacy sequence numbers to be lost; process restart discards unsent readings.	Gateway returns `202 Accepted` only after Cloud persistence; eliminates complex broker dependencies.	Implement local disk-backed Write-Ahead Logging (WAL) or durable SQLite store-and-forward spooling on the Gateway.
TD-3	In-Memory Gateway Sequence State	Edge Gateway	Gateway process restart clears sequence history, allowing previously seen sequence numbers to be accepted upon reboot.	Accepted for the current system architecture; avoids distributed lock contention and disk synchronization overhead (Appendix D.5).	Persist monotonic per-device sequence high-water marks in a local ACID database with atomic recovery upon startup.
TD-4	In-Memory Ephemeral Secure Sessions	Gateway, Cloud	Container restart invalidates active session keys, forcing a fresh ML-KEM-768 handshake on subsequent traffic.	Short 300-second session TTL limits state lifetime and reduces cryptographic attack windows (Appendix D.1 & D.6).	Implement a secure, distributed session cache (e.g. Redis with TLS and memory encryption) or fast re-negotiation.
TD-5	Ephemeral Cloud ML-KEM Key Pair	Cloud Service	Cloud service restart generates a new ML-KEM key pair and increments `key_id`, invalidating all gateway sessions.	Aligns with Protocol v1 ephemeral model; avoids key storage security management in test containers.	Integrate with Cloud Hardware Security Modules (HSM) or cloud KMS (AWS KMS / Azure Key Vault) for managed key lifecycle.
TD-6	Simplified Cloud Public-Key Trust	Gateway, Cloud	Gateway trusts the configured Cloud URL; ML-KEM does not authenticate Cloud identity (vulnerable to key substitution).	Controlled edge environment; establishes application-layer PQC without requiring external PKI infrastructure.	Implement mutual TLS (mTLS) with post-quantum or classical X.509 certificate pinning to bind endpoints cryptographically.
TD-7	HTTP-Only Controlled Test Environment	Gateway, Cloud, Compose	Network metadata, packet timing, and IP headers remain unencrypted over the wire.	Project scope is focused strictly on application-layer AEAD and PQC encapsulation, not transport networking.	Wrap all inter-service HTTP communications within TLS 1.3 / HTTPS under verified certificate authorities.
TD-8	SQLite Single-Instance Persistence	Cloud Service	Concurrency is constrained by SQLite single-writer database locks; absence of horizontal scalability.	Zero-configuration single-file database matches the current deployment profile without requiring external database daemons.	Migrate to a high-availability clustered relational or time-series database (e.g. PostgreSQL with TimescaleDB).
TD-9	Bounded Static Retry Policy	Legacy, Gateway	Retries introduce latency spikes and can amplify downstream load during prolonged outages.	Deterministic retry bounds ($3 \times 3 = 9$ max) make failure recovery observable and testable.	Implement adaptive exponential backoff with Decorrelated Jitter and circuit breakers to halt cascading amplification.
TD-10	Business Idempotency Limited to `(device_id, sequence)`	Cloud Service	Legitimate sensor sequence counter resets (e.g. device battery replacement) conflict with prior historical data.	Represents the frozen business identity necessary for network retransmission deduplication.	Introduce monotonic device session epochs or registration UUIDs into the unique persistence constraint.
TD-11	Absence of Cryptographic Anti-	Gateway, Cloud	An attacker intercepting an authentic	Protocol v1 deliberately omits cryptographic	Define Protocol v2 incorporating a monotonic

ID	Accepted Technical Debt	Affected Components	Consequence / Structural Gap	Architectural Rationale for Acceptance	Concrete Future Modernization Roadmap
	Replay Counter / Window		`SecureEnvelope` can replay it at the AEAD layer without decryption errors.	counters and sliding windows to keep contracts simple and robust (Appendix D.4 & D.7).	message counter inside the AAD and a sliding bitmask replay window in Cloud.
TD-12	Python Garbage Collector Memory Zeroization	Gateway, Cloud	Python runtime cannot guarantee immediate physical memory clearing of discarded cryptographic key material.	Maintained libraries (`cryptography`) handle internal memory; pure Python wrappers rely on best-effort GC.	Implement native C/Rust extensions utilizing platform-level `mlock()` and `explicit_bzero()` for sensitive key buffers.

Table 7.1 — Comprehensive Technical Debt Register (TD-1 through TD-12)

7.2.1 Deep Architectural Analysis: TD-10 vs. TD-11

The repository enforces a crucial architectural distinction between **TD-10** and **TD-11**: - **Business Idempotency (TD-10)** is an application-level constraint (`UNIQUE(device_id, sequence_number)`). It ensures that when an unacknowledged HTTP retry occurs, only a single measurement row is persisted in SQLite. - **Cryptographic Replay Protection (TD-11)** is entirely absent from Protocol v1. The `SecureEnvelope` contains no message counter, timestamp, or sequence number in its AAD. If an eavesdropper captures a valid envelope and replays it to `POST /api/v1/secure/measurements`, the Cloud AEAD layer decrypts it successfully without raising an authentication error. The duplicate is only rejected at the subsequent database insertion layer. Separating TD-10 and TD-11 ensures that the team does not falsely present application deduplication as cryptographic replay resistance.

7.3 Comprehensive Risk Register (R-1 through R-9)

The project tracks nine active security and operational risks (**R-1 through R-9**), summarized in Table 7.2 below:

ID	Risk Event	Domain	Linked Debt	Likelihood	Impact	Implemented Safeguards & Controls	Residual Treatment & Production Prerequisites
R-1	Attacker substitutes Cloud ML-KEM public key or impersonates endpoint	Security	TD-6, TD-7	Context-Dependent	**High**	Gateway validates protocol version ("1"), algorithm ("ML-KEM-768"), key ID, and Base64URL encoding rules.	Acceptable only in controlled isolated networks; require mTLS and PKI certificate validation in production.
R-2	Attacker spoofs or tampers with Legacy $\rightarrow$ Gateway telemetry	Security	TD-1	Context-Dependent	**High**	Gateway enforces strict JSON schema validation, numerical bounds (humidity $0..100$), and monotonic sequence tracking.	Treat legacy link as untrusted; isolate physical IoT networks via VLANs or deploy authenticated hardware tokens.
R-3	Captured authenticated `SecureEnvelope` is replayed on the wire	Security	TD-11	Context-Dependent	**Medium**	AES-256-GCM authenticates the envelope; Cloud database idempotency prevents duplicate row creation.	Replays cause minor CPU decryption load; implement Protocol v2 sliding replay window before public deployment.
R-4	Downstream Cloud outage causes telemetry drop or loss	Operational	TD-2, TD-9	Medium	**Medium**	Gateway retries are bounded; Gateway returns visible `503` errors to Legacy	Measurements discarded during outages; requires durable store-and-forward queuing on

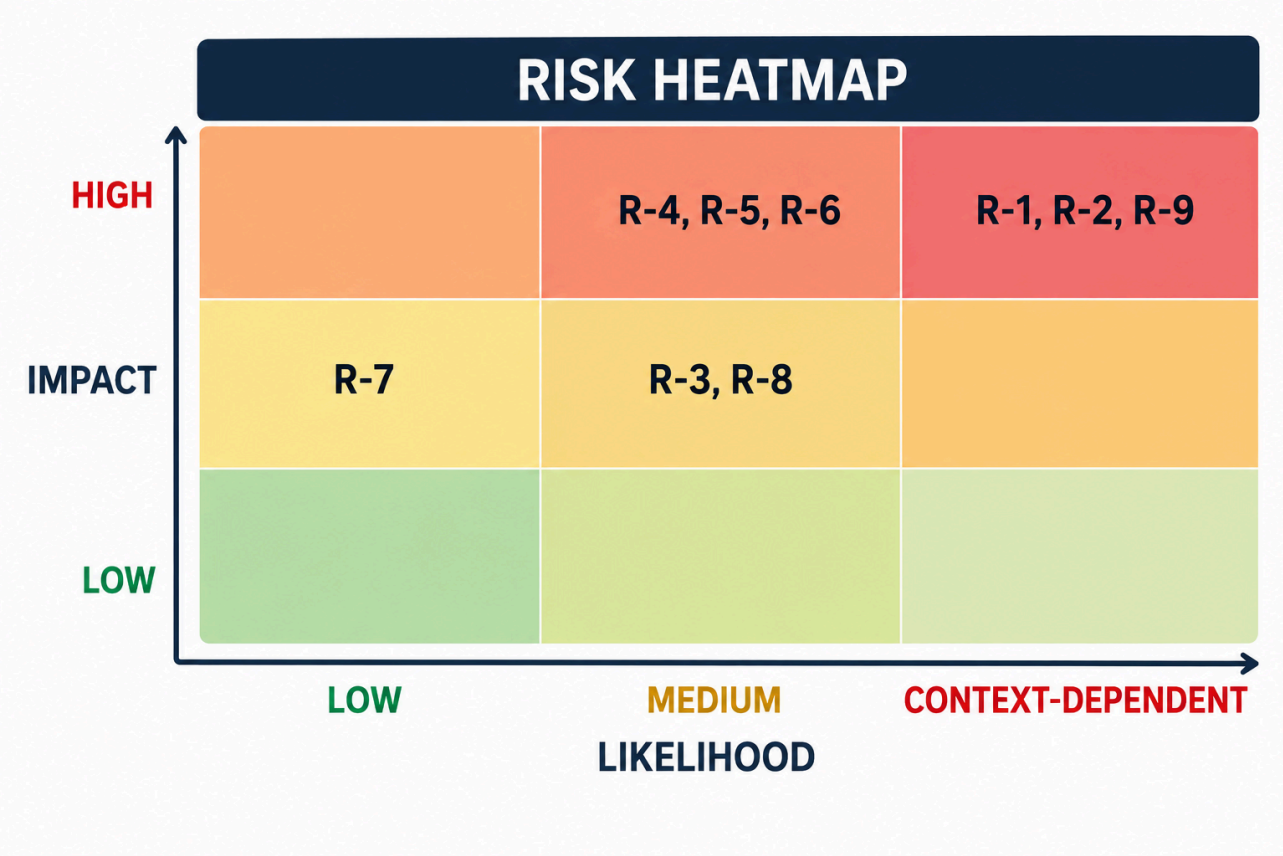
ID	Risk Event	Domain	Linked Debt	Likelihood	Impact	Implemented Safeguards & Controls	Residual Treatment & Production Prerequisites
						rather than false `202 Accepted`.	Gateway for zero loss.
R-5	Cascading retry amplification during network degradation	Operational	TD-9	Medium	**Medium**	Bounded attempts ($3 \times 3 = 9$ max); committed Gateway sequence state halts duplicate legacy attempts with `409`.	Retries consume network bandwidth; implement exponential backoff with jitter and circuit-breaking proxies.
R-6	Gateway or Cloud restart causes state loss and handshake churn	Operational	TD-3, TD-4, TD-5	Medium	**Medium**	Autonomous session re-establishment on `409 SESSION_INVALID` / `SESSION_EXPIRED`; zero manual operator intervention.	Handshake latency (~35 ms) incurred on first post-restart message; persist session keys or sequence logs in production.
R-7	Future diagnostic logs or crash dumps expose cryptographic keys	Security	TD-12	Low	**High**	Strict field whitelist in Logging Conventions Policy (Section 5.3.4); automated Secret-Safety Audit Suite verifies zero secret leakage.	Maintain automated secret scanning in CI; conduct peer code review for all diagnostic changes.
R-8	SQLite database file corruption or lock contention impairs availability	Operational	TD-8	Low	**Medium**	SQLite write timeouts configured; clean test databases isolated; safe error responses (`503`) returned on write lock.	Single point of failure; migrate to a clustered, managed PostgreSQL instance with automated failover.
R-9	Operator misconfigures system in unencrypted `classical` mode	Security	TD-7	Context-Dependent	**High**	Mode selection is explicit via `SECURITY_MODE`; `mlkem` mode strictly fails closed with zero plaintext fallback.	Restrict production deployment configurations; enforce environment policy audits and configuration locking.

Table 7.2 — Comprehensive Risk Register (R-1 through R-9)

7.3.1 Risk Evaluation Heatmap Matrix

The qualitative positioning of each identified risk across Likelihood and Impact dimensions is visualized below:

Figure 7.2 — Risk Evaluation Heatmap Matrix

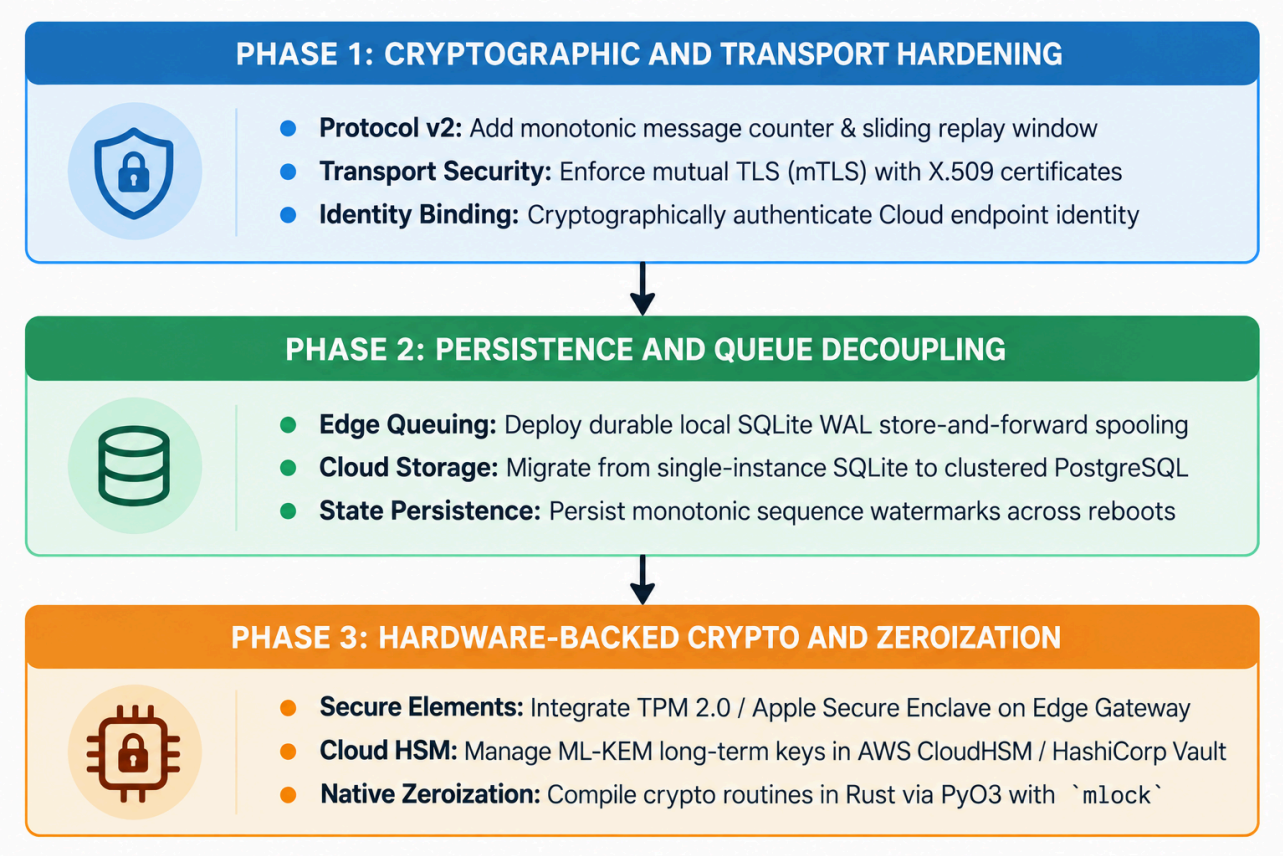


- **High-Impact / Context-Dependent Risks (R-1, R-2, R-9)**: Represent foundational security boundaries. While completely safe in the isolated test network, exposing these interfaces to the public Internet without transport-layer authentication or device authentication would create severe vulnerabilities.
- **Medium-Impact Operational Risks (R-4, R-5, R-6)**: Represent standard edge computing operational churn. Implemented error bounds prevent crashes and cascading failures, though temporary telemetry loss during extended outages is acknowledged.
- **Controlled High-Impact Risk (R-7)**: Prevented through defensive engineering and automated CI secret-scanning tests.

7.4 Production Modernization Roadmap

To transition the demonstrated edge-to-cloud post-quantum telemetry pipeline into an enterprise-scale, mission-critical production deployment, the engineering team has formulated a three-phase modernization roadmap:

Figure 7.3 — Production Modernization Roadmap



7.4.1 Phase 1: Cryptographic and Transport Hardening

1. **Protocol v2 Specification**: - Extend the `SecureEnvelope` and Associated Data (AAD) to include an explicit 64-bit monotonic sequence counter (`message_counter`). - Implement a 64-message sliding bitmask replay window in the Cloud AEAD decapsulation service to reject replayed ciphertexts at the cryptographic boundary before database invocation.
2. **Transport Layer Authentication**: - Wrap all HTTP communication in TLS 1.3. - Configure mutual TLS (mTLS) between Gateway and Cloud, utilizing X.509 certificate validation to eliminate public-key substitution and endpoint impersonation (R-1).

7.4.2 Phase 2: Persistence and Queue Decoupling

1. **Durable Edge Spooling (Store-and-Forward)**: - Introduce an embedded, disk-backed queue on the Edge Gateway. - If downstream Cloud transmission fails, telemetry is spooled to encrypted local storage and transmitted sequentially once connectivity is restored, resolving telemetry loss (R-4) without violating memory constraints.
2. **Clustered Cloud Database Architecture**: - Replace single-file SQLite with a clustered, multi-node PostgreSQL instance running TimescaleDB. - Implement connection pooling, read replicas, and automated point-in-time recovery, resolving concurrency and persistence bottlenecks (TD-8, R-8).

7.4.3 Phase 3: Hardware-Backed Key Isolation and Secure Zeroization

1. **Hardware Security Module (HSM) Integration**: - Deploy Cloud ML-KEM key generation and decapsulation within a FIPS 140-3 Level 3 Hardware Security Module (or cloud KMS). - Ensure Cloud private decapsulation keys can never be extracted into host RAM or process dumps.

2. **Native Memory Zeroization via Rust**: - Replace Python-level session key management with a compiled Rust extension module (using PyO3). - Leverage `secrecy` and `zeroize` crates to enforce guaranteed memory zeroization (`explicit_bzero`) and page locking (`mlock`), preventing sensitive key buffers from being written to swap memory (resolving TD-12 and R-7).

Chapter 8 — Appendices (Configurations, Runbooks and Key Source Code)

Appendix navigation

- Appendix A: Complete Deployment Configurations
- Appendix B: Complete CI/CD Pipeline Definitions
- Appendix C: System Operational Runbooks
- Appendix D: Core Cryptographic and Protocol Source Code
- Appendix E: Empirical Evaluation and Benchmarking Infrastructure
- Appendix F: Representative AI Prompt Catalog

Appendix A: Complete Deployment Configurations

A.1 Docker Compose Multi-Container Deployment Specification

The orchestration configuration coordinates the complete three-tier IoT architecture (`legacy-device`, `edge-gateway`, `cloud-service`) alongside an optional local monitoring dashboard. It enforces health check dependencies, non-root user isolation, service-scoped failure injection mappings, and persistent SQLite storage.

Code Listing 8.1 — A.1 Docker Compose Multi-Container Deployment Specification (yaml)

```yaml
services:
  cloud-service:
    build:
      context: .
      dockerfile: cloud_service/Dockerfile
    environment:
      HOST: 0.0.0.0
      PORT: 9000
      CLOUD_ID: ${CLOUD_ID:-cloud-001}
      DATABASE_URL: ${DATABASE_URL:-sqlite:////data/cloud.db}
      SECURITY_MODE: ${SECURITY_MODE:-classical}
      SESSION_TTL_SECONDS: ${SESSION_TTL_SECONDS:-300}
      FAILURE_MODE: ${CLOUD_FAILURE_MODE:-none}
      FAILURE_RATE: ${CLOUD_FAILURE_RATE:-0.0}
      FAILURE_SEED: ${CLOUD_FAILURE_SEED-}
      FAILURE_DELAY_SECONDS: ${CLOUD_FAILURE_DELAY_SECONDS:-2}
      LOG_LEVEL: ${LOG_LEVEL:-INFO}
    ports:
      - "127.0.0.1:${CLOUD_HOST_PORT:-9000}:9000"
    healthcheck:
      test:
        - CMD
        - python
        - -c
        - "from urllib.request import urlopen; urlopen('http://127.0.0.1:9000/health',
timeout=3).close()"
      interval: 30s
      timeout: 5s
      start_period: 15s
      retries: 3
    volumes:
      - cloud-data:/data

  edge-gateway:
    build:
      context: .
      dockerfile: edge_gateway/Dockerfile
    depends_on:
      cloud-service:
        condition: service_healthy
    environment:
      HOST: 0.0.0.0
      PORT: 8000
      GATEWAY_ID: ${GATEWAY_ID:-gateway-001}
      CLOUD_URL: ${CLOUD_URL:-http://cloud-service:9000}
      CLOUD_TIMEOUT_SECONDS: ${CLOUD_TIMEOUT_SECONDS:-3}
      CLOUD_MAX_RETRIES: ${CLOUD_MAX_RETRIES:-2}
      SECURITY_MODE: ${SECURITY_MODE:-classical}
      SESSION_TTL_SECONDS: ${SESSION_TTL_SECONDS:-300}
      POC_FAILURE_POLICY: fail_closed
      FAILURE_MODE: ${GATEWAY_FAILURE_MODE:-none}
      FAILURE_RATE: ${GATEWAY_FAILURE_RATE:-0.0}
      FAILURE_SEED: ${GATEWAY_FAILURE_SEED-}
      FAILURE_DELAY_SECONDS: ${GATEWAY_FAILURE_DELAY_SECONDS:-2}
      LOG_LEVEL: ${LOG_LEVEL:-INFO}
    ports:
      - "127.0.0.1:${GATEWAY_HOST_PORT:-8000}:8000"
    healthcheck:
      test:
        - CMD
        - python
        - -c
        - "from urllib.request import urlopen; urlopen('http://127.0.0.1:8000/health',
timeout=3).close()"
      interval: 30s
      timeout: 5s
      start_period: 15s
      retries: 3

  dashboard:
    build:
      context: .
      dockerfile: dashboard/Dockerfile
    depends_on:
      - cloud-service
      - edge-gateway
      - legacy-device
    environment:
      DASHBOARD_PORT: 8080
    ports:
      - "127.0.0.1:${DASHBOARD_HOST_PORT:-8080}:8080"
    volumes:
      # Local dashboard only: runtime overrides recreate an explicitly selected service.
      - /var/run/docker.sock:/var/run/docker.sock

  legacy-device:
    build:
      context: .
      dockerfile: legacy_system/Dockerfile
    depends_on:
      edge-gateway:
        condition: service_healthy
    environment:
      DEVICE_ID: ${DEVICE_ID:-sensor-001}
      GATEWAY_URL: ${GATEWAY_URL:-http://edge-gateway:8000}
```

```
      SEND_INTERVAL_SECONDS: ${SEND_INTERVAL_SECONDS:-5}
      REQUEST_TIMEOUT_SECONDS: ${REQUEST_TIMEOUT_SECONDS:-3}
      MAX_RETRIES: ${MAX_RETRIES:-2}
      FAILURE_MODE: ${LEGACY_FAILURE_MODE:-none}
      FAILURE_RATE: ${LEGACY_FAILURE_RATE:-0.0}
      FAILURE_SEED: ${LEGACY_FAILURE_SEED-}
      LOG_LEVEL: ${LOG_LEVEL:-INFO}

volumes:
  cloud-data:
```

A.2 Multi-Stage Container Image Specifications

A.2.1 Legacy Device Container Image

Code Listing 8.2 — A.2.1 Legacy Device Container Image (dockerfile)

```
# Build from the repository root: docker build -f legacy_system/Dockerfile .
FROM python:3.12-slim-bookworm

COPY --from=ghcr.io/astral-sh/uv:0.12.19 /uv /usr/local/bin/uv

WORKDIR /app
ENV UV_LINK_MODE=copy \
    UV_PROJECT_ENVIRONMENT=/app/.venv \
    PATH="/app/.venv/bin:${PATH}" \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

COPY pyproject.toml uv.lock README.md ./
COPY legacy_system/pyproject.toml legacy_system/pyproject.toml
COPY edge_gateway/pyproject.toml edge_gateway/pyproject.toml
COPY cloud_service/pyproject.toml cloud_service/pyproject.toml
COPY legacy_system/src legacy_system/src

RUN uv sync --frozen --package legacy-system --no-dev --no-editable

USER 10001:10001
STOPSIGNAL SIGTERM
CMD ["python", "-m", "legacy_system.main"]
```

A.2.2 Edge Gateway Container Image

Code Listing 8.3 — A.2.2 Edge Gateway Container Image (dockerfile)

```
# Build from the repository root: docker build -f edge_gateway/Dockerfile .
FROM python:3.12-slim-bookworm

COPY --from=ghcr.io/astral-sh/uv:0.12.19 /uv /usr/local/bin/uv

WORKDIR /app
ENV UV_LINK_MODE=copy \
    UV_PROJECT_ENVIRONMENT=/app/.venv \
    PATH="/app/.venv/bin:${PATH}" \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    HOST=0.0.0.0 \
    PORT=8000

COPY pyproject.toml uv.lock README.md ./
COPY legacy_system/pyproject.toml legacy_system/pyproject.toml
COPY edge_gateway/pyproject.toml edge_gateway/pyproject.toml
COPY cloud_service/pyproject.toml cloud_service/pyproject.toml
COPY edge_gateway/src edge_gateway/src

RUN uv sync --frozen --package edge-gateway --no-dev --no-editable

USER 10001:10001
EXPOSE 8000
STOPSIGNAL SIGTERM
HEALTHCHECK --interval=30s --timeout=5s --start-period=15s --retries=3 \
    CMD python -c "from urllib.request import urlopen; urlopen('http://127.0.0.1:8000/health', timeout=3).close()"
CMD ["edge-gateway"]
```

A.2.3 Cloud Service Container Image

Code Listing 8.4 — A.2.3 Cloud Service Container Image (dockerfile)

```
# Build from the repository root: docker build -f cloud_service/Dockerfile .
FROM python:3.12-slim-bookworm

COPY --from=ghcr.io/astral-sh/uv:0.12.19 /uv /usr/local/bin/uv

WORKDIR /app
ENV UV_LINK_MODE=copy \
    UV_PROJECT_ENVIRONMENT=/app/.venv \
    PATH="/app/.venv/bin:${PATH}" \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    HOST=0.0.0.0 \
    PORT=9000 \
    DATABASE_URL=sqlite:////data/cloud.db

COPY pyproject.toml uv.lock README.md ./
COPY legacy_system/pyproject.toml legacy_system/pyproject.toml
COPY edge_gateway/pyproject.toml edge_gateway/pyproject.toml
COPY cloud_service/pyproject.toml cloud_service/pyproject.toml
COPY cloud_service/src cloud_service/src

RUN uv sync --frozen --package cloud-service --no-dev --no-editable \
    && mkdir -p /data \
    && chown 10001:10001 /data

USER 10001:10001
EXPOSE 9000
STOPSIGNAL SIGTERM
HEALTHCHECK --interval=30s --timeout=5s --start-period=15s --retries=3 \
    CMD python -c "from urllib.request import urlopen; urlopen('http://127.0.0.1:9000/ready', timeout=3).close()"
CMD ["cloud-service"]
```

A.3 Environment Variable Configuration Template

Code Listing 8.5 — A.3 Environment Variable Configuration Template (ini)

```ini
# Shared Gateway/Cloud protocol defaults
# Allowed: classical, mlkem. Use mlkem only with PQC_FAILURE_POLICY=fail_closed.
SECURITY_MODE=classical
# Frozen protocol v1 value: 300 only.
SESSION_TTL_SECONDS=300

# Legacy Device
DEVICE_ID=sensor-001
GATEWAY_URL=http://edge-gateway:8000
SEND_INTERVAL_SECONDS=5
REQUEST_TIMEOUT_SECONDS=3
MAX_RETRIES=2
LEGACY_FAILURE_MODE=none
LEGACY_FAILURE_RATE=0.0
LEGACY_FAILURE_SEED=

# Edge Gateway
GATEWAY_ID=gateway-001
CLOUD_URL=http://cloud-service:9000
CLOUD_TIMEOUT_SECONDS=3
CLOUD_MAX_RETRIES=2
PQC_FAILURE_POLICY=fail_closed
GATEWAY_FAILURE_MODE=none
GATEWAY_FAILURE_RATE=0.0
GATEWAY_FAILURE_SEED=
GATEWAY_FAILURE_DELAY_SECONDS=2

# Cloud Service
CLOUD_ID=cloud-001
DATABASE_URL=sqlite:////data/cloud.db
GATEWAY_HOST_PORT=8000
CLOUD_HOST_PORT=9000
DASHBOARD_HOST_PORT=8080
CLOUD_FAILURE_MODE=none
CLOUD_FAILURE_RATE=0.0
CLOUD_FAILURE_SEED=
CLOUD_FAILURE_DELAY_SECONDS=2

LOG_LEVEL=INFO
```

Appendix B: Complete CI/CD Pipeline Definitions

B.1 Repository Quality and Multi-Stage CI Pipeline Workflow

Code Listing 8.6 — B.1 Repository Quality and Multi-Stage CI Pipeline Workflow (yaml)

```yaml
name: Repository quality

on: [push, pull_request]

permissions:
  contents: read

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - uses: astral-sh/setup-uv@v5
        with:
          enable-cache: true
      - run: uv sync --frozen
      - run: uv run ruff check .
      - name: Verify AI-history CLI compatibility
        run: |
          uv run ai-history --help
          uv run ai-history status
          uv run ai-history list
          uv run ai-history check

  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - uses: astral-sh/setup-uv@v5
        with:
          enable-cache: true
      - run: uv sync --frozen
      - name: Run unit and repository tests
        run: >-
          uv run pytest legacy_system/tests edge_gateway/tests cloud_service/tests
          tests/ai_history tests/test_*.py
      - name: Run protocol contract tests
        run: >-
          uv run pytest
          edge_gateway/tests/test_gateway_models.py
          edge_gateway/tests/test_gateway_canonical_model.py
          edge_gateway/tests/test_gateway_correlation.py
          edge_gateway/tests/test_public_key_client.py
          edge_gateway/tests/test_session.py
          edge_gateway/tests/test_secure_envelope.py
          edge_gateway/tests/test_aad.py
          edge_gateway/tests/test_kdf.py
          cloud_service/tests/test_gateway_cloud_baseline_contract.py
          cloud_service/tests/test_cloud_public_key.py
          cloud_service/tests/test_cloud_session_api.py
          cloud_service/tests/test_cloud_envelope.py
          cloud_service/tests/test_cloud_kdf.py
      - name: Run Gateway crypto checks
        run: >-
          uv run pytest
          edge_gateway/tests/test_mlkem.py
          edge_gateway/tests/test_kdf.py
          edge_gateway/tests/test_symmetric.py
          edge_gateway/tests/test_secure_envelope.py
          edge_gateway/tests/test_crypto_review_checklist.py
          edge_gateway/tests/test_transport_trust.py
          edge_gateway/tests/test_security_logging.py
          edge_gateway/tests/test_secure_transport.py
          tests/integration/test_secure_message_tamper.py
          tests/integration/test_gateway_cloud_secure.py

  integration-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - uses: astral-sh/setup-uv@v5
        with:
          enable-cache: true
      - run: uv sync --frozen
      - name: Run all integration tests
        run: uv run pytest tests/integration
      - name: Run observability and metrics sanity regressions
```

```
        run: >-
          uv run pytest
          tests/integration/test_observability_integration.py
          tests/integration/test_metrics_sanity.py
      - name: Run Cloud unit and cross-component checks
        run: >-
          uv run pytest
          cloud_service/tests
          tests/integration/test_gateway_cloud_secure.py
      - name: Run cross-component integration tests
        run: >-
          uv run pytest
          tests/integration/test_legacy_gateway.py
          tests/integration/test_gateway_cloud_baseline.py
          tests/integration/test_mlkem_session_orchestration.py
          tests/integration/test_gateway_cloud_secure_session.py
          tests/integration/test_gateway_cloud_secure.py

  secure-e2e:
    needs: [quality, unit-tests, integration-tests]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - uses: astral-sh/setup-uv@v5
        with:
          enable-cache: true
      - run: uv sync --frozen
      - name: Run full end-to-end tests
        run: >-
          uv run pytest
          tests/integration/test_end_to_end.py
          tests/integration/test_modernized_end_to_end.py
          tests/integration/test_legacy_mode_success_compatibility.py

  docker-build:
    needs: quality
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Legacy Device image
        run: docker build --file legacy_system/Dockerfile --tag course-legacy-device .
      - name: Build Edge Gateway image
        run: docker build --file edge_gateway/Dockerfile --tag course-edge-gateway .
      - name: Build Cloud Service image
        run: docker build --file cloud_service/Dockerfile --tag course-cloud-service .

  clean-environment:
    needs: [secure-e2e, docker-build]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - uses: astral-sh/setup-uv@v5
        with:
          enable-cache: true
      - name: Install from the committed lockfile
        run: uv sync --frozen
      - name: Validate the Compose configuration
        run: docker compose config --quiet
      - name: Run Compose recovery regression
        run: uv run pytest tests/integration/test_gateway_compose_e2e.py
```

Appendix C: System Operational Runbooks

C.1 Operational Lifecycle and Verification Commands

Code Listing 8.7 — C.1 Operational Lifecycle and Verification Commands (bash)

```bash
# 1. Clean build and configuration validation
docker compose config --quiet
docker compose build

# 2. Start Classical Baseline Stack
SECURITY_MODE=classical docker compose up --build --wait

# 3. Start Modernized ML-KEM Stack (Fail-Closed)
docker compose down
SECURITY_MODE=mlkem PQC_FAILURE_POLICY=fail_closed docker compose up --build --wait

# 4. Probe Health and Readiness
curl --fail http://127.0.0.1:8000/health
curl --fail http://127.0.0.1:8000/ready
curl --fail http://127.0.0.1:9000/health
curl --fail http://127.0.0.1:9000/ready

# 5. Query Metrics
curl --fail http://127.0.0.1:8000/metrics
curl --fail http://127.0.0.1:9000/metrics

# 6. Query Stored Measurements from Cloud
curl --fail http://127.0.0.1:9000/api/v1/measurements

# 7. Clean Teardown and SQLite Volume Reset
docker compose down --volumes --remove-orphans
```

Appendix D: Core Cryptographic and Protocol Source Code

D.1 Edge Gateway ML-KEM-768 Encapsulation Module

Code Listing 8.8 — D.1 Edge Gateway ML-KEM-768 Encapsulation Module (python)

```python
"""ML-KEM-768 encapsulation for the Gateway.

The Cloud public key is raw FIPS 203 bytes. Base64URL decoding belongs to
the protocol client, not this wrapper.
"""

from __future__ import annotations

from cryptography.exceptions import UnsupportedAlgorithm
from cryptography.hazmat.primitives.asymmetric.mlkem import MLKEM768PublicKey

PUBLIC_KEY_LENGTH = 1184
CIPHERTEXT_LENGTH = 1088
SHARED_SECRET_LENGTH = 32


class MlKemError(Exception):
    """The Gateway could not encapsulate to the supplied public key."""


def validate_public_key(public_key: bytes) -> None:
    """Accept only a raw ML-KEM-768 public key. Does not encapsulate or log it."""
    if not isinstance(public_key, bytes) or len(public_key) != PUBLIC_KEY_LENGTH:
        raise MlKemError("ML-KEM-768 public key must be 1184 bytes.")
    try:
        MLKEM768PublicKey.from_public_bytes(public_key)
    except (ValueError, UnsupportedAlgorithm) as exc:
        raise MlKemError("ML-KEM-768 public key was rejected.") from exc


def encapsulate(public_key: bytes) -> tuple[bytes, bytes]:
    """Return `(ciphertext, shared_secret)` for an ML-KEM-768 public key.

    The shared secret stays with the caller. This function does not log it.
    """
    validate_public_key(public_key)
    try:
        key = MLKEM768PublicKey.from_public_bytes(public_key)
        shared_secret, ciphertext = key.encapsulate()
    except (ValueError, UnsupportedAlgorithm) as exc:
        raise MlKemError("ML-KEM-768 encapsulation failed.") from exc
    if (
        len(shared_secret) != SHARED_SECRET_LENGTH
```

```
        or len(ciphertext) != CIPHERTEXT_LENGTH
    ):
        raise MlKemError("ML-KEM-768 encapsulation returned unexpected lengths.")
    return ciphertext, shared_secret
```

D.2 HKDF-SHA-256 Session Key Derivation Module

Code Listing 8.9 — D.2 HKDF-SHA-256 Session Key Derivation Module (python)

```python
"""Session-key derivation for protocol version 1.

Both Gateway and Cloud must call this contract. `salt=None` is the
`cryptography` HKDF default: RFC 5869 expands a missing salt to HashLen
zero bytes.
"""

from __future__ import annotations

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF

SESSION_KEY_LENGTH = 32
SESSION_KEY_INFO = b"ml-kem-project:v1:gateway-cloud-session"
SHARED_SECRET_LENGTH = 32


class KdfError(Exception):
    """The shared secret cannot be used for session-key derivation."""


def derive_session_key(shared_secret: bytes) -> bytes:
    """Derive the 32-byte AES-256-GCM session key. Does not log key material."""
    if not isinstance(shared_secret, bytes) or len(shared_secret) != SHARED_SECRET_LENGTH:
        raise KdfError("ML-KEM shared secret must be 32 bytes.")
    return HKDF(
        algorithm=hashes.SHA256(),
        length=SESSION_KEY_LENGTH,
        salt=None,
        info=SESSION_KEY_INFO,
    ).derive(shared_secret)
```

D.3 Edge Gateway AES-256-GCM Authenticated Encryption Module

Code Listing 8.10 — D.3 Edge Gateway AES-256-GCM Authenticated Encryption Module (python)

```
"""AES-256-GCM for protocol version 1.

`generate_nonce()` is only for a new logical encryption. An ordinary transport
retry resends the already-built envelope: the same nonce, ciphertext, and AAD
under the same session key. This module does not build that envelope.
"""

from __future__ import annotations

import secrets

from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

NONCE_LENGTH = 12
KEY_LENGTH = 32


class AeadError(Exception):
    """AES-256-GCM could not encrypt or authenticate the message."""


def generate_nonce() -> bytes:
    """Return a fresh 12-byte nonce from the operating-system CSPRNG."""
    return secrets.token_bytes(NONCE_LENGTH)


def encrypt(key: bytes, nonce: bytes, plaintext: bytes, associated_data: bytes) -> bytes:
    """Encrypt and authenticate. The result includes the GCM authentication tag."""
    key_bytes = _require_key(key)
    nonce_bytes = _require_nonce(nonce)
    plaintext_bytes = _require_bytes(plaintext, "Plaintext")
    aad_bytes = _require_bytes(associated_data, "Associated data")
    try:
        return AESGCM(key_bytes).encrypt(nonce_bytes, plaintext_bytes, aad_bytes)
    except ValueError as exc:
        raise AeadError("AES-256-GCM encryption failed.") from exc


def decrypt(key: bytes, nonce: bytes, ciphertext: bytes, associated_data: bytes) -> bytes:
    """Authenticate and decrypt. A failure does not return plaintext."""
    key_bytes = _require_key(key)
    nonce_bytes = _require_nonce(nonce)
    ciphertext_bytes = _require_bytes(ciphertext, "Ciphertext")
    aad_bytes = _require_bytes(associated_data, "Associated data")
    try:
```

```
        return AESGCM(key_bytes).decrypt(nonce_bytes, ciphertext_bytes, aad_bytes)
    except (InvalidTag, ValueError) as exc:
        raise AeadError("AES-256-GCM authentication failed.") from exc


def _require_key(key: object) -> bytes:
    if not isinstance(key, (bytes, bytearray)) or len(key) != KEY_LENGTH:
        raise AeadError("AES-256-GCM key must be 32 bytes.")
    return bytes(key)


def _require_nonce(nonce: object) -> bytes:
    if not isinstance(nonce, (bytes, bytearray)) or len(nonce) != NONCE_LENGTH:
        raise AeadError("AES-256-GCM nonce must be 12 bytes.")
    return bytes(nonce)


def _require_bytes(value: object, label: str) -> bytes:
    if not isinstance(value, (bytes, bytearray)):
        raise AeadError(f"{label} must be bytes.")
    return bytes(value)
```

D.4 Canonical 4-Field Associated Data Builder

Code Listing 8.11 — D.4 Canonical 4-Field Associated Data Builder (python)

```python
"""Four-field AES-GCM associated data for protocol version 1.

Gateway and Cloud must each produce these bytes. The object is exactly
`protocol_version`, `gateway_id`, `session_id`, and `request_id`.
"""

from __future__ import annotations

import json

PROTOCOL_VERSION = "1"


class AadError(Exception):
    """The AAD fields are not the frozen four-field object."""


def build_aad(
    *,
    protocol_version: str,
    gateway_id: str,
    session_id: str,
    request_id: str,
) -> bytes:
    """Return the canonical UTF-8 AAD object. Does not log the fields."""
    if not isinstance(protocol_version, str) or protocol_version != PROTOCOL_VERSION:
        raise AadError('protocol_version must be "1".')
    _require_id(gateway_id, "gateway_id")
    _require_id(session_id, "session_id")
    _require_id(request_id, "request_id")
    payload = {
        "protocol_version": protocol_version,
        "gateway_id": gateway_id,
        "session_id": session_id,
        "request_id": request_id,
    }
    return json.dumps(payload, sort_keys=True, separators=(",", ":")).encode("utf-8")


def _require_id(value: object, name: str) -> None:
    if not isinstance(value, str) or value == "" or value != value.strip():
        raise AadError(f"{name} must be a non-empty string.")
```

D.5 Edge Gateway Monotonic Sequence Tracker

Code Listing 8.12 — D.5 Edge Gateway Monotonic Sequence Tracker (python)

```python
"""In-memory Legacy sequence policy, independent of downstream delivery."""

import logging
from threading import Lock

from edge_gateway.application_log import log_application
from edge_gateway.errors import GatewayError, GatewayErrorCode
from edge_gateway.metrics import GatewayMetrics


class SequenceTracker:
    """Track the last valid sequence accepted for each device in one process.

    Share one instance across requests. Restarting loses history. Call only
    after schema/domain validation and before Cloud delivery; accepted state
    is never rolled back for a later delivery failure.

    The lock protects only the local check and commit. Cloud I/O belongs to
    the caller after this method returns and must not hold the sequence lock.
    Instances do not share state across processes or persist it to storage.
    """

    def __init__(self, metrics: GatewayMetrics | None = None) -> None:
        self._metrics = metrics or GatewayMetrics()
        self._last_sequence: dict[str, int] = {}
        self._lock = Lock()

    def check_and_update(self, device_id: str, sequence_number: int) -> None:
        """Atomically accept a validated sequence or raise its frozen 409 error.

        First values (including zero) and higher values are accepted. Gaps are
        logged. Rejections leave the previously accepted state unchanged.
        """
        with self._lock:
            previous = self._last_sequence.get(device_id)
            if previous is not None:
                if sequence_number == previous:
                    self._metrics.increment("duplicate_measurements")
                    self._metrics.increment("measurements_rejected")
                    log_application(
                        "sequence_duplicate_detected", logging.WARNING,
                        device_id=device_id, sequence_number=sequence_number,
                    )
                    raise GatewayError(GatewayErrorCode.DUPLICATE_SEQUENCE)
                if sequence_number < previous:
                    self._metrics.increment("out_of_order_measurements")
```

```
                    self._metrics.increment("measurements_rejected")
                    log_application(
                        "sequence_out_of_order_detected", logging.WARNING,
                        device_id=device_id, sequence_number=sequence_number,
                        previous_sequence_number=previous,
                    )
                    raise GatewayError(GatewayErrorCode.OUT_OF_ORDER_SEQUENCE)
            self._last_sequence[device_id] = sequence_number

        if previous is not None and sequence_number > previous + 1:
            self._metrics.increment("sequence_gaps")
            log_application(
                "sequence_gap_detected",
                device_id=device_id,
                sequence_number=sequence_number,
                previous_sequence_number=previous,
            )
```

D.6 Cloud Service ML-KEM-768 Decapsulation Module

Code Listing 8.13 — D.6 Cloud Service ML-KEM-768 Decapsulation Module (python)

```python
"""ML-KEM-768 key generation and decapsulation using PyCA cryptography.

Only this module depends on the vendor ML-KEM API. Private keys are opaque
handles, never serialized. No input, key or shared secret is logged.
"""

from cryptography.hazmat.primitives.asymmetric.mlkem import MLKEM768PrivateKey
from cryptography.hazmat.primitives.serialization import Encoding, PublicFormat

PUBLIC_KEY_LENGTH = 1184
CIPHERTEXT_LENGTH = 1088
SHARED_SECRET_LENGTH = 32


class MlKemError(Exception):
    """Safe error that contains no vendor diagnostics or input material."""


class PrivateKey:
    """Opaque process-local key handle with explicit reference release.

    Closing prevents further use. Python cannot guarantee memory zeroization.
    The default object repr deliberately includes no key fields.
    """

    __slots__ = ("_key",)

    def __init__(self, key: MLKEM768PrivateKey):
        self._key = key

    def close(self) -> None:
        self._key = None


def generate_keypair() -> tuple[bytes, PrivateKey]:
    """Return raw public bytes and an opaque private handle; use library RNG."""
    try:
        key = MLKEM768PrivateKey.generate()
        public_key = key.public_key().public_bytes(Encoding.Raw, PublicFormat.Raw)
        if not isinstance(public_key, bytes) or len(public_key) != PUBLIC_KEY_LENGTH:
            raise ValueError
    except Exception:
        # Vendor exceptions may contain sensitive diagnostics. Suppress chaining.
        raise MlKemError("ML-KEM-768 key generation failed") from None
    return public_key, PrivateKey(key)

```

```

def decapsulate(ciphertext: bytes, private_key: PrivateKey) -> bytes:
    """Return the raw 32-byte secret. Correct-length tampering may not raise."""
    if not isinstance(ciphertext, bytes) or len(ciphertext) != CIPHERTEXT_LENGTH:
        raise MlKemError("ML-KEM-768 ciphertext must be 1088 bytes")
    if not isinstance(private_key, PrivateKey) or private_key._key is None:
        raise MlKemError("ML-KEM-768 private key is unavailable")
    try:
        shared_secret = private_key._key.decapsulate(ciphertext)
        if not isinstance(shared_secret, bytes) or len(shared_secret) != SHARED_SECRET_LENGTH:
            raise ValueError
    except Exception:
        raise MlKemError("ML-KEM-768 decapsulation failed") from None
    return shared_secret
```

D.7 Cloud Service AEAD Authenticated Decryption Module

Code Listing 8.14 — D.7 Cloud Service AEAD Authenticated Decryption Module (python)

```
"""Authenticated decryption only; no parsing, persistence or plaintext logs."""

from cryptography.hazmat.primitives.ciphers.aead import AESGCM

KEY_LENGTH = 32
NONCE_LENGTH = 12
TAG_LENGTH = 16


class AeadError(Exception):
    """Safe secure-message rejection without vendor or secret diagnostics."""


def decrypt(key: bytes | bytearray, nonce: bytes | bytearray,
            ciphertext: bytes | bytearray, associated_data: bytes | bytearray) -> bytes:
    """Return plaintext only after library authentication succeeds."""
    if not isinstance(key, (bytes, bytearray)) or len(key) != KEY_LENGTH:
        raise AeadError("AES-256-GCM key must be 32 bytes")
    if not isinstance(nonce, (bytes, bytearray)) or len(nonce) != NONCE_LENGTH:
        raise AeadError("AES-256-GCM nonce must be 12 bytes")
    if not isinstance(ciphertext, (bytes, bytearray)) or len(ciphertext) < TAG_LENGTH:
        raise AeadError("AES-256-GCM ciphertext must include an authentication tag")
    if not isinstance(associated_data, (bytes, bytearray)):
        raise AeadError("Associated data must be bytes")
    try:
        plaintext = AESGCM(bytes(key)).decrypt(bytes(nonce), bytes(ciphertext),
bytes(associated_data))
        if not isinstance(plaintext, bytes):
            raise ValueError
    except Exception:
        raise AeadError("AES-256-GCM authentication failed") from None
    return plaintext
```

Appendix E: Empirical Evaluation and Benchmarking Infrastructure

E.1 Matched Mode Comparison Benchmarking Harness

Code Listing 8.15 — E.1 Matched Mode Comparison Benchmarking Harness (python)

```python
"""Run the equivalent classical and ML-KEM M5 evaluation workloads.

Example: ``uv run python scripts/measure_mode_comparison.py --samples 10``.
The output contains actual observations from that invocation; it intentionally
does not ship fixed timing values.
"""

from __future__ import annotations

import argparse
import importlib.util
import json
from pathlib import Path


def _load(name: str):
    path = Path(__file__).with_name(name)
    spec = importlib.util.spec_from_file_location(path.stem, path)
    if spec is None or spec.loader is None:
        raise RuntimeError(f"could not load {path.name}")
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module


def run(samples: int = 10) -> dict[str, object]:
    """Run one matched workload in each explicit security mode."""
    if type(samples) is not int or not 2 <= samples <= 100:
        raise ValueError("samples must be an integer from 2 to 100")
    baseline = _load("measure_baseline_e2e.py").run(samples)
    modernized = _load("measure_modernized_e2e.py").run(samples)
    baseline_first = baseline["first_message"]
    baseline_steady = baseline["steady_state"]
    modern_first = modernized["first_message"]
    modern_steady = modernized["steady_state"]
    if not all(isinstance(value, dict) for value in (baseline_first, baseline_steady, modern_first, modern_steady)):
        raise RuntimeError("matched workload did not produce first and steady-state observations")
    session = modernized["session_establishment"]
    if not isinstance(session, dict):
        raise RuntimeError("modernized report did not contain session metrics")
    baseline_successful = baseline["successful_messages"]
    modernized_successful = modernized["successful_messages"]
    if not isinstance(baseline_successful, int) or not isinstance(modernized_successful, int):
```

```
        raise RuntimeError("workload reports did not contain successful-message counts")
    return {
        "scope": "Matched local Legacy -> Gateway -> Cloud -> SQLite workloads",
        "samples": samples,
        "comparison": {
            "first_message_latency_ms": {
                "classical": baseline_first["legacy_gateway_end_to_end_ms"],
                "mlkem": modern_first["legacy_gateway_end_to_end_ms"],
            },
            "steady_state_latency_ms": {
                "classical": baseline_steady["legacy_gateway_end_to_end_ms"],
                "mlkem": modern_steady["legacy_gateway_end_to_end_ms"],
            },
            "gateway_to_cloud_payload_bytes": {
                "classical": baseline["canonical_payload_bytes"],
                "mlkem": modernized["secure_envelope_bytes"],
            },
            "session_setup": {"classical": "not_applicable", "mlkem": session},
            "successful_messages": {
                "classical": baseline_successful,
                "mlkem": modernized_successful,
            },
            "failed_messages": {
                "classical": samples - baseline_successful,
                "mlkem": samples - modernized_successful,
            },
            "storage_success": {
                "classical": baseline["persisted_rows"],
                "mlkem": modernized["persisted_rows"],
            },
        },
        "baseline_report": baseline,
        "modernized_report": modernized,
    }


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--samples", type=int, default=10)
    print(json.dumps(run(parser.parse_args().samples), indent=2, sort_keys=True))
```

Appendix F: Representative AI Prompt Catalog

This appendix catalogs representative AI prompts and human task specifications across the project lifecycle, documenting the exact scoping, constraints, and oversight mechanisms applied during implementation. In accordance with the project governance framework (Chapter 6), all tasks were initiated with bounded, deterministic prompts and subjected to human review prior to merging.

F.1 Governance, Tooling, and Architecture Infrastructure

Task 0001: AI History and Human Review Infrastructure

- **Agent / Subsystem**: OpenAI Codex / Governance Infrastructure
- **Scope & Context**: Initial repository bootstrapping. Establish the `ai-history` CLI, state journal, and review lifecycle to enforce the 4-phase human-in-the-loop workflow.
- **Human Prompt**:

  > "Implement repository-wide AI-development history, human-review, revision, and final-verification infrastructure without implementing application services. The user requested a Python/uv CLI, canonical instructions for Codex, Claude Code, GitHub Copilot, Cursor, Gemini, and other agents, Markdown task history, machine-readable active-task state, strict validation, automated tests, and a focused CI workflow. The supplied instruction requires complete AI work before human review, preservation of every review/revision round, separate final human verification, no fabricated review/check results, and no secrets or source/diff capture in history."

- **Review Outcome**: Accepted. Verified that the CLI prevents starting concurrent tasks and enforces read-only review states.

Task 0002: Workflow Handbook and Dependency Management Policy

- **Agent / Subsystem**: OpenAI Codex / Developer Tooling
- **Scope & Context**: Codify developer workflows, dependency pinning policies, and multi-agent instructions.
- **Human Prompt**:

  > "Implement the development workflow handbook, repository guidelines, and package dependency constraints. Enforce uv sync --frozen for deterministic dependency resolution and forbid pip/requirements.txt workflows."

- **Review Outcome**: Modified across 4 rounds. Human reviewers removed complex external CI tooling, enforced strict lockfile adherence, and added Group-Explanation course compliance checklists.

F.2 Legacy IoT Device Subsystem (Member 1)

Task 0008: Legacy Configuration Validation

- **Agent / Subsystem**: OpenAI Codex / Legacy Device
- **Scope & Context**: Environment variable loading and numerical validation for simulated IoT sensor parameters.
- **Human Prompt**:

  > "Implement configuration loading and validation for legacy_system using pydantic-settings or standard library. Parse DEVICE_ID, GATEWAY_URL, SEND_INTERVAL_SECONDS, REQUEST_TIMEOUT_SECONDS, and MAX_RETRIES. Add validation for all fields and unit tests."

- **Review Outcome**: Modified (2 rounds). Reviewers rejected permissive type coercion and enforced strict positive boundary checks on timing and retry bounds.

Task 0012: Legacy Canonical Measurement Schema

- **Agent / Subsystem**: OpenAI Codex / Legacy Device
- **Scope & Context**: Domain dataclass for sensor readings and strict JSON serialization.
- **Human Prompt**:

> "Implement the Legacy Measurement data model and JSON serialization according to the frozen protocol specification. Ensure field names and types match: device_id (string), sequence_number (integer), timestamp (ISO-8601 UTC string), temperature (float), humidity (float). Write focused unit tests."

- **Review Outcome**: Modified (2 rounds). Reviewers enforced strict ISO-8601 formatting with trailing 'Z' and rejected microsecond variations that could break wire compatibility.

Task 0027: Monotonic Sensor Simulation Engine

- **Agent / Subsystem**: OpenAI Codex / Legacy Device
- **Scope & Context**: Periodic sensor reading generation with monotonic sequence counters.
- **Human Prompt**:

> "Implement the Legacy Device sensor simulator loop in legacy_system/sensor.py. Generate realistic temperature and humidity values, incrementing sequence_number monotonically starting from 0. Provide deterministic seeding support."

- **Review Outcome**: Modified (2 rounds). Corrected initial sequence counter offset to ensure sequence numbers strictly begin at 0.

Task 0034: Sensor Failure Injection Subsystem

- **Agent / Subsystem**: GitHub Copilot / Legacy Device
- **Scope & Context**: Simulating wire faults (duplicate sequences, out-of-order sequences, invalid schemas, corruption).
- **Human Prompt**:

> "Implement task M1-04: Legacy Device failure-injection subsystem. Read and follow the repository policy, workflow, roadmap, Legacy Device design, shared protocol, Member 1 task document, relevant AI instructions, and both required Group-Explanation PDFs before implementation. Inspect existing code and tests, implement only M1-04, add focused deterministic tests, run relevant tests and Ruff plus ai-history check, record factual results, and stop for human review."

- **Review Outcome**: Modified across 4 rounds (detailed in Case Study 5, Section 6.4.5). Eliminated domain model mutation, enforced PRNG seed isolation, and ensured failure injection operates only on the outbound serialization dictionary.

Task 0052: Legacy Gateway HTTP Client and Error Domain Mapping

- **Agent / Subsystem**: OpenAI Codex / Legacy Device
- **Scope & Context**: Synchronous HTTP client transmitting telemetry to the Edge Gateway.
- **Human Prompt**:

> "Implement the Legacy HTTP client in legacy_system/client.py to send serialized JSON measurements to the Edge Gateway. Handle HTTP timeouts, network connection errors, and Gateway error status codes (400, 409, 422, 503)."

- **Review Outcome**: Modified (3 rounds). Reviewers caught unhandled raw HTTP exceptions escaping the client layer and enforced categorization into domain-specific exceptions (`GatewayConnectionError`, `GatewayResponseError`).

F.3 Edge Gateway Application and Sequence Logic (Member 2)

Task 0026: Gateway Configuration and Environment Parsing

- **Agent / Subsystem**: OpenAI Codex / Edge Gateway
- **Scope & Context**: Ingestion configuration for Edge Gateway service.
- **Human Prompt**:

> "Implement Edge Gateway configuration loading in edge_gateway/config.py. Support GATEWAY_ID, HOST, PORT, CLOUD_URL, CLOUD_TIMEOUT_SECONDS, CLOUD_MAX_RETRIES, SECURITY_MODE, and SESSION_TTL_SECONDS. Validate frozen defaults."

- **Review Outcome**: Accepted. Confirmed that `SECURITY_MODE` defaults to `classical` and `SESSION_TTL_SECONDS` is strictly validated to 300.

Task 0038: Legacy Measurement Ingestion Route and Validation

- **Agent / Subsystem**: OpenAI Codex / Edge Gateway
- **Scope & Context**: FastAPI endpoint `POST /api/v1/legacy/measurements`.
- **Human Prompt**:

> "Implement the legacy ingestion endpoint on the Edge Gateway. Validate inbound JSON syntax (400 on malformed syntax), schema and field types (422 on missing/invalid fields), and pass valid payloads to sequence validation."

- **Review Outcome**: Accepted. Enforced the frozen contract separating 400 (unparseable JSON) and 422 (schema/domain validation failure).

Task 0048: In-Memory Monotonic Sequence Tracking Policy

- **Agent / Subsystem**: OpenAI Codex / Edge Gateway
- **Scope & Context**: Per-device sequence tracking with duplicate and out-of-order rejection.
- **Human Prompt**:

> "Implement thread-safe in-memory sequence tracking for the Edge Gateway in edge_gateway/sequence.py. Reject duplicate sequence numbers with 409 DUPLICATE_SEQUENCE and lower sequence numbers with 409 OUT_OF_ORDER_SEQUENCE. Accept higher sequence numbers and log sequence gaps."

- **Review Outcome**: Accepted. Confirmed that sequence state is updated prior to Cloud delivery and never rolled back on downstream Cloud failures.

F.4 Edge Gateway Post-Quantum Cryptography and Transport (Member 3)

Task 0006: ML-KEM-768 and HKDF-SHA-256 Cryptographic Wrappers

- **Agent / Subsystem**: OpenAI Codex / Edge Gateway Cryptography
- **Scope & Context**: Core cryptographic primitive wrappers utilizing the maintained `cryptography` library.
- **Human Prompt**:

> "Start Member 3 at the current primitive phase: record the ML-KEM-768 library decision, add it to edge_gateway with uv, and implement the Gateway encapsulate wrapper plus the frozen HKDF-SHA-256 session-key wrapper with tests. Do not implement HTTP clients, sessions, AES-GCM, the Cloud service, or any protocol change."

- **Review Outcome**: Accepted. Confirmed that HKDF info string is frozen to `b"ml-kem-project:v1:gateway-cloud-session"`, salt is `None`, output length is 32 bytes, and no key material is logged.

Task 0011: Gateway In-Memory Secure Session Establishment

- **Agent / Subsystem**: OpenAI Codex / Edge Gateway Cryptography
- **Scope & Context**: Ephemeral session manager with concurrent handshake locking.
- **Human Prompt**:

  > "Member 3 tasks M3-09 through M3-13. Add an in-memory secure session and establish it with GET /api/v1/crypto/public-key, ML-KEM-768 encapsulation, the frozen HKDF call, and POST /api/v1/crypto/sessions. Use one async lock so concurrent callers share one handshake. Validate the session response and do not keep a session when establishment fails. Do not implement AES-GCM, secure measurement sends, measurement retries, or the Cloud service."

- **Review Outcome**: Accepted. Enforced single asyncio lock preventing handshake race conditions during session renewal.

Task 0019: AES-256-GCM Encryption and 4-Field Associated Data Builder

- **Agent / Subsystem**: OpenAI Codex / Edge Gateway Cryptography
- **Scope & Context**: Bulk AEAD encryption and canonical AAD formatting.
- **Human Prompt**:

  > "Implement AES-256-GCM encryption with 12-byte CSPRNG nonce generation in edge_gateway/crypto/symmetric.py. Implement the canonical 4-field Associated Data (AAD) builder according to Protocol v1: protocol_version, gateway_id, session_id, request_id. Enforce sorted UTF-8 compact JSON encoding."

- **Review Outcome**: Accepted. Verified that AAD contains exactly the four frozen fields and rejects extraneous metadata.

Task 0098: Secure Transport Retries and Session Recovery

- **Agent / Subsystem**: OpenAI Codex / Edge Gateway Cryptography
- **Scope & Context**: Transport resilience, distinguishing ordinary network retries from session invalidation.
- **Human Prompt**:

  > "Implement bounded retry logic with exponential backoff for the Edge Gateway secure transport client. Handle transient network errors and HTTP timeout exceptions when sending telemetry envelopes to the Cloud Service."

- **Review Outcome**: Modified (detailed in Case Study 3, Section 6.4.3). Enforced that ordinary transport retries resend the identical SecureEnvelope without re-encrypting, while 409 session errors trigger session invalidation and re-handshake.

F.5 Cloud Service Telemetry Ingestion and Storage (Member 4)

Task 0018: SQLite Storage Layer and Business Idempotency

- **Agent / Subsystem**: OpenAI Codex / Cloud Service
- **Scope & Context**: SQLite database initialization, schema migration, and measurement insertion repository.
- **Human Prompt**:

> "Complete all remaining Member 4 work for PR 2: M4-05 through M4-07, as one AI-history task. Implement and test the complete remaining PR 2 storage scope, then stop once for human review. Do not start M4-08 or any later task. Do not modify shared/project-wide files other than required AI-history files without my approval."

- **Review Outcome**: Accepted. Enforced `UNIQUE(device_id, sequence_number)` database index and verified that duplicate submissions return `202 Accepted` without creating duplicate database records.

Task 0041: Cloud ML-KEM Key Lifecycle and Public-Key Route

- **Agent / Subsystem**: OpenAI Codex / Cloud Service
- **Scope & Context**: In-memory ML-KEM-768 keypair generation and `/api/v1/crypto/public-key` endpoint.
- **Human Prompt**:

> "Complete M4-16 through M4-20 as one AI-history task. Implement Cloud ML-KEM-768 lifecycle, compatible maintained-library wrapper, tests and public-key endpoint. No M4-21 or later. Ask before changing edge_gateway or shared/project-wide files. Stop for human review."

- **Review Outcome**: Modified (2 rounds). Reviewers corrected Base64URL encoding rules and purged extraneous key confirmation fields proposed by the model.

Task 0072: Cloud Secure Envelope Decryption and Canonical Ingestion

- **Agent / Subsystem**: OpenAI Codex / Cloud Service
- **Scope & Context**: Decrypting `SecureEnvelope` at `POST /api/v1/secure/measurements`.
- **Human Prompt**:

> "Implement Cloud SecureEnvelope ingestion endpoint. Lookup active session by session_id, decrypt AES-256-GCM ciphertext using derived session key and reconstructed 4-field AAD. Validate decrypted measurement schema and persist to database."

- **Review Outcome**: Accepted. Verified that AEAD decryption failures return `401 Unauthorized` or `422 Unprocessable Entity` without disclosing cryptographic diagnostics or secret buffers.

F.6 Operations, Integration, and Performance Evaluation (Member 5)

Task 0080: Gateway-to-Cloud Cross-Service Integration Tests

- **Agent / Subsystem**: OpenAI Codex / Integration Testing
- **Scope & Context**: Automated pytest suite validating full handshake and payload delivery between Gateway and Cloud.
- **Human Prompt**:

> "Implement cross-component integration test suite for Edge Gateway and Cloud Service in tests/integration/test_gateway_cloud_secure.py. Test full session establishment, valid telemetry encryption and persistence, expired session recovery, and corrupted ciphertext rejection."

- **Review Outcome**: Accepted. Verified test isolation using ephemeral test databases and mock server fixtures.

Task 0185: Complete Three-Tier Docker Compose Topology

- **Agent / Subsystem**: OpenAI Codex / Deployment & Operations
- **Scope & Context**: Multi-container Docker Compose file and healthcheck orchestration.
- **Human Prompt**:

> "Restore the complete three-tier containerized topology in docker-compose.yml including legacy-device, edge-gateway, and cloud-service. Ensure healthcheck dependency chains, non-root user execution, and persistent SQLite storage volume."

- **Review Outcome**: Accepted (reversing premature omission from Task AD-7). Confirmed that the legacy sensor automatically waits for Gateway health before starting transmissions.

Task 0211: Matched-Mode Empirical Evaluation Harness

- **Agent / Subsystem**: OpenAI Codex / Empirical Evaluation
- **Scope & Context**: Benchmarking script executing matched workloads across classical and ML-KEM modes.
- **Human Prompt**:

> "Implement the matched-mode evaluation script scripts/measure_mode_comparison.py to execute paired workloads through the complete 3-tier pipeline in classical and ML-KEM modes. Measure first-message latency, steady-state latency, wire payload bytes, and storage success. Output clean JSON."

- **Review Outcome**: Accepted. Verified that the script captures real empirical measurements without hardcoding timing constants.

Task 0219: Protocol Security Mode Sequence Diagrams

- **Agent / Subsystem**: OpenAI Codex / Architecture Documentation
- **Scope & Context**: Generating detailed Mermaid protocol sequence diagrams for Classical Baseline and Modernized Post-Quantum modes.
- **Human Prompt**:

> "Create formal Mermaid sequence diagrams illustrating the end-to-end message flow for classical mode and ML-KEM modernized mode. Show transport protocols, exact status codes, payload structures, session caching, and error paths."

- **Review Outcome**: Accepted. Verified that diagrams strictly reflect frozen Protocol v1 contracts with no unauthenticated confirmations or extra parameters.